华为认证鲲鹏应用开发者系列教程

HCIA-Kunpeng Application Developer

学员用书

版本：2.0



华为技术有限公司

版权所有 © 华为技术有限公司 2023。 保留一切权利。

非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

商标声明



和其他华为商标均为华为技术有限公司的商标。

本文档提及的其他所有商标或注册商标，由各自的所有人拥有。

注意

您购买的产品、服务或特性等应受华为公司商业合同和条款的约束，本文档中描述的全部或部分产品、服务或特性可能不在您的购买或使用范围之内。除非合同另有约定，华为公司对本文档内容不做任何明示或暗示的声明或保证。

由于产品版本升级或其他原因，本文档内容会不定期进行更新。除非另有约定，本文档仅作为使用指导，本文档中的所有陈述、信息和建议不构成任何明示或暗示的担保。

华为技术有限公司

地址：　深圳市龙岗区坂田华为总部办公楼　邮编：518129

网址：　https://e.huawei.com

华为认证体系介绍

华为认证是华为公司基于“平台+生态”战略，围绕“云-管-端”协同的新ICT技术架构，打造的覆盖**ICT**（Information and Communications Technology，信息通信技术）全技术领域的认证体系，包含**ICT**基础设施认证、基础软硬件认证、云平台及云服务认证三类认证。

根据ICT从业者的学习和进阶需求，华为认证分为工程师级别、高级工程师级别和专家级别三个认证等级。

华为认证覆盖ICT全领域，符合ICT融合的技术趋势，致力于提供领先的人才培养体系和认证标准，培养数字化时代新型ICT人才，构建良性ICT人才生态。

HCIA-Kunpeng Application Developer（Huawei Certified ICT Associate - Kunpeng Application Developer ，华为认证鲲鹏应用开发者工程师）定位于培养与认证能够应用华为鲲鹏计算平台进行业务部署与迁移，性能测试与调优，常见解决方案设计与规划的工程师。

通过HCIA-Kunpeng Application Developer V2.0认证，您将掌握对鲲鹏计算平台的使用与维护方法，具备对鲲鹏计算平台上的应用进行全生命周期管理的能力，能够胜任鲲鹏平台的应用开发和运维岗位。

华为职业认证

云平台及云服务	云平台及云服务领域	Cloud Computing	Cloud Service	Big Data	GaussDB	IoT
基础软硬件	基础软硬件领域	openEuler	openGauss	HarmonyOS	AI	Kunpeng
ICT基础设施	IT基础设施领域	Storage	Collaboration	Intelligent Vision	Digital Power	MDC
	数通领域	Datacom	WLAN	Security		
	光领域	Transmission	Access			
	无线领域	5G	LTE			



Huawei Certified ICT Expert



Huawei Certified ICT Professional



Huawei Certified ICT Associate

前　言

简介

本书为 HCIA-Kunpeng Application Developer 认证培训学员用书，适用于准备参加 HCIA-Kunpeng Application Developer 考试的学员或者希望了解有关于鲲鹏的相关生态，应用迁移原理，迁移工具使用方法，鲲鹏应用调优方法，华为鲲鹏解决方案，鲲鹏社区等技术知识的读者。

内容描述

本学员用书共包含 6 个章节，从鲲鹏体系介绍开始，逐一介绍了鲲鹏生态体系、鲲鹏应用移植、应用性能测试与调优、鲲鹏即时交流平台综合实验、华为鲲鹏解决方案以及鲲鹏社区的相关内容。

- 章节一为鲲鹏体系介绍，具体包括鲲鹏生态体系介绍、华为鲲鹏处理器介绍、Taishan 服务器介绍、openEuler 操作系统介绍、openGauss 数据库介绍。
- 章节二为鲲鹏应用移植，具体包括软件迁移原理与迁移过程、鲲鹏迁移指导、容器迁移指导、迁移常见问题及解决思路、鲲鹏开发者支持套件。
- 章节三为应用性能测试与调优，具体包括软件性能测试、鲲鹏通用性能调优方法、鲲鹏性能分析工具。
- 章节四为鲲鹏即时交流平台综合实验，具体包括鲲鹏即时交流平台介绍、基础实验环境部署、鲲鹏代码迁移、鲲鹏即时交流平台搭建，以及基于 Hive 的数据处理。
- 章节五为华为鲲鹏解决方案，具体包括华为鲲鹏解决方案全景，以及华为鲲鹏通用解决方案。
- 章节六为鲲鹏社区各版块介绍，具体包括鲲鹏社区的定位及作用、鲲鹏社区各版块介绍、鲲鹏开发套件 Devkit、鲲鹏应用使能套件 Boostkit、鲲鹏开发者学习专栏、鲲鹏生态伙伴网络及伙伴计划、支持与资讯。

目录

前　言 3
简介 3
内容描述 3
1 鲲鹏体系介绍 8
1.1 鲲鹏生态体系介绍 8
1.1.1 计算产业发展趋势 8
1.1.2 鲲鹏简介 11
1.1.3 鲲鹏计算产业概述 11
1.1.4 鲲鹏生态体系介绍 15
1.2 华为鲲鹏处理器介绍 18
1.2.1 处理器架构介绍 18
1.2.2 华为鲲鹏处理器概述 19
1.2.3 华为鲲鹏处理器型号及规格 19
1.2.4 华为鲲鹏处理器技术创新 20
1.3 Taishan 服务器介绍 20
1.3.1 服务器通识类知识简介 20
1.3.2 Taishan 服务器概述 21
1.3.3 Taishan 200 机架服务器介绍 22
1.4 openEuler 操作系统介绍 22
1.4.1 Linux 操作系统介绍 22
1.4.2 openEuler 操作系统介绍 24
1.4.3 嵌入式 OS 介绍 26
1.5 openGauss 数据库介绍 26
1.5.1 数据库概述 26
1.5.2 openGauss 数据库介绍 27
1.5.3 应用开发简介 28
2 鲲鹏应用移植 29
2.1 软件迁移原理与迁移过程 29
2.1.1 软件迁移原理 29
2.1.2 软件迁移过程 32
2.1.3 软件迁移典型案例 33

2.2 鲲鹏迁移指导 33
2.2.1 鲲鹏代码迁移工具介绍 33
2.2.2 编译型语言应用迁移 34
2.2.3 解释型语言应用迁移 34
2.2.4 二进制指令翻译工具 ExaGear 34
2.3 容器迁移指导 35
2.3.1 容器相关概念介绍 35
2.3.2 iSula 容器引擎介绍 35
2.3.3 容器迁移背景和原理 36
2.3.4 容器迁移主要流程 36
2.4 迁移常见问题及解决思路 36
2.4.1 常见编译问题 36
2.4.2 常见功能问题 36
2.4.3 常见工具问题 37
2.5 鲲鹏开发者支持套件 37
2.5.1 鲲鹏开发套件 DevKit 37
2.5.2 鲲鹏应用使能套件 BoostKit 38
3 应用性能测试与调优 39
3.1 软件性能测试 39
3.1.1 性能测试方法与工具 39
3.1.2 常用软件性能测试案例 41
3.2 鲲鹏通用性能调优方法 45
3.2.1 性能调优 45
3.2.2 CPU 和内存子系统性能调优 45
3.2.3 网络子系统性能调优 48
3.2.4 磁盘 IO 子系统调优 51
3.2.5 应用程序调优 55
3.3 鲲鹏性能分析工具 60
3.3.1 工具介绍 60
3.3.2 调优助手 62
3.3.3 系统性能分析 64
3.3.4 Java 性能分析 67
3.3.5 系统诊断 69
4 鲲鹏即时交流平台综合实验 71
4.1 鲲鹏即时交流平台介绍 71

4.1.1 平台功能简介及实现方法 71
4.1.2 MySQL 介绍 75
4.1.3 Nginx 介绍 76
4.2 基础实验环境部署 77
4.2.1 主机环境选择 77
4.2.2 基础环境部署 78
4.2.3 MySQL 部署步骤 78
4.2.4 Nginx 部署步骤 78
4.3 鲲鹏代码迁移 78
4.3.1 JNA 简介 78
4.3.2 任务流程 79
4.4 鲲鹏即时交流平台搭建 80
4.5 基于 Hive 的数据处理 80
4.5.1 大数据简介 80
4.5.2 性能调优思路 86
4.5.3 实验流程介绍 88
5 华为鲲鹏解决方案 90
5.1 华为鲲鹏解决方案全景 90
5.1.1 华为鲲鹏解决方案全景 90
5.1.2 华为鲲鹏云全栈解决方案竞争力 91
5.1.3 Taishan 服务器覆盖主流应用场景 91
5.2 华为鲲鹏通用解决方案 92
5.2.1 高性能计算 HPC 解决方案 92
5.2.2 大数据基础设施解决方案 96
5.2.3 鲲鹏云手机解决方案 102
6 鲲鹏社区 107
6.1 鲲鹏社区的定位及作用 107
6.1.1 鲲鹏社区的定位 107
6.1.2 鲲鹏社区的作用 107
6.2 鲲鹏社区各版块介绍 107
6.2.1 产品板块 107
6.2.2 解决方案板块 108
6.2.3 开发者板块 108
6.2.4 合作伙伴板块 108
6.2.5 高校板块 108

6.2.6 鲲鹏解决方案市场板块 109
6.2.7 产业资讯板块 109
6.2.8 支持与服务板块 109
6.3 鲲鹏开发者 - 鲲鹏开发套件 DevKit 109
6.3.1 鲲鹏代码迁移工具 110
6.3.2 鲲鹏开发框架 110
6.3.3 鲲鹏编译工具链 110
6.3.4 云测试服务 110
6.3.5 鲲鹏性能分析工具 110
6.3.6 华为动态二进制翻译工具（ExaGear） 110
6.4 鲲鹏开发者 - 鲲鹏应用使能套件 BoostKit 110
6.4.1 大数据场景 111
6.4.2 分布式存储场景 111
6.4.3 数据库场景 111
6.4.4 虚拟化场景 111
6.4.5 ARM 原生场景 111
6.4.6 Web 应用场景 112
6.4.7 CDN 112
6.4.8 HPC 高性能计算场景 112
6.5 鲲鹏开发者 - 学习专栏 112
6.5.1 鲲鹏在线课程 112
6.5.2 鲲鹏在线实验 112
6.5.3 鲲鹏认证体系 113
6.6 合作伙伴 - 鲲鹏生态伙伴网络及伙伴计划 113
6.6.1 鲲鹏生态伙伴 113
6.6.2 鲲鹏生态伙伴网络的优势 113
6.6.3 鲲鹏生态伙伴类型 114
6.6.4 鲲鹏生态伙伴网络成员级别 114
6.7 支持与资讯 114

1 鲲鹏体系介绍

1.1 鲲鹏生态体系介绍

1.1.1 计算产业发展趋势

1.1.1.1 大数据技术对算力的挑战

计算产业是 IT 技术的基础，是每一次产业变革的驱动力，从云计算、大数据、人工智能到区块链、边缘计算、物联网等都离不开强大的计算能力的支持。

现在主要围绕大数据三个阶段，具体看一下大数据技术对算力的挑战情况：

大数据 1.0 时代：此阶段的主要需求驱动是随着互联网的发展而发展，相比传统的企业电子办公，出现了大量的非结构化数据，因此对数据的分布式存储和并行技术提出了很高的要求。Google 就是传统互联网的典型代表，发表了 GFS、MR、BigTable 等重要的大数据论文，从而引导了大数据的元年。

大数据 2.0 时代：随着移动互联网的蓬勃发展，在互联网上的应用远远比传统互联网（PC 访问）更加丰富，数据的结构也发生了很大的变化，对实时处理的能力和大量并发的要求相较以前会有巨大的提高。手机银行就是一个典型的代表，现在的手机银行 APP 能够提供各种丰富的内容，一方面除了传统的网上交易，还包括资讯，娱乐，购物等。因此对数据处理的能力通常就要求多种技术配合来完成，如批处理，内存计算，交互式查询，流式实时处理等，这样就推动了 MR、SPARK、Solr、Storm 和 YARN 的发展。

大数据 3.0 时代：随着物联网的发展，未来将是一个完全的数字化的世界，任何东西都将连接到网络上，而最关键的是要将这些东西关联起来，发现其背后存在的规律，从而为人类的生活、生产创造更大的价值，这就是人工智能时代的大数据处理。

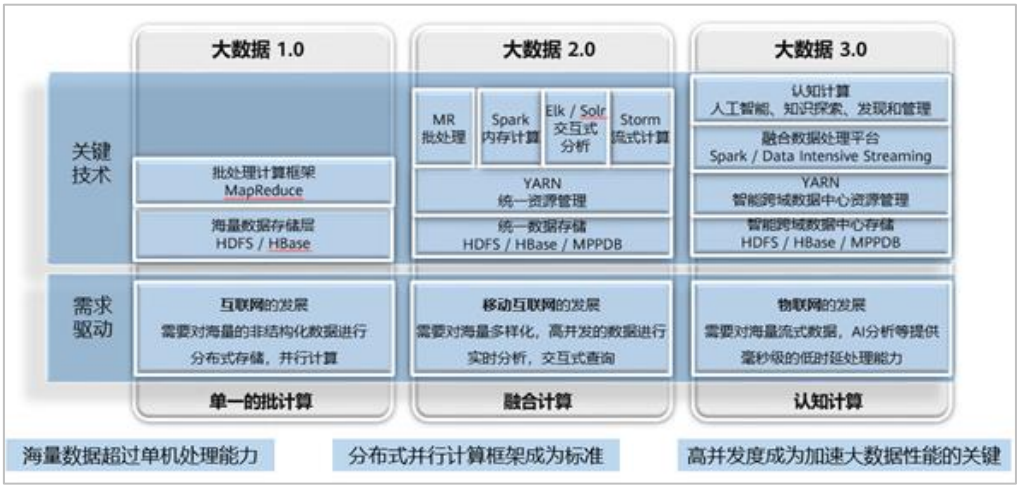


图1-1 大数据发展的三个阶段

1.1.1.2 人工智能对算力的挑战

目前的人工智能，更多的是代表智能的个体，特指能够通过自身的持续学习能力，智能的完成单点决策。

AI 产业爆发三大条件：**算法、算力、数据**。

机器经验需要由大量的历史数据得出，所以数据收集无处不在，数据的增长会是几何级数的。整体来看，当前无论是存储还是通信模式都是集中式模式，因此在未来是无论如何都无法通过一个巨型单点支撑如此大的体量，可以预见，存储和通信能力都会成为瓶颈。

算力成本也是人工智能行业的一大痛点。现在的人工智能企业的硬件投入巨大，人工智能对计算的需求非常大，因此对高性能计算定制深度学习芯片要求很高，意味着很多企业要花很多钱买算力、建很多计算中心，造成了很大的资源浪费。

AI 计算发展趋势演变过程中也面临着巨大的挑战：随着模型所需的精度越高，所需的计算量也会呈现增长趋势。未来算法的发展对整个计算需求所造成的挑战会变得更大，提高整个 AI 计算系统的性能与效率显得尤为重要。

此外摩尔定律的失效，CPU 性能的提升也遭遇瓶颈。Intel 宣布正式停用“Tick-Tock”处理器研发模式，未来研发周期将从两年周期向三年期转变。单颗 CPU 性能的提升在放缓，传统服务器难以满足并行算力需求，服务器 CPU 出货量增长停滞。

整体来看，AI 面临着巨大的计算挑战，提高 AI 计算系统性能与效率变得尤为重要，需要从系统的角度进行综合考虑。

1.1.1.3 应用和数据的多样性需要新的计算架构

未来信息量巨大，计算无处不在，计算应用的场景多种多样，从扫地机器人到智能手机、智慧家庭、IoT 物联网、自动驾驶等。场景的多样性，带来数据的多样性，数字、文本、图片、视频、图像以及结构化数据、非结构化数据等。

整型计算适合文本处理、存储、大数据等场景；浮点计算适合科学计算、视频处理等场景。

1.1.1.4 新兴计算产业呼唤新的算力

当前计算产业呈现两个大的变化趋势：

1. 移动智能终端取代传统 PC：2018 年全球传统 PC 出货量 2.5 亿台，连续七年下滑。与之相反，2018 年全球移动智能终端出货量突破 16 亿部。这导致计算正在从 x86->ARM；应用正从 PC 应用->移动应用->移动应用云化。
2. 世界正在进入万物互联的时代：2018 年全球联接设备的数量已超过 230 亿；到 2025 年预计这一数字将突破 1000 亿，将带来海量的数据。例如：自动驾驶每天产生 64TB 数据、XX 平安城市每天 1000TB 数据。

对于海量数据的处理的算力需求：

1. 边缘侧实时智能处理，需要 AI 的算力；
2. 数据中心侧分析、处理和存储海量的数据，需要高并发、高性能、高吞吐的算力。

多样性计算满足算力需求

诚如刚刚所述，计算进入多样性时代，将带来大量异构计算的需求，没有一个单一的计算架构能够满足所有场景、所有数据类型的处理。比如我们看到各种 CPU、DSP、GPU、AI 芯片、FPGA 等同时存在，主要此种结合有如下优势：

1. 多种计算架构共存的异构计算是最优的解决路径；
2. 多种计算架构共存的异构计算，将实现摩尔定律的重构，从而更好满足多业务多样性、数据多样性需求。

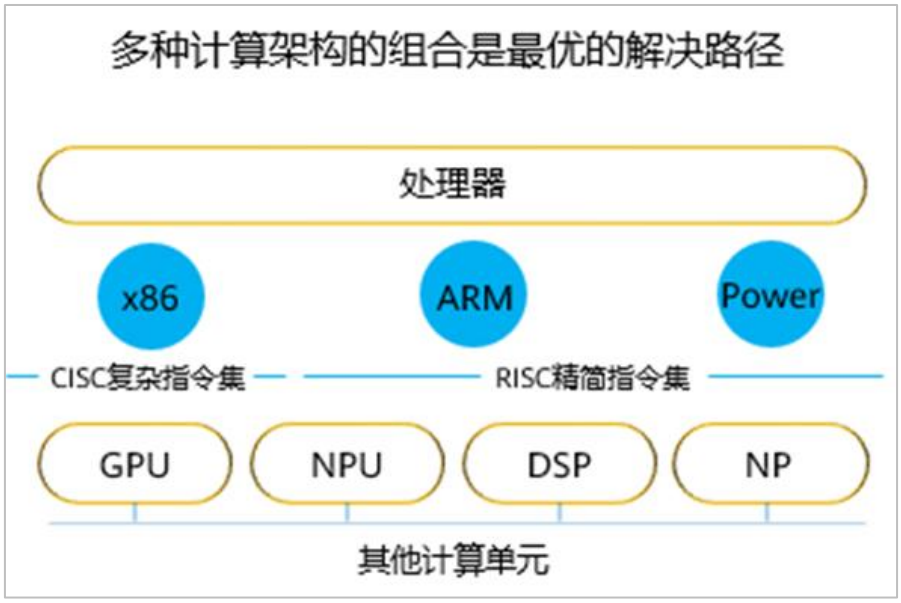


图1-2 多样化算力

华为主打鲲鹏+昇腾结合，打造核心算力。

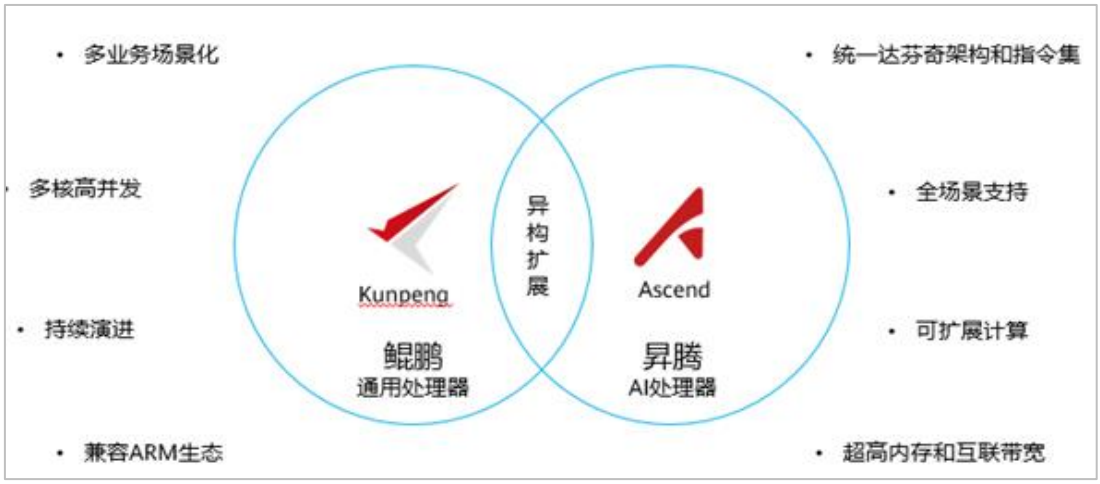


图1-3 鲲鹏+昇腾

1.1.2 鲲鹏简介

1.1.2.1 什么是鲲鹏

在上一节中我们说到，华为主打鲲鹏+昇腾结合，提高算力，那什么是鲲鹏呢？接下来可以如下这张图直观看一下：

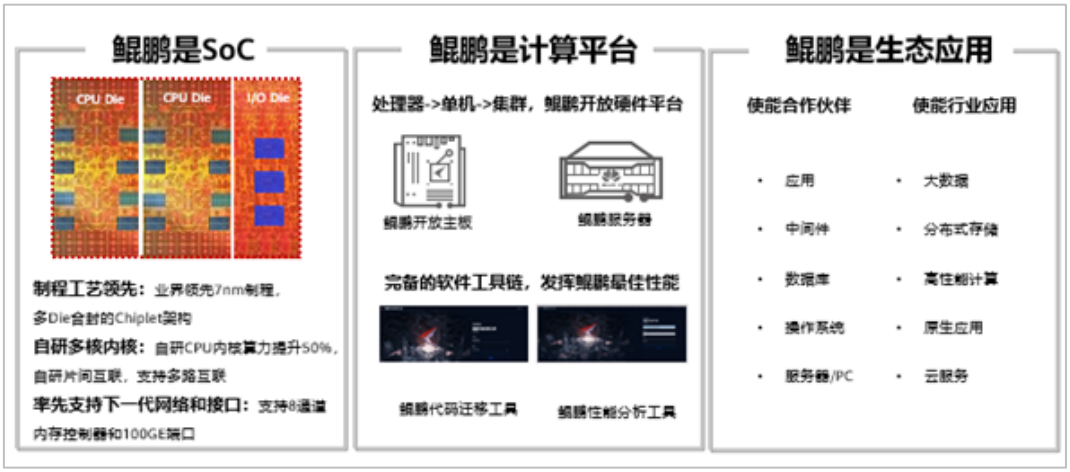


图1-4 什么是鲲鹏

可以看出，鲲鹏不仅是芯片，还是一个计算平台，更可以是一个生态应用。

1.1.3 鲲鹏计算产业概述

1.1.3.1 超万亿规模的计算产业空间

新应用、新技术、新计算架构，百亿级联接、爆炸式数据增长将重塑 ICT 产业新格局，催生新的计算产业链条，涌现出新的厂家和新的生态体系：

- 软件：操作系统和虚拟化软件、数据库、中间件、大数据平台、企业应用软件，云服务、数据中心管理服务；
- 硬件：服务器及部件、企业存储设备；
- 云服务、数据中心管理服务。



图1-5 计算产业规模

1.1.3.2 鲲鹏计算产业概述

鲲鹏计算产业是基于鲲鹏处理器构建的全栈 IT 基础设施、行业应用及服务，包括 PC、服务器、存储、操作系统、中间件、虚拟化、数据库、云服务、行业应用以及咨询管理服务等。



图1-6 鲲鹏计算产业

1.1.3.3 鲲鹏处理器 - 坚持持续创新

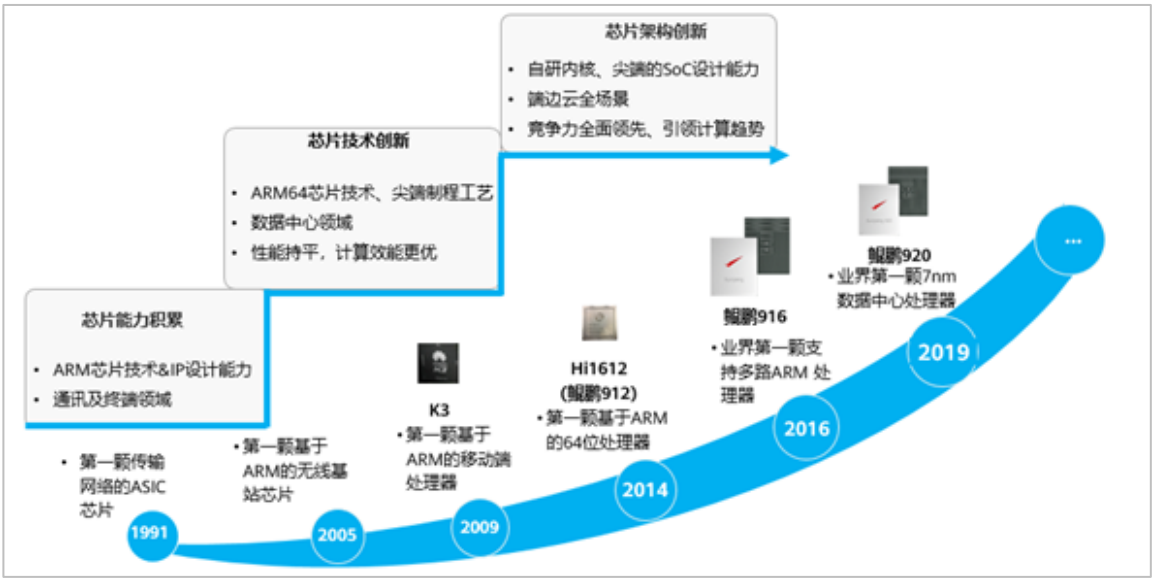


图1-7 鲲鹏处理器演进

华为在芯片及处理器领域的探索已经超过了二十年。

1991 年，研发了第一颗用于传输网络的 ASIC 芯片。

2005 年，研发了第一颗基于 ARM 架构的无线基站芯片。

2009 年，推出智能手机 CPU K3，麒麟芯片的前身，如今麒麟芯片已经把手机带入智慧时代。

2016 年，推出第一代鲲鹏处理器，面向数据中心应用的鲲鹏 916 处理器。推出服务器 CPU，也就是鲲鹏芯片的前身。

2019 年，推出鲲鹏 920，是业界第一颗 7nm 数据中心处理器。

华为每年将销售收入的 10%以上投入研发，其中芯片/SoC（System on a Chip，系统芯片）相关投资超过 1/3，使得华为的芯片研发能力实现了从初始技术积累、到技术创新，再到架构创新的飞跃式提升。

未来，鲲鹏处理器还将保持长期投入、全面布局、前向兼容、持续演进的产品策略，量产一代、研发一代、规划一代。

1.1.3.4 全面创“芯”- 构建全自研服务器 TaiShan

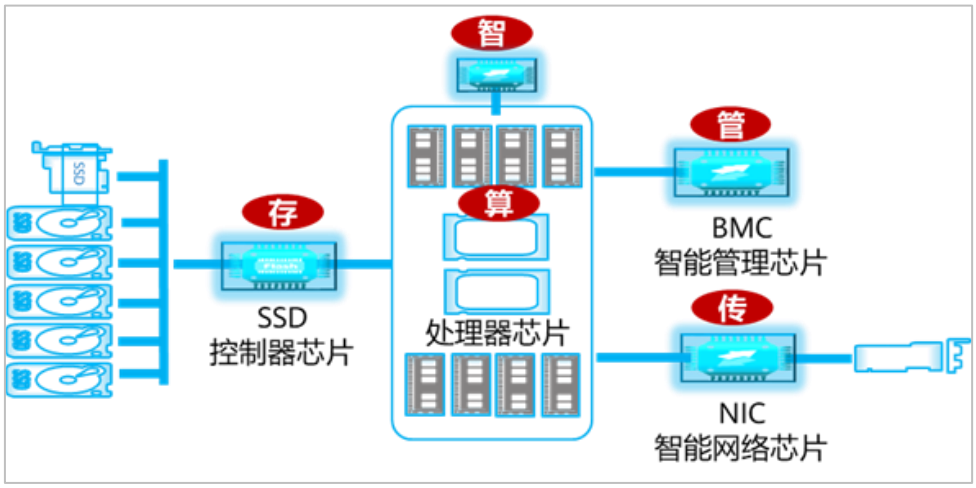


图1-8 多功能融合芯片

涵盖了算、存、传、管、智等 5 个方面，对应的有诸如如大家较为熟悉的华为鲲鹏 920，是首款 7nm ARM 服务器处理器，32/48/64 核，2.6GHz；Hi1812，为智能 SSD 控制芯片，通过 PCIe NVMe 与 SAS 融合进行智能加速；Hi1822 为智能融合网络芯片，通过以太与 FC 融合，进行协议加速；Hi1710 为内置智能管理引擎的一款智能管理芯片，适合一些智能故障管理场景；Ascend 310/910 为人工智能芯片，采用达芬奇架构，提升能效和性能。

1.1.3.5 鲲鹏计算产业优势

在鲲鹏计算产业的框架之下，产业链上下游的厂商可以基于鲲鹏和 Arm 已有生态，开发出具有差异化优势的产品和解决方案。

华为作为鲲鹏计算产业的一员，掌握 Arm64 处理器核、微架构及芯片设计的关键技术，拥有 Armv8 永久架构授权，并在此基础上发展了 Kunpeng 系列处理器。同时华为在芯片领域上的长期投入和持续创新，面向计算、存储、传输、管理和 AI 打造了全面的具有差异化竞争力的芯片体系，有能力提供支持鲲鹏计算产业持续发展的算力底座需求。

鲲鹏计算产业首要任务是基于各行业用户的需求，孵化和完善面向行业的应用，打开产业空间，为所有厂家找到与之相匹配的生产要素和市场机会，通过商业成功促进产业链各环节的持续创新，形成螺旋式上升的发展路径。随着鲲鹏计算产业在中国市场的发展壮大，吸纳更多的海内外企业加入，最终成为具有持续创新能力和全球领先优势的计算产业。

1.1.3.6 鲲鹏计算产业的典型应用场景

鲲鹏计算产业的发展壮大，离不开产业界上下游伙伴的共同努力。在过去几年时间里，华为与各行业 ISV 共同孵化、完善了五大鲲鹏计算解决方案，在多个行业实现了规模商用。

譬如大数据场景，鲲鹏 920 处理器最高支持 64 核，多核优势意味着可并行处理更多的数据。实测数据显示，鲲鹏处理器的大数据性能比传统平台提升 30%以上。

分布式存储场景，鲲鹏处理器通过内置数据压缩引擎，多核并行计算提升分布式存储软件的运行效率，相比传统处理器算法方案减少 66%的压缩时间，并实现 IOPS 性能提升 20%以上。

在鲲鹏处理器设计之初，华为就充分考虑了如何满足分布式并行计算的需求。在鲲鹏天生多核的基础上，增强了高 I/O 带宽、高内存带宽、高数据吞吐的设计，更好的满足了 IT 分布式演进的发展趋势。



图1-9 典型应用场景

1.1.4 鲲鹏生态体系介绍

1.1.4.1 鲲鹏计算产业生态全景



图1-10 鲲鹏生态全景

华为携手业界技术厂商打造开放包容的鲲鹏计算产业生态，携手伙伴面向行业提供针对性解决方案，通过社区建设以及开发者大赛等活动为开发者提供交流竞赛的平台，积极开展高校合作，进行人才培养。

1.1.4.2 产业生态 - 鲲鹏计算产业发展总览



图1-11 鲲鹏产业生态

鲲鹏计算产业目标是建立完善的开发者和产业人才体系，通过产业联盟、开源社区、鲲鹏创新中心旗舰店、行业标准组织一起完善产业链，打通行业全栈，使鲲鹏生态成为开发者和用户的首选。

1.1.4.3 高校合作 - 携手高校，培养鲲鹏计算人才



图1-12 高校合作计划

每一个新兴产业的发展，都离不开人才的培养。华为面向中国高校推出了促进计算产业人才培养的“1234 鲲鹏高校人才计划” ：

1. 投入“1”千万：2019 年华为将通过教育部产学合作协同育人项目首批投入一千万元软硬件资源，与高校专家共同建设计算机体系结构、操作系统、数据库、人工智能等课程的实验资源；
2. 服务“2”大人群：服务教师和学生两大人群：面对教师，联合开发不少于 20 门的精品课程和数字教材，并组织鲲鹏生态相关的师资培训和技术研讨；面向学生，推出创新训练营和拔尖人才计划，并例行举办各类创新大赛；
3. 联合“3”类组织：华为将联合教学指导委员会（计算机类专业教执委、软件工程教指委等）、联盟（国家示范性软件学院联盟、信息技术新工科产学研联盟等）、学会（中国计算机学会、中国高等教育学会等）3 类教育专家组织，制订关键核心技术领域人才培养方

案和专业标准、设计与开发产学合作课程，建设与推广计算产业开源社区等，探索鲲鹏计算产业人才培养合作的新模式；

4. 培养“4”类人才：基于 4 类华为认证标准：“鲲鹏应用开发者认证”、 “GaussDB 数据库认证”、 “智能计算认证”、 “人工智能认证”，支持中国高校开展鲲鹏生态所需人才的培养，培养计算产业人才。

1.1.4.4 伙伴生态 - 鲲鹏伙伴计划

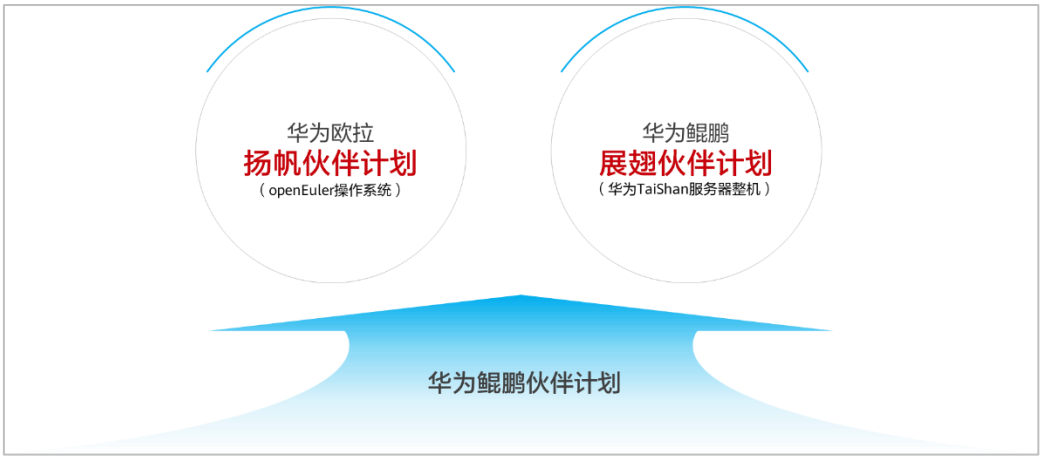


图1-13 鲲鹏伙伴计划

欧拉扬帆伙伴计划：欧拉扬帆伙伴计划（华为）是华为围绕 openEuler 推出的一项生态伙伴计划，旨在保障 openEuler 生态健康有序发展，加速生态伙伴基于 openEuler 系列产品和部件构建产品与解决方案，使能伙伴生态发展与商业成功。加入我们，您可以获得：

1. openEuler 职业认证培训服务及考券。
2. 鲲鹏生态创新中心伙伴资源。
3. 欧拉生态创新中心伙伴资源。
4. 营销材料及展会可使用华为伙伴等级 Logo 及证书。
5. 在 openEuler 产业营销活动中，获得参会及品牌露出优先权。
6. 在华为或华为销售伙伴主导的市场机会中，获得项目推荐优先权。

鲲鹏展翅伙伴计划：鲲鹏展翅伙伴计划是华为围绕鲲鹏计算平台推出的一项生态伙伴计划，旨在帮助伙伴整合鲲鹏计算产业的全栈技术、品牌资源，构建产品与解决方案竞争力，联接客户与鲲鹏生态系统，创造无限商机。加入该计划，伙伴可以获得：

1. 伙伴技术资料下载。
2. 鲲鹏生态创新中心构建资源。
3. 营销材料及展会可使用华为伙伴等级 Logo 及证书。
4. 支持伙伴营销活动。
5. 在华为或华为销售伙伴主导的市场机会中，获得项目推荐优先权。
6. 提供等值技术认证支持服务。

1.1.4.5 社区生态 - 打造一站式云上鲲鹏专区



图1-14 社区生态

1.1.4.6 开发者生态 - 沃土计划 2.0

沃土计划主要提供如：培训赋能、沃土高校、华为认证、鲲鹏创新中心旗舰店、联合营销、创新基金、沃土工厂等，2019-2024 年产业生态发展计划投入 15 亿美金。

1.2 华为鲲鹏处理器介绍

1.2.1 处理器架构介绍

传统的 CISC（Complex Instruction Set Computer，复杂指令集计算机）体系由于指令集庞大，指令长度不固定，指令执行周期有长有短，使指令译码和流水线的实现在硬件上非常复杂，给芯片的设计开发和成本的降低带来了极大困难。

随着计算机技术的发展需要不断引入新的复杂的指令集，为支持这些新增的指令，计算机的体系结构会越来越复杂。然而，在 CISC 指令集的各种指令中，其使用频率却相差悬殊，大约有 20%的指令会被反复使用，占整个程序代码的 80%。而余下的 80%的指令却不经常使用，在程序设计中只占 20%，显然，这种结构是不太合理的。

针对这些明显的弱点，1979 年美国加州大学伯克利分校提出了 RISC（Reduced Instruction Set Computer，精简指令集计算机）的概念，RISC 并非只是简单地去减少指令，而是把着眼点放在了如何使计算机的结构更加简单合理地提高运算速度上。

RISC 结构优先选取使用频率最高的简单指令，避免复杂指令；将指令长度固定，指令格式和寻址方式种类减少；以控制逻辑为主，不用或少用微码控制等措施来达到上述目的。

将 x86 和 ARM 架构相比较，可以看出 ARM 架构具有更好的并发性能。

目前超过 90%的移动终端采用的是 ARM 架构的处理器。随着 IOT、AI 和业务云化的发展，ARM 在终端的优势地位将会带动其进入数据中心市场，成为下一个快速增长的市场领域。

1.2.2 华为鲲鹏处理器概述

芯片是构建各类计算产品、使能上层软件和应用的底座，也是全产业链可持续创新和发展的驱动力。作为鲲鹏计算产业底座的 Kunpeng 处理器，华为将持续保持重点投入，秉承量产一代、研发一代、规划一代的演进节奏，落实“长期投入、全面布局，后向兼容和持续演进”的基础战略，通过对产业界提供以 Kunpeng 系列处理器为核心的芯片族和相应的产品，高效满足市场对新算力的需求。华为鲲鹏处理器是华为自主研发的基于 ARM 架构的企业级系列处理器产品，包含“算、存、传、管、智”五个产品系统体系。

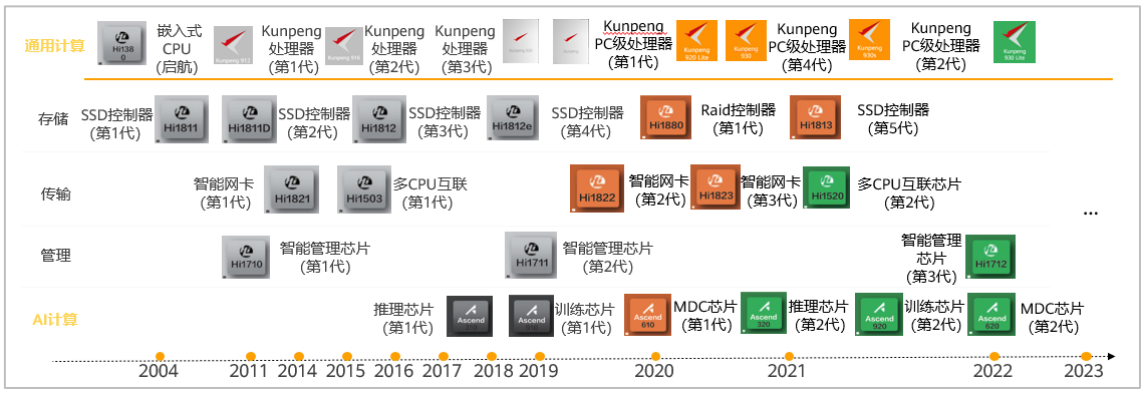


图1-15 华为鲲鹏处理器演进

华为鲲鹏处理器架构特点：

- 优点：

1. 采用 ARM 架构，同样功能性能占用的芯片面积小、功耗低、集成度更高，更多的硬件 CPU 核具备更好的并发性能。
2. 支持 64 位指令集，能很好地兼容从 IoT、终端到云端的各类应用场景。
3. 大量使用寄存器，大多数数据操作都在寄存器中完成，指令执行速度更快。
4. 采用 RISC 指令集，指令长度固定，寻址方式灵活简单，执行效率高。

- 缺点：

1. 在数据中心领域仍属于新进入者，目前其生态仍处于发展之中。

1.2.3 华为鲲鹏处理器型号及规格

华为从 2004 年开始基于 ARM 技术自研芯片。在通用计算处理器领域，2014 年发布 Kunpeng 912 处理器，2016 年发布 Kunpeng 916 处理器，2019 年 1 月发布 Kunpeng 920 处理器，Kunpeng 920 处理器是业界第一颗采用 7nm 工艺的数据中心级的 ARM 架构处理器。



图1-16 鲲鹏处理器型号

1.2.4 华为鲲鹏处理器技术创新

- 内核全自研
- 内存/网络接口&IO 协议
- 制程工艺领先
- 可靠性提升

1.3 Taishan 服务器介绍

1.3.1 服务器通识类知识简介

1.3.1.1 什么是服务器?

服务器是计算机的一种，它是在网络操作系统的控制下为网络环境里的客户机（如 PC）提供共享资源（包括查询、存储、计算等）的高性能计算机，它的高性能主要体现在高速度的 CPU 运算能力、长时间的可靠运行、强大的 I/O 外部数据吞吐能力等方面。

1.3.1.2 服务器发展历程

- 世界上最早的计算机，是 2600 年前中国发明的算盘
- 1964 年 IBM 推出第一台大型机– System/360，成为真正意义上的服务器
- 1989 年康柏推出全球第一款 IA 架构 的服务器：SystemPro，采用 Intel 486 微处理器
- 90 年代衍生出针对中小企业低成本 的小型机
- 服务器的核心部件“CPU”，一直在遵从摩尔定律，绝大部分服务器采用基于 x86 架构的 Intel 处理器

1.3.1.3 服务器的构成

服务器主要由 CPU、内存、硬盘三大件组成，配合电源、主板、机箱等基础硬件以提供信息服务。

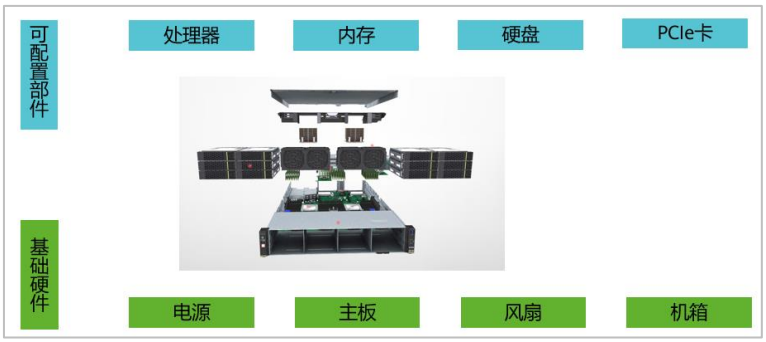


图1-17 服务器构成

1.3.1.4 服务器主流类型

- 按 CPU 指令集分类

ARM 服务器、x86 服务器

- 按服务器外形分类

塔式、机架式、刀片式、高密

- 按负载类型分类

数据库服务器、应用服务器、Web 服务、接入服务器、文件服务器

- 按应用层次分类

入门级服务器、工作组级服务器、部门级服务器、企业级服务器

1.3.2 Taishan 服务器概述

1.3.2.1 基于华为鲲鹏处理器，构建整机计算能力

2016 年推出第一代 Taishan 100 服务器基于鲲鹏 916 处理器，以华为鲲鹏处理器为基础，构建整机计算能力。

2019 年推出 Taishan 200 服务器基于鲲鹏 920 处理器，目前仍然是市场的主打产品。

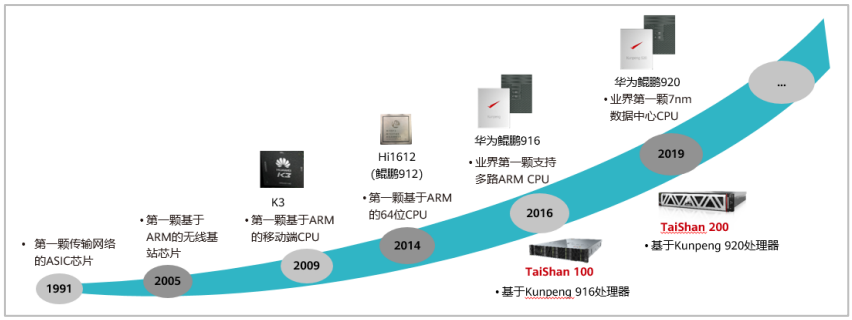


图1-18 Taishan 服务器演进

Taishan 服务器有如下特性：

- 高性能计算：提供兼容 ARM 架构的高性能华为鲲鹏处理器、TaiShan 服务器和解决方案，将高效能计算带入数据中心

- 安全可靠：华为鲲鹏处理器基于自研内核，TaiShan 服务器计算芯片全自研
- 开放生态：提供开放平台功能，支持业界主流软硬件；构建鲲鹏生态，与开发者、伙伴和产业组织共同打造智能计算新底座

1.3.3 Taishan 200 机架服务器介绍

基于 Kunpeng 处理器，华为打造了鲲鹏计算整机产品标杆 – TaiShan 服务器。在 TaiShan 200 服务器上充分应用了散热液冷、高速互联、可靠性设计与质量品控等工程工艺技术，将 TaiShan 服务器打造成为精品，为 Kunpeng 处理器应用在数据中心服务器领域树立了一个行业标杆。

TaiShan 服务器目前已经规模商用的有 2280 均衡型、5280 存储型、X6000 高密型。

1.4 openEuler 操作系统介绍

1.4.1 Linux 操作系统介绍

1.4.1.1 操作系统介绍

操作系统是一个管理计算机硬件与软件资源的程序，同时也是计算机系统的内核与基石。操作系统身负诸如管理与配置内存、决定系统资源供需的优先次序、控制输入与输出设备、操作网络与管理文件系统等基本事务。操作系统也提供一个让使用者与系统交互的操作接口。小到智能穿戴设备、电视盒子，大到电脑、服务器、路由交换设备，都有自己的操作系统。

1.4.1.2 Linux 操作系统起源

- 1991 年 10 月 5 日，Linus Torvalds 在 comp.os.minix 新闻组织上正式对外宣布 Linux 内核的诞生，并且开源。
- 在开源社区参与者的努力下，1994 年 3 月 Linux 1.0 内核版本正式发布。
- Linux 是一套免费使用和自由传播的类 Unix 操作系统，是一个基于 POSIX 和 Unix 的多用户、多任务、支持多线程和多 CPU 的操作系统。

Linux 操作系统组成：

- Linux 操作系统构成：Linux 内核、应用程序
- 常见的 Linux 操作系统：RedHat、CentOS、Fedora、Kaili Linux、Debian、Ubuntu、deepin、中标麒麟等。

1.4.1.3 CentOS 简介

CentOS Linux 发行版是一个稳定的，可预测的，可管理的和可复现的平台，源于 Red Hat Enterprise Linux（RHEL）依照开放源代码（大部分是 GPL 开源协议）规定释出的源码所编译而成。CentOS Linux 发行版是一个稳定的，可预测的，可管理的和可复现的平台，源于 Red Hat Enterprise Linux（RHEL）依照开放源代码（大部分是 GPL 开源协议）规定释出的源码所编译而成。2020 年 12 月 08 日，CentOS 官方宣布 CentOS Linux 项目停止，因此当前使用 CentOS 系统的用户面临停服后如何更新、维护、系统迁移等问题。

注：TaiShan 服务器支持 Cent OS 7.4 for ARM 及以上版本，但并不是所有服务器都兼容这些版本，具体兼容信息请在智能计算产品兼容性查询助手上查询。

1.4.1.4 Debian 简介

Debian 是迄今为止最遵循 GNU 规范的 Linux 系统。提供了接近十万种不同的开源软件支持，认可度和使用率较高，对于各类内核架构支持性良好，稳定性、安全性强，更有免费的技术支持。

基于其优秀的稳定性和资源开销小等特点，Debian 在虚拟专用服务器中应用比较多。

1.4.1.5 Ubuntu 简介

Ubuntu 是 Debian 的一款衍生版，是一个以桌面应用为主的，开源且免费的操作系统。Ubuntu 侧重于在服务器、云计算、甚至一些运行 Ubuntu Linux 的移动设备上进行系统部署。

Ubuntu 具有庞大的社区力量，用户可以方便地从社区获得帮助。

注：TaiShan 服务器支持 Ubuntu 16.04.3 LTS ARM 及以上版本，但并不是所有服务器都兼容这些版本，具体兼容信息请在智能计算产品兼容性查询助手上查询。

1.4.1.6 deepin 简介

deepin 操作系统是由武汉深之度科技有限公司开发的 Linux 发行版。deepin 操作系统拥有自主设计的特色软件：深度软件中心、深度截图、深度音乐播放器和深度影音，全部使用自主的 deepinUI。

deepin 操作系统专注于使用者对日常办公、学习、生活和娱乐的操作体验的极致，适合笔记本、桌面计算机和一体机。

注：TaiShan 服务器支持 deepin V15 for ARM 及以上版本，但并不是所有服务器都兼容这些版本，具体兼容信息请在智能计算产品兼容性查询助手上查询。

1.4.1.7 中标麒麟简介

中标麒麟是中标 Linux 和银河麒麟合并后发布的一款 Linux 发行版本，中标麒麟操作系统采用强化的 Linux 内核，分成桌面版、通用版、高级版和安全版等，满足不同客户的要求，已经广泛地使用在能源、金融、交通、政府、央企等行业领域。

注：TaiShan 服务器支持 NeoKylin Server v5.0 U5 for ARM 及以上版本，但并不是所有服务器都兼容这些版本，具体兼容信息请在智能计算产品兼容性查询助手上查询。

表1-1 各行业常用 OS

行业	常用 OS
电信运营商	CentOS 等
银行、证券	CentOS、中标麒麟、银河麒麟等
互联网	企业定制 OS、Debian 等

政府、央企	CentOS、中标麒麟、银河麒麟、统信等
电力、能源	CentOS、中标麒麟、银河麒麟、Ubuntu 等
交通运输	CentOS、中标麒麟等
安平	CentOS 等
教育、医疗	CentOS、银河麒麟等

1.4.1.8 华为鲲鹏处理器 OS 兼容性

表1-2 鲲鹏处理器 OS 兼容性

	常用 OS
Kunpeng	deepin
	ubuntu
	银河麒麟
	suse
	Debian
	CentOS
	中标麒麟

此处列举了华为鲲鹏处理器兼容的一部分操作系统，目前鲲鹏处理器仅支持 Linux 类型操作系统，上述列举出了鲲鹏处理器支持的部分操作系统。具体可以到《智能计算产品兼容性查询助手》查询：http://support.huawei.com/onlinetoolweb/ftca/index?serise=9。

1.4.2 openEuler 操作系统介绍

1.4.2.1 openEuler 概述

openEuler 脱胎于 EulerOS，EulerOS 是华为公司自 2010 年起研发使用的服务器操作系统，Linux 发行版之一，名字来源于著名数学家莱昂哈德·欧拉（Leonhard Euler）。

2019 年 9 月，EulerOS 正式开源，命名为 openEuler。

openEuler 是一款开源操作系统。当前 openEuler 内核源于 Linux，支持鲲鹏及其它多种处理器，能够充分释放计算芯片的潜能，是由全球开源贡献者构建的高效、稳定、安全的开源操作系统。openEuler 适用于数据库、大数据、云计算、人工智能等应用场景。同时，openEuler 也是一个面向全球的操作系统开源社区名称。

1.4.2.2 openEuler 技术特性-A-Tune 资源调优自动化

A-Tune 是一种通过非侵入式系统画像的负载感知方法，识别业务并匹配最佳资源模型，实时响应业务特征变化的 AI 自动调优系统。

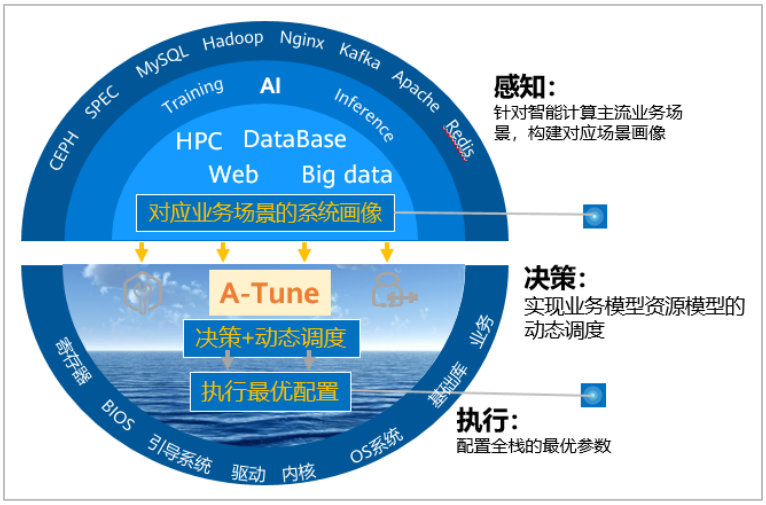


图1-19 A-Tune 资源调优自动化

- A-Tune 基于系统画像完成业务负载归类，识别鲲鹏主流行业场景，实现资源模型建模和系统自优化，发挥鲲鹏算力。
- 基于机器学习完成系统画像，结合聚类+分类多次迭代实现业务负载的精确归类，涵盖当前 ICT 业务主流计算场景。
- 基于业务特征完成系统资源建模，合理优化系统资源及参数配置，实现系统能力的价值最大化。

1.4.2.3 openEuler 技术特性 - iSulad 容器解决方案

iSula 通用容器引擎（iSulad）是一种新的容器解决方案，提供统一的架构设计来满足 CT 和 IT 领域的不同需求。相比 Golang 编写的 Docker，轻量级容器具有轻、灵、巧、快的特点，不受硬件规格和架构的限制，底噪开销更小，可应用领域更为广泛。openEuler-20.03-LTS 中提供容器运行的基础平台 iSula。

容器根据部署模型可以分成三种模型：应用容器、安全容器、系统容器。

- 应用容器：应用最广泛的容器形态，容器中仅运行客户定义的应用程序。openEuler-20.03-LTS 集成 moby 18.09 版本，并在版本基础上进行了 bugfix 和稳定性增强。
- 安全容器：本质上是虚拟机，但是接口封装为容器接口。openEuler-20.03-LTS 集成 kata-container1.7 版本，在社区版本的基础上，进行了稳定性和可靠性加固，增加了异构计算支持，更适于生产环境的 Host CGroup 配置与 DPDK/SPDK 高性能网络、存储支持。
- 系统容器：本质上是容器，但是使用方式和虚拟机相同；着重解决虚拟机的厚重问题。iSula 容器平台支持创建系统容器，并能支持在系统容器内动态调整设备、运行资源，且提供更优秀的 user namespace 隔离。

1.4.2.4 鲲鹏硬件相关的特性 – MPAM

MPAM（Memory Partitioning and Monitoring）适用于控制/监控 L3 Cache 和内存带宽的分配和使用，对标 x86 高端芯片的 RDT 特性。

在 openEuler 的 21.03 版本，也支持通过 Arm64 架构 Cache QoS 以及内存带宽控制技术进行资源管控。

1.4.3 嵌入式 OS 介绍

1.4.3.1 嵌入式操作系统概述

嵌入式操作系统（Embedded Operating System，简称：EOS）是指用于嵌入式系统的操作系统。嵌入式操作系统是一种用途广泛的系统软件，通常包括与硬件相关的底层驱动软件、系统内核、设备驱动接口、通信协议、图形界面、标准化浏览器等。嵌入式操作系统负责嵌入式系统的全部软、硬件资源的分配、任务调度，控制、协调并发活动。它必须体现其所在系统的特征，能够通过装卸某些模块来达到系统所要求的功能。目前在嵌入式领域广泛使用的操作系统有：嵌入式实时操作系统 μC/OS-II、嵌入式 Linux、Windows Embedded、VxWorks 等，以及应用在智能手机和平板电脑的 Android、iOS 等。

当前工业、医疗、军工/航空航天等行业，嵌入式 OS 基本被国外垄断，国内嵌入式软硬件能力虽已初步具备，但整体呈现“小、散、弱” 的特点，技术水平和市场竞争力不足，缺乏完整技术体系。

1.5 openGauss 数据库介绍

1.5.1 数据库概述

数据库是“按照数据结构来组织、存储和管理数据的仓库”，是一个长期存储在计算机内的、有组织的、可共享的、统一管理的大量数据的集合 。

关系型数据库

关系模型的提出是数据库发展史上具有划时代的重大事件。关系理论研究和关系数据库管理系统研制的巨大成功进一步促进了关系数据库的发展。

关系型数据库主流应用场景

- 联机事务处理 OLTP：OLTP 是传统的关系型数据库的主要应用，事务性非常高的系统，一般都是高可用的在线系统，以小的事务以及小的查询为主，主要是基本的、日常的事务处理，例如银行交易。OLTP 系统强调数据库内存效率，评估其系统的时候，一般看其每秒执行的 Transaction 以及 Execute SQL 的数量。强调内存各种指标的命令率，强调绑定变量，强调并发操作。
- 联机分析处理 OLAP：OLAP 是数据仓库系统的主要应用，支持复杂的分析操作，侧重决策支持，并且提供直观易懂的查询结果。在这样的系统中，语句的执行量不是考核标准，因为一条语句的执行时间可能会非常长，读取的数据也非常多。OLAP 系统则强调数据分析，考核的标准往往是磁盘子系统的吞吐量（带宽），强调 SQL 执行时长，强调磁盘 I/O，强调分区等。

1.5.2 openGauss 数据库介绍

openGauss 是一款携手伙伴共同打造全球领先的企业级开源关系型数据库，提供面向多核的极致性能、全链路的业务和数据安全、基于 AI 的调优和高效运维的能力。openGauss 全面友好开放，采用木兰宽松许可证 v2 发行，深度融合华为在数据库领域多年的研发经验，结合企业级场景需求，持续构建竞争力特性。

1.5.2.1 openGauss 发展历程

openGauss 发展历程整体可以分成如下四个阶段：

- 内部自用：2001-2011 年，在华为多个产品，内部自用，称为企业级内存数据库。
- 产品化：2011-2019 年，从自用推出正式产品进行商用，如 G 行核心数据仓库、DWS 华为云商用；Z 行核心业务系统替换商业数据库；支撑公司内部 40+主力产品，在全球 70+运营商规模商用 3 万+套，服务全球 20+亿人口。
- 云&开源：2019 年 5 月 15 日，GaussDB 全球发布，旨在构筑合作伙伴生态，兼容行业主流生态，完成金融等行业对接；2020 年 6 月 30 日，openGauss 开源。
- 生态构建：2021 年起，开源后，华为在 openGauss 方面主要在于分享企业级数据管理能力，引领生态建设，促进数据库教育事业发展。

1.5.2.2 openGauss 特性

- 高性能：提供了面向多核架构的并发控制技术结合鲲鹏硬件优化，在两路鲲鹏下 TPCC Benchmark 达成性能 150 万 tpmc；针对当前硬件多核的架构趋势，在内核关键结构上采用了 NUMA-Aware 的数据结构；提供 Sql-bypass 智能快速引擎技术。
- 高可用：支持主备同步、异步以及级联备机多种部署模式；数据页 CRC 校验，损坏数据页通过备机自动修复；备机并行恢复，10 秒内可升主提供服务。
- 高安全：支持全密态计算、访问控制、加密认证、数据库审计、动态数据脱敏等安全特性，提供全方位端到端的数据安全保护。
- 易运维：基于 AI 的智能参数调优和索引推荐，提供 AI 自动参数推荐；慢 SQL 诊断，多维性能自监控视图，实施掌控系统的性能表现；提供在线自学习的 SQL 时间预测。
- 全开放：采用木兰宽松许可证协议，允许对代码自由修改、使用、引用；数据库内核能力全开放；提供丰富的伙伴认证、培训体系和高校课程。

1.5.2.3 SQL 定义

SQL（Structured Query Language，结构性查询语言）是一种特定目的编程语言，用于管理关系数据库管理系统，或在关系流数据管理系统中进行流处理。

SQL 基于关系代数和元组关系演算，包括一个数据定义语言和数据操作语言。SQL 的范围包括数据插入、查询、更新和删除，数据库模式创建和修改，以及数据访问控制。

openGauss 支持标准的 SQL92/SQL99/SQL2003/SQL2011 规范，支持 GBK 和 UTF-8 字符集，支持 SQL 标准函数与分析函数，支持存储过程。

SQL 语法简介

- DDL（Data Definition Language）数据定义语言

用于定义或修改数据库中的对象。如：表、索引、视图、数据库、序列、用户、角色、表空间、存储过程等。

- DML（Data Manipulation Language）数据操纵语言

用于对数据库表中的数据进行操作，如插入，更新和删除。

- DCL（Data Control Language）数据控制语言

用来设置或更改数据库事务、授权操作（用户或角色授权，权限回收，创建角色，删除角色等）、锁表（支持 SHARE 和 EXCLUSIVE 两种锁表模式）、停机等。

- DQL（Data Query Language）数据查询语言

用来查询数据库内的数据，如查询数据、合并多个 select 语句的结果集。

1.5.3 应用开发简介

1.5.3.1 ODBC

开放数据库连接（Open Database Connectivity，ODBC）是为解决异构数据库间的数据共享而产生的，现已成为 WOSA（The Windows Open System Architecture（Windows 开放系统体系结构））的主要部分和基于 Windows 环境的一种数据库访问接口标准。

ODBC 为异构数据库访问提供统一接口，允许应用程序以 SQL 为数据存取标准，存取不同 DBMS 管理的数据；使应用程序直接操纵 DB 中的数据，免除随 DB 的改变而改变。用 ODBC 可以访问各类计算机上的 DB 文件，甚至访问如 Excel 表和 ASCII 数据文件这类非数据库对象。

1.5.3.2 JDBC

Java 数据库连接，（Java Database Connectivity，简称 JDBC）是 Java 语言中用来规范客户端程序如何来访问数据库的应用程序接口，提供了诸如查询和更新数据库中数据的方法。JDBC 也是 Sun Microsystems 的商标。我们通常说的 JDBC 是面向关系型数据库的。

1.5.3.3 Psycopg

psycopg 是一个 PostgreSQL 数据库的 Python 语言的接口；

它的主要优点是它支持完整的 python DBAPI 2.0。它是针对大量多线程的应用程序设计的；

Psycopg 分布包括 Zpsycopgda，Zope 数据库适配器。

2 鲲鹏应用移植

2.1 软件迁移原理与迁移过程

2.1.1 软件迁移原理

通过第一章的学习，我们了解了鲲鹏硬件和鲲鹏生态。本门鲲鹏应用开发者课程旨在帮助广大开发者加入到鲲鹏生态中来，除了基于鲲鹏做原生开发之外，更多的是将已经成熟的 x86 应用迁移到鲲鹏平台上并进行调优完成适配。所以本章鲲鹏应用移植重点讨论如何将原本运行在 x86 平台上的应用程序迁移到鲲鹏平台上，并使其正常运行。

在讨论数据中心领域的迁移之前，我们首先以身边常见的 x86 架构的 Windows 电脑和 ARM 架构的安卓手机为例。比如将 Windows 系统的某个可执行程序，例如 putty.exe，传送到手机上，这也是一个 x86 向 ARM 迁移的案例。显而易见，在 x86 电脑上正常运行的 putty，在手机上并不能运行使用，那么大家有没有思考过本质原因是什么?

要解答这个问题，我们先来看一看计算机的技术栈和程序的执行过程。

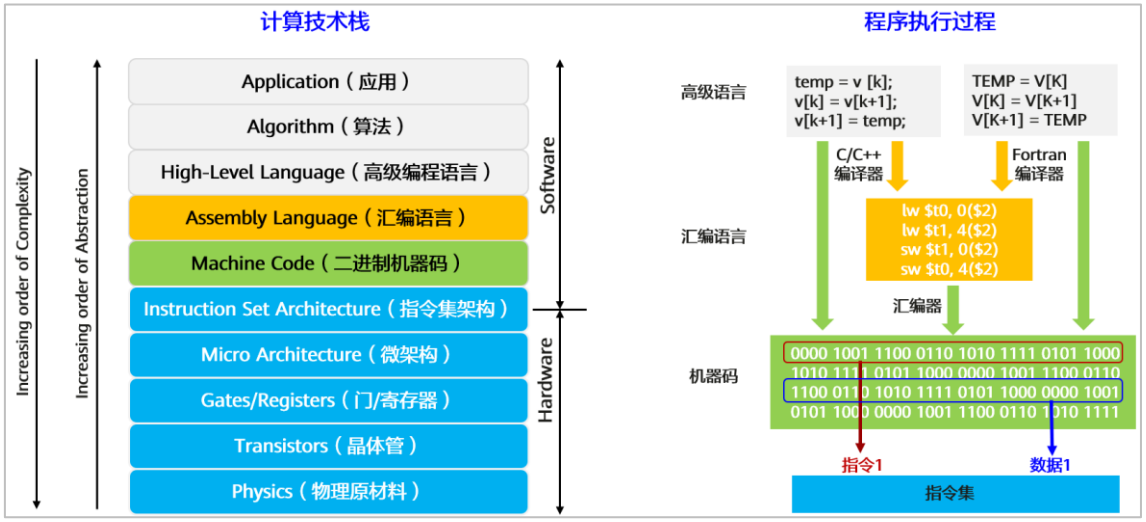


图2-1 计算技术栈与程序执行过程

如上左图为计算技术栈，可以看到硬件的最底层是物理材料、晶体管来实现门电路和寄存器，再组成 CPU 的微架构。软件层面应用程序通过一定的软件算法完成业务功能，程序通常使用如 C/C++/Java/Go/Python 等高级语言开发。而和计算机直接交互的是二进制机器码，一个程序在磁盘上存在的形式，是一堆指令和数据所组成二进制机器码，也就是我们通常说的二进制文件。CPU 的指令集是硬件和软件的接口，处于枢纽地位，指令集指导了硬件的实现，应用程序通过指令集中定义的指令驱动硬件完成计算。

如上右图为程序的执行过程。高级语言需要编译成汇编语言，再由汇编器按照 CPU 指令集转换成二进制的机器码。由此可见，指令集和低级语言是一一对应的，不同的指令集对应不同的机器码和汇编语言。

回到我们举的例子，Windows 电脑上的 putty.exe 文件是二进制机器码，而且是基于 x86 指令集编译得到的机器码，该二进制文件传到手机上后，ARM 架构的手机并不能识别，所以无法运行。

接下来我们回到数据中心领域，以几行简单的 C 程序为例，看下鲲鹏处理器和 x86 处理器的指令差异。

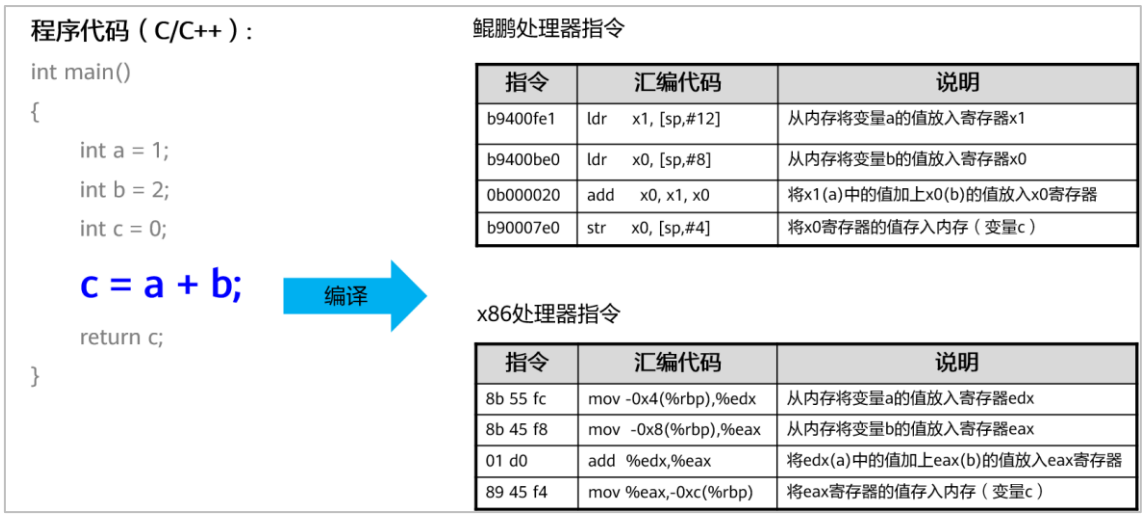


图2-2 指令差异

如图，一行简单的 C/C++代码，c=b+a，在鲲鹏处理器和 x86 处理两个平台下编译的指令有很大不同。鲲鹏处理器下是由两条 ldr 指令完成从内存将数据加载到寄存器，一条 add 指令完成加法计算，最后再使用一条 str 指令将数据存储到内存中。x86 处理器下，使用了三条 mov 指令和一条 add 指令。两条 mov 指令将内存中的数据加载到寄存器，一条 add 指令完成加法，最后再使用一条 mov 指令将计算后的数据存储到变量 c 对应的内存。从改图中可以直观地看到鲲鹏和 x86 处理器，在汇编代码中表示相同功能的助记符不同，寄存器表示也不同。机器码层面 x86 处理器是不等长的指令，最长可达 15 字节，而鲲鹏处理器 ARM 指令定长 4 个字节。

编译器将源码编译成的可执行程序（windows 下.exe 文件，linux 下是二进制的 bin 文件），这个程序的文件有它自己的结构，但最主要的就是由一堆指令和数据所组成。程序在执行时，CPU 将程序中的指令和数据加载到 cache 缓存中，然后将指令一条条取出，执行。

所以不同的 CPU 所使用的指令集不同，因此同样一个源代码、同样一个功能，在不同的 CPU 平台下需要执行不同的指令。而这个从源码到指令的翻译工作，就是通常我们所说的编译，由编译器完成。因此当我们将程序移植到不同的 CPU 平台时，如果 CPU 的指令集不同，那么就一定需要重新从源码，按照目标平台的指令集，重新翻译一遍。

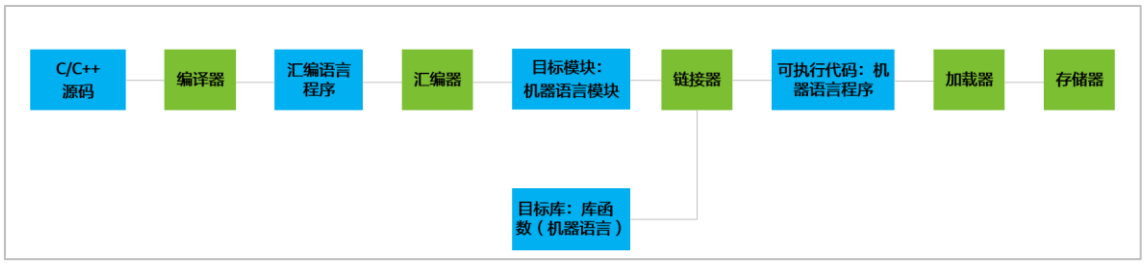


图2-3 源码编译过程

编译型语言在执行的时候需要通过编译器得到可执行程序才能加载到内存中进行执行。我们经常会使用 gcc test.c -o test 一步完成，但其实它经历了多个阶段。

第一步：预处理，读取源码，对其中的伪指令（以#开头的指令，也就是宏）和特殊符号进行“替代”处理，经过此处理，生成一个没有宏定义、没有条件编译指令、没有特殊符号的输出文件。这个文件的含义同没有经过预处理的源文件是相同的，仍然是 C 文件。但内容有所不同。可以通过 gcc -E test.c -o test.i 只执行预处理操作得到预编译文件。

第二步：编译，编译为汇编代码，可以通过 gcc -S test.i -o test.s 将预编译文件编译输出汇编文件。

第三步：汇编，将汇编代码文件通过汇编器汇编成目标文件，可以通过 gcc -c test.s -o test.o 完成。

第四步：链接，将目标文件与所需的附加目标文件（静态链接库和动态链接库、C 标准输入输出库）连接起来，最终生成可执行文件 test：gcc test.o -o test。

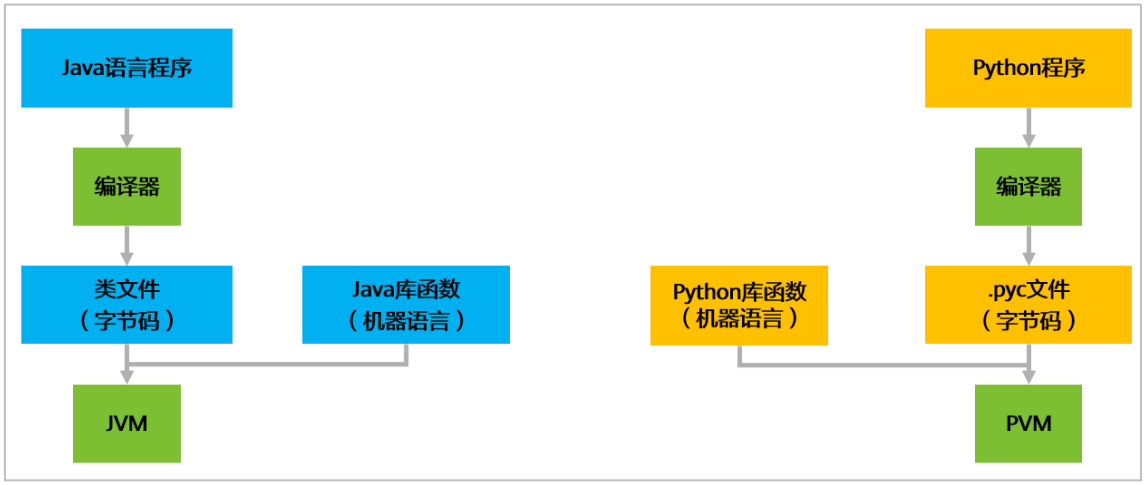


图2-4 执行过程对比

解释型语言在执行时需要先通过初步编译得到字节码文件，然后通过相应语言的虚拟机解释执行。

编译型语言程序和解释型语言程序根据不同的执行过程采用不同的迁移方式。

2.1.2 软件迁移过程

软件迁移从更大局观的角度可分为五个阶段：技术分析、编译迁移、功能验证、性能调优和规模商用。第二章我们重点介绍前两个阶段。每个阶段从三个方面展开：软件移植、迁移环境和管理工作。

首先是第一阶段技术分析，从软件移植方面，需要整体分析软硬件技术栈并制定迁移策略。软硬件技术栈可大体分为两部分：运行环境和业务软件。运行环境包括迁移环境，这里我们可以通过获取 Taishan 服务器测试样机或者申请远程服务器资源获取鲲鹏测试环境，后面会具体展开介绍。运行环境还包括驱动、操作系统、编译器、JDK/PVM 等，这些都需要替换成支持鲲鹏、Taishan 版本的相应环境。鲲鹏硬件环境可支持的操作系统可以通过该链接查询：https://info.support.huawei.com/computing/ftca/kunpeng。另一部分业务软件主要有商用软件、自研软件和开源软件，商用软件我们无法获取源代码，需要联系相应厂商获取对应的 ARM 版本，如无法获取到兼容版本，则需要替换其他类似软件，比如 Oracle 数据库替换成国产高斯系列数据库等。自研软件和开源软件的相同点就是我们都可以获取源代码，那么就可以根据编译型代码和解释型代码不同过程重新编译或重新解释执行，需要注意的是依赖包的替换。另外大部分开源软件官网已经有 ARM 版本，可以直接下载使用。接下来我们具体看下第一阶段获取迁移环境的几种方式。第一种性价比最高，可以通过华为云购买鲲鹏云服务器搭建开发环境，缺点是公有云上的服务器本质是虚拟机有一定性能损耗，如果是对性能要求比较高的业务场景，云服务器可能无法满足，或者租金较贵。第二种是购买物理服务器搭建本地环境，解决了云上性能不足的问题，相应地成本和技术要求都会更高。第三种可以基于鲲鹏创新中心旗舰店申请远程服务器，需要联系当地的创新中心获取支持。第四种是通过鲲鹏社区申请远程实验室资源，远程实验室提供的鲲鹏开发环境预装了“一站式”开发套件，包括后面会介绍到的代码迁移工具、编译器、性能分析工具等，可供开发者灵活使用，缺点是只能按时段预约。从管理工作方面，第一阶段需要制定迁移计划和迁移策略并组建迁移团队，迁移团队需要具备的能力是金字塔结构，最基础的是运维能力，比如熟悉服务器操作，Shell 脚本和 Linux 常用命令。另外在编译迁移的时候会涉及到源码的修改和重新编译，所以需要具备一定的开发能力。最顶层是性能调优的能力，因为影响性能的因素有很多，比如应用程序本身，它使用的高级语言，线程并行，锁操作等；应用运行的环境也会影响性能，主要是 CPU、内存、磁盘和网络以及 JVM 等。性能调优工程师不仅需要熟悉业务代码也要熟悉硬件，能够使用工具测试性能并找到瓶颈点调参调优，所以技术要求非常高。在管理方面，组建好迁移团队后就可以制定迁移计划了， 比如五个阶段分别有哪些详细的工作内容。

阶段二是执行编译迁移，我们先从编译型语言程序比如 C/C++代码的迁移看起。首先获取源码，自研软件直接对接自有的代码仓库，开源软件可以通过 GitHub、Gitee 或第三方开源社区获取源码；然后准备编译环境，安装编译器 gcc 等；接下来就可以使用预编译生成 Makefile 文件，然后 make 或 cmake 执行编译，需要注意的是编译型语言程序经常包含需要 SO 库，我们需要将这些依赖库替换成 ARM 版本或者根据依赖库的源码重新编译；最后在鲲鹏环境上编译得到的可执行程序就可以正常运行。这个过程是理想情况，比如对于一些简单的代码像 helloworld 这些，源码不涉及低级语言，和硬件没有强相关，我们替换依赖库重新编译就可以正常运行。但是对于现实业务中的代码，往往比较复杂，功能实现在不同的 CPU 平台上需要有不同的源码表示，这种情况下不修改代码直接重新编译时会出现报错，我们需要根据报错修改相应源码直至成功编译。这里有一些常见的问题，大家可以在编译前修改相应源码以避免报错。比如有数据类型的定义，x86 代码中定义数据为 char 类型，系统默认为有符号

字符，而在鲲鹏平台上，char 默认为无符号类型，所以代码程序从 x86 迁移到鲲鹏平台上时需要手动将 char 类型重定义为 signed char 类型，或者在编译时通过编译选项使所有 char 字符在编译时替换为有符号 char。另外还有 builtin 函数，表示相同功能的函数在 x86 和鲲鹏平台上是不同的表示，这时候我们就需要查找手册将函数替换，才能在鲲鹏平台上成功编译。除了源码之外还有编译选项、编译宏等也需要进行替换。这些需要修改替换的内容看似比较麻烦，后面会为大家介绍工具帮助我们简化迁移操作。介绍完编译型语言程序的迁移，接下来我们来看解释性语言程序比如 Java、Python 等。Java 程序在进行移植时，我们需要先准备迁移环境，包括安装 ARM 版本的 JDK，配置环境变量，然后初步编译得到字节码，再通过 JVM 解释执行，需要注意的是 jar 包中的依赖库大多是编译型语言程序，需要替换成 ARM 版本重新打包，或者获取依赖库源码重新编译再重新打包。所以 Java 程序的移植必不可少需要使用到构建工具，比如 maven。在使用 maven 迁移 java 程序时，maven 会先查询本地仓库找到依赖包，如果本地仓库没有，就会从远程仓库获取，这里获取的依赖库是不区分架构的，等到构建过程中扫描依赖库时如果发现是 x86 版本的，就会获取源码重新编译再次构建。Python 程序的迁移和 Java 大体一致，主要点在于准备环境和移植依赖库。其他语种像 GO、PHP 等，同样根据编译型语言和解释型语言程序不同的执行过程进行不同的移植操作。搭建程序运行环境时可以选择鲲鹏平台操作系统自带的解释器软件直接安装，也可以从官网下载 ARM 版本软件包解压使用，或者根据 x86 源码自行编译迁移后使用。在迁移的时候如果涉及到商用的闭源软件，比如 Oracle 商用数据库，一种方法是联系对应厂商询问是否有 ARM 版本进行替换部署，如果没有的话，就需要考虑替换其他数据库软件，比如国产的 GaussDB、达梦等都有 ARM 版本，或者是选择开源数据库。

2.1.3 软件迁移典型案例

某行业伙伴核心应用系统从原有的 2.4GHz，48 核的 x86+Debian OS 环境迁移到 2.6GHz，96 核的鲲鹏平台，操作系统补充了 30 个适配补丁，业务系统包含 C++、Java、汇编等 520+ 源码文件，迁移时涉及到 55 个依赖库的移植，出现 39 个编译问题，修改了 195 行代码，最终 5 人月完成了源码文件的移植，迁移后性能提升 56%。其中环境准备 1 人月，修改代码编译移植耗时 2 人月，迁移后性能优化 2 人月。

2.2 鲲鹏迁移指导

2.2.1 鲲鹏代码迁移工具介绍

前面我们了解了迁移原理和迁移流程，在重新编译前需要对源码修改的部分会感觉难度高又繁琐，为了发展鲲鹏生态，鲲鹏社区有提供相应的工具来简化我们的迁移工作。鲲鹏代码迁移工具是华为鲲鹏开发套件 DevKit 在迁移环节提供的一款可以简化客户应用迁移到基于鲲鹏 916/920 的服务器过程的工具。工具仅支持 x86 Linux 软件到 Kunpeng Linux 上的扫描、分析与迁移，不支持 Windows 软件代码的扫描、分析与迁移。通过这款工具我们可以方便地将 x86 Linux 软件迁移到鲲鹏 Linux 平台，代码迁移工具有三大常用功能：软件迁移评估（分析非源码包并提供迁移报告）、源码迁移（分析源码包并提供移植建议）、软件包重构（将 x86 软件包重构成鲲鹏软件包）。这三个功能基本覆盖了我们迁移过程的所有需要，另外有两个增强功能：专项软件迁移和鲲鹏亲和分析。在我们对源码文件进行迁移时，可以通过源码迁移功

能分析源码，工具会生成一个分析报告，包括该源码文件涉及修改替换的代码行数、依赖文件个数和迁移预估工作量，此外工具还会针对需要修改替换的源码给出具体的修改建议甚至是一键替换的按钮，方便迁移工程师快速修改源码，保存后重新编译验证。在对非源码文件比如二进制软件包或 java、python 程序进行迁移时，可以通过软件迁移评估功能分析软件包，得到分析报告，主要展示软件包涉及的依赖库个数，以及哪些是已有 ARM 版本的可以在分析报告界面直接下载使用，哪些依赖库需要自己编译替换。之后可以通过软件包重构功能，重构需要替换的依赖库。

使用代码迁移工具时需要将工具包安装在服务器上，然后通过 Web 访问或者 IDE 插件使用。如果安装时选择./install cli，那就只能通过命令行方式使用了。常用的 IDE 工具如 VS Code、IntelliJ 都支持鲲鹏代码迁移工具插件的安装和配置。

2.2.2 编译型语言应用迁移

基于编译型语言开发的应用程序，例如 C/C++语言应用程序，其编译后得到可执行程序，可执行程序执行时依赖的指令是 CPU 架构相关的。因此，基于 x86 架构编译的 C/C++语言应用程序，无法直接在 TaiShan 服务器或华为鲲鹏云服务器上运行，需要进行移植编译。应用移植的流程包括源码分析、根据建议修改源码、编译安装和测试验证。具体迁移过程和操作可以参考实验手册中 megahit 源码迁移的案例。

Fortran 作为编译型语言，迁移过程同样按照源码分析、移植建议、编译安装、测试验证流程。这里有个示例 Fortran 源码包：https://gitlab.com/DL_POLY_Classic/dl_poly/-/archive/RELEASE-1-10/dl_poly-RELEASE-1-10.tar.gz，将其上传源码包至服务器 sourcecode 目录下；进入解压后的目录，拷贝 Makefile，并更改文件属主。执行源码分析扫描过程。扫描根路径选择 Makefile 文件所在的 source 路径，类型勾选上 Fortran。然后查看扫描结果。显示有 1 个需要迁移的源码文件，涉及到 10 行代码。点击源码迁移建议查看右下图的原始源代码标红部分，根据提示建议完成修改并保存，然后运行验证。

2.2.3 解释型语言应用迁移

基于解释型语言开发的应用程序，是与 CPU 架构不相关的，例如 Java、Python，将这类应用程序移植到 TaiShan 服务器或华为鲲鹏云服务器，无需修改和重新编译，按照与 x86 一致的方式部署和运行应用程序即可。Java 应用程序 jar 包内，可能包含基于 C/C++语言开发的 SO 库文件，这类 SO 库需要移植编译，使用编译得到的 SO 库重新打包 jar 包。移植具体方法与 C/C++类应用相同。一般流程为：上传.jar 文件，新建分析软件包任务，分析得到需要迁移的依赖库文件；配置 Maven 环境，重新编译打包；解压进入 jar 包查看新的 SO 库验证迁移成功后部署运行。

2.2.4 二进制指令翻译工具 ExaGear

ExaGear 是华为自研动态二进制翻译工具，通过在运行时，将 x86 应用指令翻译为 ARM64 指令并执行，从而支持 Linux x86 应用无需重新编译就能运行在 ARM64 服务器上，帮助客户将 Linux x86 无源码应用快速迁移到 ARM 服务器上，且能稳定可靠运行。ExaGear 利用动态二进制翻译技术，结合动态二进制优化能力，能够稳定支持无源码的 x86 和 ARM32 存量业务运行在鲲鹏平台上，并且适于部署的场景广泛，既可以直接部署于操作系统，也可以部署于容

器中，甚至是在 ExaGear 中再部署容器，能够在无源码的情况下屏蔽底层平台差异，低成本解决应用的平滑迁移，释放鲲鹏平台澎湃算力。

在迁移过程中，用户业务按重要性差异，往往可分为三类：无源码业务、有源码的非关键业务和有源码的关键业务。行业相关的无源码商业软件。这类软件只有二进制、没有源码，在行业内又有相当的影响力。在这样的场景下，需要 ExaGear 动态二进制翻译技术保障顺利完成迁移。有源码的非关键业务。使用频率低、对性能不敏感。对于这部分业务，客户就可以利用 ExaGear 完成动态的二进制翻译，使存量业务应用不需要通过代码移植，就可以直接运行在鲲鹏平台上。这一过程省却了大量的移植或优化源代码所需的人力和时间，而且没有因移植代码而引入额外的稳定性隐患。性能敏感且有源码的关键业务。建议通过手工的代码移植和性能优化完成迁移，达到最优的性能预期。

2.3 容器迁移指导

2.3.1 容器相关概念介绍

容器是一种轻量级、可移植、自包含的软件打包技术，使应用程序可以在几乎任何地方以相同的方式运行。开发人员在自己笔记本上创建并测试好的容器，无需任何修改就能够在生产系统的虚拟机、物理服务器或公有云主机上运行。容器和虚拟机从该架构图来看最明显的区别就是每个虚拟机有自己的 Guest OS，而所有容器共享宿主机 OS 内核。从参数上看，容器相较虚拟机启动速度更快，部署密度更高，镜像更轻量，性能更优，但是安全性不如虚拟机。

Docker 是最受大众关注的容器技术，并且现在“几乎”成为事实上的容器标准。Docker 容器通过 Docker 镜像来创建。容器与镜像的关系类似于面向对象编程中的对象与类。镜像是多层的文件系统，当镜像运行成一个容器时，就多了一层可写的容器层。Copy on write 的特性保证了镜像的共享性，新增的内容只在容器层，需要修改的内容会复制一份在容器层里，不会影响底层镜像文件，同样，需要删除文件时，并不会真的删除底层镜像内容，而是在容器层新增一个对应的 whiteout 文件，表示该内容已被擦除。Docker 架构分为三部分：客户端、引擎服务、镜像仓库。使用者在客户端输入命令，Docker daemon 执行命令并根据命令和仓库交互。比如执行 docker run 的命令运行镜像创建容器。镜像可以通过 docker pull 命令从本地仓库或者远程仓库拉取，也可以通过 docker build 命令指定 Dockerfile 脚本文件构建，还可以通过 docker commit 命令将容器打包成镜像。Dockerfile 是一个文本格式的配置文件，包含创建镜像所需要的全部指令。基于 Dockerfile 中的指令，用户可以快速创建自定义的镜像。Dockerfile 配置文件说明了镜像的每一层内容，通过 docker build 命令可以基于 Dockerfile 文件构建镜像。Dockerfile 文件的第一行指定镜像的第一层：base image，一般是 FROM 某一个基础操作系统镜像或者 scratch 空镜像。

2.3.2 iSula 容器引擎介绍

iSula 是除了 Docker 之外的容器技术，openEuler 操作系统同时支持 Docker 和 iSula。iSula 通用容器引擎相比 Docker，是一种新的容器解决方案，提供统一的架构设计来满足 CT 和 IT 领域的不同需求；相比 Golang 编写的 Docker，iSula 轻量级容器使用 C/C++实现，具有轻、灵、巧、快的特点，不受硬件规格和架构的限制，底层开销更小，可应用领域更为广泛。同时

根据不同使用场景，提供多种容器形态，包括：适合大部分通用场景的普通容器。适合强隔离与多租户场景的安全容器。适合使用 systemd 管理容器内业务场景的系统容器。

iSula 容器运行侧和构建侧是分离的，构建侧需要另外安装 isula-build 工具，这种分离式的设计满足了部分用户更轻量的需求。iSulad 是一个基于 OCI 标准的容器运行引擎，强调简单性、健壮性和轻量化。作为守护进程，iSulad 提供容器生命周期管理相关服务：包括镜像的传输和存储、容器执行和监控管理、容器资源管理以及网络等。iSulad 对外提供与 docker 类似的 CLI 命令行接口，可使用该命令行进行容器管理；并且提供符合 CRI 接口标准的 gRPC API，可供 kubernetes 按照 CRI 接口协议调用。

2.3.3 容器迁移背景和原理

业务的运行载体可以是裸机也可以是虚拟机或容器，那容器应用在跨平台的时候该如何迁移呢？如果是相同架构，可以简单地通过容器打包成的镜像平滑迁移，直接在新平台上获取镜像，运行镜像，就可以运行相同应用。不同架构下，从仓库获取的 x86 容器镜像在 ARM 平台上无法成功运行，就需要重新构建新架构平台的镜像和容器。

2.3.4 容器迁移主要流程

比如要实现 Redis 容器应用的迁移，直接获取 x86 的 Redis 容器镜像无法在鲲鹏平台使用，我们可以通过编写 Dockerfile 配置文件构建新的 Redis 镜像并运行的方式获得鲲鹏平台的 Redis 容器应用。在鲲鹏平台使用 iSula 运行 Redis 容器应用的配置步骤：安装 iSulad 容器引擎和构建工具，制作 Dockerfile 文件，构建镜像，运行容器并验证 Redis。具体操作过程详见实验手册。

2.4 迁移常见问题及解决思路

2.4.1 常见编译问题

x86 源码在迁移到鲲鹏平台时经常出现的编译报错有以下几种：

char 数据类型默认符号不一致问题，char 变量在不同 CPU 架构下默认符号不一致，在 x86 架构下为 signed char，在 ARM64 平台为 unsigned char，移植时需要指定 char 变量为 signed char；可以通过在编译选项中加入"-fsigned-char"选项，指定 ARM64 平台下的 char 为有符号数，或者将 char 类型直接声明为有符号 char 类型：signed char。

代码中使用汇编指令，而汇编指令与 cpu 指令集强相关。在 x86 架构 CPU 中的汇编指令需要修改为鲲鹏处理器平台的指令才能编译通过，实现功能替换。

-m64 是 x86 64 位应用编译选项，m64 选项设置 int 为 32bits 及 long、指针为 64 bits，为 AMD 的 x86 64 架构生成代码。在鲲鹏处理器平台无法支持。将鲲鹏处理器平台对应的编译选项设置为-mabi=lp64，重新编译即可。

2.4.2 常见功能问题

x86 源码在迁移到鲲鹏平台时可能出现的功能性问题有以下几种：

C/C++双精度浮点型数转整型数据时，如果超出了整型的取值范围，鲲鹏处理器的表现与 x86 平台的表现不同。x86（指令集）中的浮点到整型的转换指令，定义了一个 indefinite integer value——“不确定数值”（64bit：0x8000000000000000），大多数情况下 x86 平台确实都在遵循这个原则，但是在从 double 向无符号整型转换时，又出现了不同的结果。鲲鹏的处理则非常清晰和简单，在上溢出或下溢出时，保留整型能表示的最大值或最小值。

对结构体中的变量进行原子操作时程序异常。鲲鹏处理器对变量的原子操作、锁操作等用到了 ldaxr、stlxr 等指令，这些指令要求变量地址必须按变量长度对齐，否则执行指令会触发异常，导致程序 coredump。一般是因为代码中对结构体进行强制字节对齐，导致变量地址不在对齐位置上，对这些变量进行原子操作、锁操作等会触发问题。代码中搜索“#pragma pack”关键字（该宏改变了编译器默认的对齐方式），找到使用了字节对齐的结构体，如果结构体中变量会被作为原子操作、自旋锁、互斥锁、信号量、读写锁的输入参数，则需要修改代码保证这些变量按变量长度对齐。

2.4.3 常见工具问题

代码迁移工具不同的分析任务需要将待分析软件包放置不同的服务器路径下。软件迁移评估需要在 package 目录下，源码分析需要在 sourcecode 目录下，软件包重构需要在 packagerebuild 目录下。

maven 软件仓库编译错误：无法找到依赖库。需要修改 maven 安装路径下的 conf/settings.xml，在 mirror 标签中设置 maven 远程仓库路径。

2.5 鲲鹏开发者支持套件

2.5.1 鲲鹏开发套件 DevKit

在鲲鹏社区首页找到开发者，选择技术主题：鲲鹏开发套件，可以找到这一系列简化我们做原生开发或者 x86 向鲲鹏迁移、迁移后测试调优操作的工具。鲲鹏开发套件面向全研发作业流程，提供涵盖代码迁移、开发调试、编译、测试、调优及诊断等各环节的开发使能工具，提升应用迁移和调优效率，加速原生开发，方便开发者快速开发出鲲鹏亲和的高性能软件。除了前面介绍过的代码迁移工具和动态二进制翻译工具外，鲲鹏开发套件还包括鲲鹏开发框架插件、鲲鹏编译调试插件、毕昇编译器、毕昇 JDK、GCC for openEuler、云测服务、鲲鹏性能分析工具。

鲲鹏开发框架插件是一款使能用户在鲲鹏服务器上进行开发的工具。鲲鹏开发框架充分利用鲲鹏平台的各类型算力及性能更优的第三方组件，提供鲲鹏工程向导、启发式编程、代码亲和检查等能力，一键引入鲲鹏加速库、快速构建鲲鹏应用软件框架，帮助开发者更便捷地开发鲲鹏应用，使能开发者高效创新。

鲲鹏编译调试插件一键安装即插即用，支持 CPU 应用和 GPU 应用的多种调试方式，以及鲲鹏平台远程调试功能。用户可以在操作中设置断点、查看线程、函数堆栈、寄存器信息或变量信息，支持汇编指令的断点执行和单步调试；提供图形化界面，大幅提升调试效率。

毕昇编译器基于开源 LLVM 开发，并进行了优化和改进，同时支持 Fortran 语言前端，是针对鲲鹏平台的高性能编译器。除 LLVM 通用功能和优化外，对中端及后端的关键技术点进行了深度优化，并集成 Auto-tuner 特性支持编译器自动调优。

毕昇 JDK 基于 OpenJDK 开发，是一个高性能、可用于生产环境的 OpenJDK 发行版。毕昇 JDK 运行在华为内部多个产品上，积累了大量使用场景和 Java 开发者反馈的问题和诉求，解决了业务实际运行中遇到的多个问题，并在 ARM 架构上进行了性能优化。毕昇 JDK 运行在大数据等场景下可以获得更好的性能。毕昇 JDK 8 与 Java SE 标准兼容，目前支持 Linux/aarch64 和 Linux/x86_64 平台。毕昇 JDK 同时是 OpenJDK 的下游，会持续稳定为 OpenJDK 社区做出贡献。

GCC for openEuler 是基于开源 GCC 开发的编译器工具链（包含编译器，汇编器，链接器），在 openEuler 社区开源发布，并通过鲲鹏社区免费提供二进制包，支持 aarch64 处理器架构。

云测服务面向开发者提供基于鲲鹏平台的兼容性测试、可靠性测试、安全测试、功能测试、性能测试服务功能，帮助您快速识别和定位应用程序在运行阶段的问题。

性能分析工具支持鲲鹏平台上的系统性能分析、Java 性能分析和系统诊断，提供系统全景及常见应用场景下的性能采集和分析功能，并基于调优专家系统给出优化建议。同时提供调优助手，指导用户快速调优系统性能。

2.5.2 鲲鹏应用使能套件 BoostKit

鲲鹏应用使能套件帮助我们提升业务应用在鲲鹏平台的性能表现，共享 8 大场景最佳实践，从硬件、基础软件到应用开展全栈优化（包括算法优化、软硬协同、原生架构独创技术等），提供高性能开源组件、基础加速软件包、应用加速软件包，使能应用极致性能。

鲲鹏 BoostKit 大数据使能套件：https://www.hikunpeng.com/zh/developer/boostkit/big-data。

鲲鹏 BoostKit 分布式存储使能套：https://www.hikunpeng.com/zh/developer/boostkit/sds。

鲲鹏 BoostKit 数据库使能套件：https://www.hikunpeng.com/zh/developer/boostkit/database。

鲲鹏 BoostKit 虚拟化使能套件：https://www.hikunpeng.com/zh/developer/boostkit/virtualization。

鲲鹏 BoostKit ARM 原生使能套件：https://www.hikunpeng.com/zh/developer/boostkit/arm-native。

鲲鹏 BoostKit Web 使能套件：https://www.hikunpeng.com/zh/developer/boostkit/web。

鲲鹏 BoostKit CDN 使能套件：https://www.hikunpeng.com/zh/developer/boostkit/cdn。

鲲鹏 HPC：https://www.hikunpeng.com/zh/developer/hpc。

3 应用性能测试与调优

3.1 软件性能测试

软件性能测试和软件开发虽都与软件相关，但它们的目的和内容却存在着明显区别。下面来看看这两者之间的区别：

- **目的不同**：软件开发的目的是为了开发出具有特定功能的软件，并且确保软件的正确性和可靠性。而软件性能测试的目的则是为了测试软件在各种情况下的性能表现，以便确定软件是否能够在实际应用中高效、稳定地运行。
- **内容不同**：软件开发的内容主要包括需求分析、设计、编码和测试等。而软件性能测试的内容则主要包括响应时间测试、负载测试、并发测试、安全测试等。
- **测试流程不同**：软件开发的测试流程是以开发为主，测试是其中的一个环节。而软件性能测试则是测试为主，测试是整个流程的核心环节。软件性能测试需要在软件开发完成后进行，以便对软件的性能进行测试和验证。

软件开发的目的是为了开发出满足用户需求的软件，而软件性能测试的目的则是为了测试软件在各种情况下的性能表现，确保软件能够高效、稳定地运行。

3.1.1 性能测试方法与工具

了解软件性能测试与软件开发的区别后，来探讨下软件性能测试的目的。

软件性能测试的目的，简单来看，可以罗列为以下几个方面：

1. 评估软件系统能力：验证软件系统负荷与响应时间的关系，是否符合需求，并帮助开发管理人员决策。
2. 评估软件系统抗压能力。
3. 识别软件系统在压力的情况下，是否有未知的 bug，包括但不限于：
   - 压力下，软件响应非常缓慢，无法提供服务。
   - 压力下，系统程序退出，数据丢失。
   - 压力降低后，程序无法自愈，服务无法恢复。
4. 评估软件系统耐久性能力：验证软件系统长时间压力运行的稳定性以及可靠性。

实际测试环节，又有哪些方法论来指导软件性能测试呢？

软件性能测试的过程通常包括测试计划制定、测试环境搭建、测试设计、测试执行和测试报告等阶段。

在测试计划制定阶段，需要确定测试范围、测试目标、测试方法和测试时间等。

在测试环境搭建阶段，需要创建测试环境，包括硬件和软件配置等。

在测试设计阶段，需要设计测试方案和测试用例。

在测试执行阶段，需要执行测试用例并记录测试结果。

在测试报告阶段，需要对测试结果进行分析和总结，并向相关人员汇报测试结果。

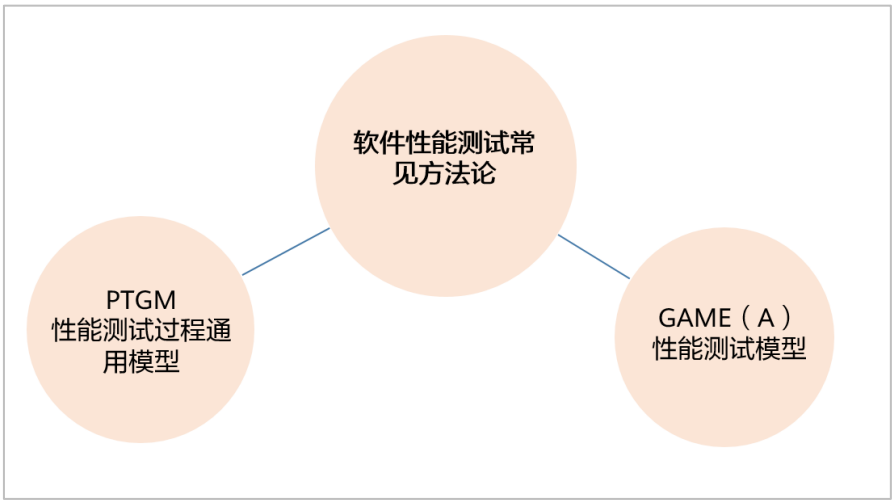


图3-1 性能测试方法论

在过去 40 多年软件工程发展中，积累了 PTGM 和 GAME（A）等系统的软件测试方法论。

PTGM（Performance Testing General Model）是一种通用性能测试方法，它分为以下 5 个阶段：

1. 测试前期准备：系统基础功能验证、组建测试团队、测试工具需求确认、性能预备测试。
2. 测试工具引入：工具选择、工具应用技能培训、确定工具应用过程。
3. 测试计划：性能测试领域分析、用户活动剖析与业务建模、确定性能目标、制定测试时间计划。
4. 测试设计和开发：测试环境设计、测试场景设计、测试用例设计、脚本与辅助工具开发。
5. 测试执行和管理：建立测试环境、部署测试脚本和测试场景、执行测试和记录结果。

性能测试 GAME（A）模型，包含以下五个重要项目：

目标（Goal）：确定本次测试目标，选择测试设计策略。

分析（Analysis）：分析性能需求，分析系统架构。

度量（Metrics）：场景的定义、事务的定义、虚拟用户的 pass/fail 的标准。

执行（Execution）：环境、数据、脚本的准备，场景及监控器的运行，生成报告。

调整（Adjust）：应用程序的修改，中间件的调优。

了解了系统的软件测试方法论，接下来为大家介绍一款强大软件测试工具-LoadRunner。

LoadRunner 是一款常用的性能测试工具，可以模拟用户在不同负载下的行为，以便测试软件性能表现。LoadRunner 支持不同类型的测试，包括 Web 测试、数据库测试、ERP 测试等。LoadRunner 使用脚本进行测试，脚本可以使用 LoadRunner 录制器进行录制，也可以手动编写。在测试完成后，LoadRunner 会生成详细的测试报告，包括测试结果、性能指标和问题分

析等。通过使用 LoadRunner 进行性能测试，可识别软件性能瓶颈，并为软件的优化和改进提供依据。

3.1.2 常用软件性能测试案例

前面介绍了软件性能测试的基础方法论和一款系统工具 LoadRunner，本节将为大家介绍常用软件性能测试案例。



图3-2 常用性能测试工具

如图所示，从互联网上检测常用性能测试工具，将工具的使用频度绘制成词云，图中测试工具的字体越大，代表使用的频次越高，不妨罗列出常使用的几款工具：

1. LoadRunner
2. JMeter
3. Mysqlslap

这些工具的用途是什么？在实际场景中又该如何使用呢？下面来为大家介绍

LoadRunner 是由 HP 开发的 Web 服务器性能测试工具，体系庞大，功能丰富，可以提供各种传输协议以分析服务器性能。

Jmeter 是 java 平台的性能开源测试工具，功能强大，方便易用。

MySQLslap 是 MySQL 自带的基准测试工具，优点：查询数据，语法简单，灵活容易使用。MySQLslap 为 MySQL 性能优化前后提供了直观的验证依据，建议系统运维和 DBA 人员应该了解常见压力测试工具，才能处理线上数据库支撑的用户流量上限及其抗压性等问题。

在实际工作中，通常会粗粒度的划分测试场景，例如 web 服务器、数据库。

- Web 服务器

常见的 web 服务器有 Nginx、Tomcat。

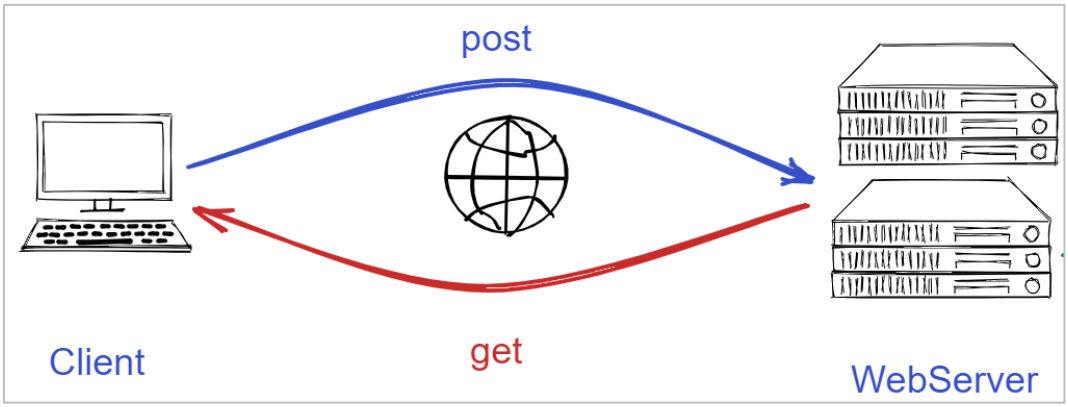


图3-3 web 服务器

Nginx 是一款高性能 Web 和反向代理服务器，是一款非常流行的开源软件。Nginx 的设计目标是高并发、高性能、低内存占用和高可靠性。Nginx 可以支持多种协议，如 HTTP、HTTPS、SMTP、POP3 和 IMAP 等。Nginx 的反向代理功能可以将请求转发给后端应用服务器，以便实现负载均衡和高可用性。Nginx 也可以作为静态资源服务器，以提供静态文件的快速访问。Nginx 的配置非常灵活，可以通过配置文件对其进行各种定制化的配置。Nginx 的高性能和灵活性使其成为 Web 服务器和反向代理服务器领域的佼佼者。

Tomcat 是一款流行的 Java Web 应用服务器，它由 Apache 软件基金会开发和维护。Tomcat 的主要功能是为 Java Web 应用程序提供 Servlet 和 JSP 容器。它能够处理 HTTP 请求，将请求转发给后端的 Java Web 应用程序，然后将响应返回给客户端。Tomcat 还支持 SSL/TLS 加密和多种认证方式，以提高 Web 应用程序的安全性。Tomcat 也可以与其他 Web 服务器如 Apache HTTP 服务器配合使用，以提供更高效、更安全的 Web 服务。

- Web 服务器测试方法、指标

针对上述两种开源 web 服务器工具，可使用 Wrk 来进行性能测试。wrk 是一款开源的 HTTP 性能测试工具，它可以模拟多个并发用户对 Web 应用程序进行访问，并对 Web 应用程序的性能进行评估。wrk 的主要特点是高并发、高性能和易于使用。wrk 使用 Lua 脚本进行测试，用户可以编写自己的测试脚本，以适应不同的测试需求。wrk 的测试结果包括请求的吞吐量、延迟、错误率等指标，可以帮助用户识别 Web 应用程序的性能瓶颈，并进行优化和改进。由于 wrk 的高效性和易用性，它已被广泛应用于 Web 应用程序性能测试领域。

wrk 测试 http 服务器时，需要关注以下参数：

1. 并发数（-c）：模拟的并发用户数，一般建议设置为机器 CPU 核数的两倍。
2. 连接数（-t）：模拟的连接数，一般建议设置为并发数的两倍。
3. 持续时间（-d）：测试的持续时间，一般建议设置为至少 1 分钟。
4. 请求速率（-R）：每秒钟发送的请求数，一般建议根据实际情况进行调整。
5. 测试脚本：根据实际情况编写测试脚本，以模拟真实的业务场景。

通过设置以上参数，可以对 http 服务器进行全面、有效、高效的性能测试，并为优化和改进提供依据。

wrk 的测试结果可从以下几个指标来评估：

1. 请求速率：每秒发送的请求数，这个指标越高越好。

2. 吞吐量：每秒钟处理的请求数，这个指标越高越好。

3. 延迟：请求的响应时间，这个指标越低越好。

4. 错误率：请求的错误率，这个指标越低越好。

通过比较测试结果中不同参数的表现，可以识别性能瓶颈并进行优化和改进。

- 数据库

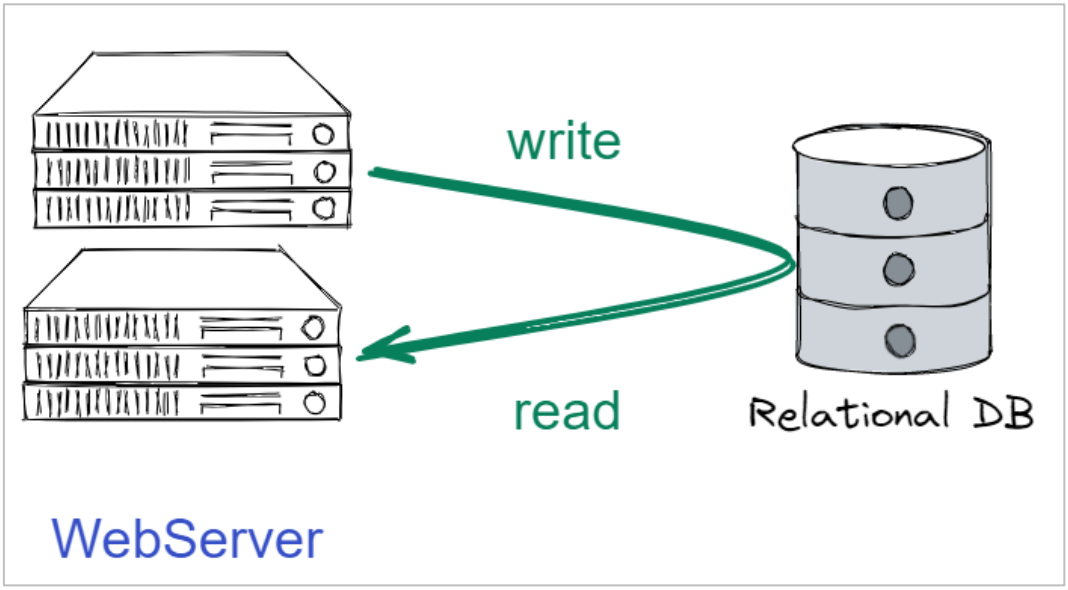


图3-4 数据库

数据库是一种用于存储和管理数据的软件系统。它可以存储各种类型的数据，如文本、数字、图像、音频和视频等。数据库通常由一个或多个表组成，每个表包含各种数据记录。每条记录包含多个属性，如名称、年龄、地址等。数据库还提供了一些查询功能，以便用户可以检索、排序和过滤数据。常见的数据库软件包括 Oracle、MySQL、Microsoft SQL Server 和 PostgreSQL 等。数据库在各种应用程序中得到广泛应用，如企业管理、电子商务、社交媒体、游戏等。

MySQL 是一种流行的开源关系型数据库管理系统（RDBMS），由 Oracle 公司维护和开发。MySQL 支持多种操作系统，如 Linux、Windows 和 macOS 等，以及多种编程语言，如 PHP、Java、Python 和 C++等。MySQL 是一个成熟、稳定、高性能的数据库系统，具有以下特点：

开源、免费：MySQL 是一种开源软件，可以免费使用和修改。

高性能、高可靠性：MySQL 支持多种优化技术，如索引、分区、缓存等，以提高数据库的性能和可靠性。

安全、易用：MySQL 提供了多种安全功能，如加密、访问控制、备份和恢复等，以保护数据库的安全。同时，MySQL 也提供了多种易用的工具和接口，如 MySQL Workbench、phpMyAdmin 和 JDBC 等。

MySQL 在各种应用场景中得到广泛应用，如 Web 应用程序、企业管理系统、电子商务系统等。MySQL 已成为世界上最流行的开源数据库之一，拥有广泛的用户群体和社区支持。

PostgreSQL 作为另一款常用的开源关系型数据库，其使用场景和 mysql 有相同之处，但也有着显著的差别：

1. 开发历史：MySQL 最初是由瑞典的 MySQL AB 公司开发的，后来被 Sun Microsystems 收购，现在属于 Oracle 公司。而 pgsql 最初是由加拿大的计算机科学家 Michael Stonebraker 等人开发的，现在由 PostgreSQL Global Development Group 维护和开发。

2. 许可证：MySQL 使用 GPL 和商业许可证进行许可，而 pgsql 使用 BSD 许可证。

3. 数据类型：MySQL 和 pgsql 都支持各种数据类型，如整数、浮点数、日期和时间等。但 pgsql 支持更多的数据类型，如数组、范围、网络地址和 JSON 等。

4. 性能：MySQL 和 pgsql 在性能方面都表现良好，但它们优化性能的方式有所不同。MySQL 使用索引、缓存和分区等技术来提高性能，而 pgsql 则使用继承、视图和触发器等技术来提高性能。

5. 扩展性：MySQL 和 pgsql 都支持水平和垂直扩展。但 pgsql 在水平扩展方面表现更好，它使用分片和复制等技术来实现数据的水平扩展。

6. 兼容性：MySQL 和 pgsql 都支持 SQL 标准，但它们在某些方面的实现不同。例如，pgsql 支持更多的 SQL 标准功能，如窗口函数、递归查询和 WITH 语句等。

OpenGauss 数据库的原型是基于 pgsql。



图3-5 OpenGauss 数据库

- 数据库的测试方法、指标

sysbench 是一款流行的开源基准测试工具，它可以模拟多个并发用户对系统进行测试，并评估系统的性能和稳定性。sysbench 支持多种测试模式，如 CPU、内存、磁盘、文件 IO 和数据库等。其中，数据库测试模式是 sysbench 最为流行的测试模式之一，它可以模拟多个并发用户对数据库进行访问，并测量数据库的吞吐量、延迟和错误率等指标。

sysbench 支持多种数据库系统，如 MySQL、PostgreSQL、MariaDB 和 MongoDB 等。sysbench 使用 Lua 脚本进行测试，用户可以编写自己的测试脚本，以适应不同的测试需求。sysbench 的测试结果包括请求的吞吐量、延迟、错误率等指标，可以帮助用户识别系统的性能瓶颈，并进行优化和改进。

sysbench 的测试参数包括：

1. 并发数（--threads）：模拟的并发用户数，一般建议设置为机器 CPU 核数的两倍。

2. 测试时间（--time）：测试的持续时间，一般建议设置为至少 1 分钟。

3. 数据库大小（--db-size）：测试时使用的数据库大小，一般建议根据实际情况进行调整。

4. 测试脚本：根据实际情况编写测试脚本，以模拟真实的业务场景。

通过设置以上参数，可以对系统进行全面、有效、高效的性能测试，并为优化和改进提供依据。

sysbench 的测试结果包括：

1. 吞吐量：每秒钟处理的请求数，这个指标越高越好。
2. 延迟：请求的响应时间，这个指标越低越好。
3. 错误率：请求的错误率，这个指标越低越好。

3.2 鲲鹏通用性能调优方法

3.2.1 性能调优

在第二章中，详细介绍了软件从 x86 平台迁移到鲲鹏平台的 5 个步骤，不知道大家还几步记得性能调优是第几个步骤呢？答案是第四步！



图3-6 软件迁移的五个阶段

为什么说在鲲鹏平台做软件迁移，性能调优是必要的过程，其原因在于：大部分程序在 x86 平台的工作过程与鲲鹏平台存在底层差异。

大家可以回忆下具体有哪些差异?

软件性能测试和调优是同一主题的两个阶段，只有通过测试，才能了解软件当前性能状态，在业务指标明确的前提下，通过软硬件调优，来达到软件性能优化的目的。

下面介绍鲲鹏通用性能调优方法。之所以说是通用方法，是因为不同软件系统在开发过程中代码逻辑和方法调用上存在固有差异，这导致实际运行中软件的瓶颈点各异，掌握通用的调优方法，可以解决绝大部分遇到的问题，至于极端的代码层面的问题，如 java 程序中的死锁，后续可借助专用的鲲鹏性能分析工具来定位和优化。

通用性能调优方法，从计算、存储、网络三个资源层面展开，调优的逻辑是，先找到谁占用的资源多，是正常占用还是异常占用，如果是正常占用，则需要考虑横向扩展，增加硬件服务能力；如果是异常占用，则需要找到、分析具体原因，通过合理方法来完成调优。

3.2.2 CPU 和内存子系统性能调优

性能优化思路如下：

如果 CPU 利用率不高，说明资源没有充分利用，可以通过工具（如 perf）查看应用程序阻塞在哪里，网络或应用程序业务逻辑是否有休眠或信号等待，这些优化措施在其他章节描述。

如果 CPU 利用率高，可以选择更好的硬件，优化硬件的配置参数来适配业务场景，或者通过优化软件来降低 CPU 占用率。

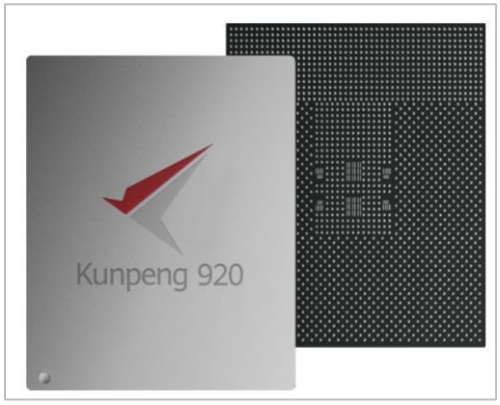


图3-7 鲲鹏 920 处理器

perf 是一款 Linux 系统性能分析工具，可以对 CPU、内存、磁盘等进行性能分析。其中，perf list 可以列出支持的事件类型，perf stat 可以统计各种事件的数量和占比，perf record 可以记录事件数据，perf report 可以生成报告并展示性能数据。对于应用程序阻塞点的分析，可以使用 perf record 和 perf report 命令进行分析。具体步骤如下：

1. 使用 perf list 命令列出支持的事件类型，找到适合的事件类型。

2. 使用 perf record 命令记录事件数据，例如：perf record -e sched:sched_stat_wait -g --sleep 10。

3. 在应用程序阻塞时，使用 Ctrl+C 停止 perf record 命令。

4. 使用 perf report 命令生成报告并展示性能数据，例如：perf report --sort=dso,comm,symbol。

top 是一款常用的 Linux 系统监控工具，可以实时显示系统的各项指标和进程信息。top 可以显示 CPU、内存、磁盘、网络等系统资源的使用情况，以及各个进程的 CPU、内存、IO 等指标。top 默认按 CPU 使用率排序进程，并以交互式的方式实时更新显示。用户可以通过 top 命令了解系统的负载情况，以及进程的运行情况，并通过 top 的交互式命令进行操作和调整。

top 的常用参数包括：

d：指定 top 的更新时间间隔。

p：指定 top 只显示指定进程 ID 的信息。

u：指定 top 只显示指定用户名的进程信息。

c：显示进程的完整命令行信息。

H：显示进程的线程信息。

除了常用参数之外，top 还支持多种交互式命令，如：

k：向指定进程发送信号。

r：修改指定进程的优先级。

z：将指定进程挂起或恢复。

f：切换显示进程的信息字段。

o：指定排序字段和排序方式。

q：退出 top。

通过交互式命令，用户可以对进程进行操作和调整，以满足不同的需求。top 是一款非常实用的系统监控工具，可以帮助用户了解系统的状态和进程的运行情况，以及进行系统优化和调整。

top 的输出结果包括系统总体的负载情况和各个进程的运行情况。在系统总体的负载情况中，可以查看系统的总体负载、CPU 使用率、内存使用率、交换分区使用率等信息，并可以根据需要进行排序和过滤。在进程运行情况中，可以查看进程 PID、用户、CPU 使用率、内存使用率、进程状态等信息，并可以根据需要进行排序和过滤。用户可以通过 top 的交互式命令进行操作和调整，以满足不同需求。

除了 top 之外，Linux 还有一些其他的系统监控工具，如 vmstat、sar、iostat 等，它们可以提供更详细和全面的系统性能信息，以帮助用户了解系统的状态和性能瓶颈，并进行优化和调整。用户可以根据自己的需求选择合适的系统监控工具，并根据输出结果进行优化和调整，以提高系统的性能和稳定性。

第一章介绍了鲲鹏处理器的多核特性，结合本节 CPU 和内存调优，有必要了解下 NUMA。

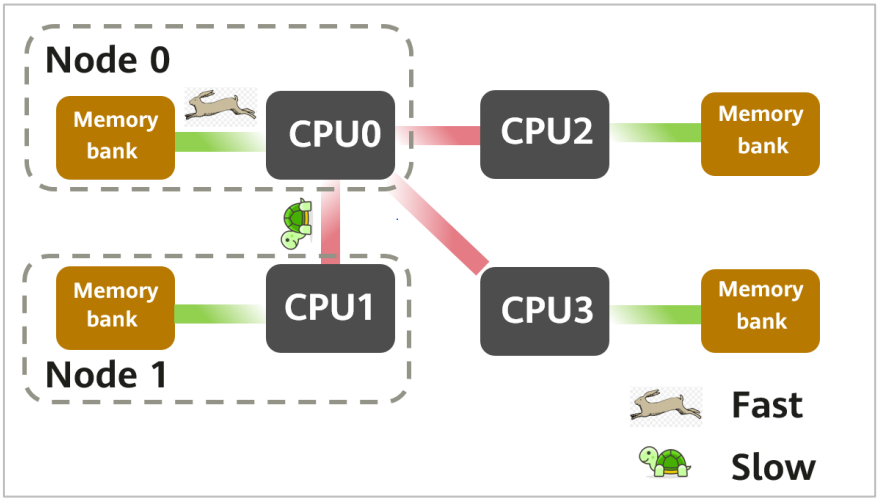


图3-8 NUMA 架构示意图

NUMA（Non-Uniform Memory Access）是一种计算机系统架构，它将内存分为多个区域，并将每个区域分配给一个或多个处理器。这种架构可以提高系统的可扩展性和性能，特别是在多处理器系统中。每个处理器只能访问其本地区域内的内存，但是处理器之间需要通过高速互联网络进行通信和访问其他区域的内存。

NUMA 架构的优点在于，它可以将内存和处理器分布在不同的节点上，避免了传统的共享内存架构中的竞争和瓶颈问题，从而提高了系统的效率和并发能力。此外，NUMA 架构还可以根据应用程序的访问模式动态地调整内存分配和处理器分配，以最大化系统的性能和资源利用率。

然而，NUMA 架构也存在一些缺点。首先，由于处理器分布在不同的节点上，需要进行额外的优化来避免内存碎片和数据不一致等问题。其次，由于需要通过高速互联网络进行访问，会增加系统的延迟和复杂度。最后，NUMA 架构的设计和实现也需要考虑处理器数量、内存大小和互联网络带宽等因素，这也增加了系统的设计和管理难度。

处理器的 NUMA 特性可通过 Numactl 来设置。

Numactl 是一款 Linux 系统工具，用于管理 NUMA 架构下的内存和处理器分配。使用 Numactl 命令可以将进程绑定到指定的 NUMA 节点上，并控制进程对内存的访问方式和策略。Numactl 支持多种操作模式和参数，如：

-hardware：显示系统的 NUMA 架构和节点信息。

-show：显示当前进程的 NUMA 策略和内存分配情况。

-cpubind：将进程绑定到指定的 CPU 节点上。

-membind：将进程分配内存到指定的 NUMA 节点上。

-interleave：将进程的内存分配在所有 NUMA 节点之间交错分配。

-preferred：将进程的内存分配在指定的 NUMA 节点上，如果无法分配，则在其他节点上分配。

通过使用 Numactl 命令，用户可以优化应用程序的性能和资源利用率，特别是在多处理器系统中。Numactl 是一款非常实用的系统工具，可以帮助用户了解系统的 NUMA 架构和性能特征，并进行优化和调整。

3.2.3 网络子系统性能调优

对于开发人员来说，网络子系统是容易忽略的性能调优模块，因为网络中有大量硬件，以互联网流量为例，先来看看计算机外部需要经过哪些硬件模块：

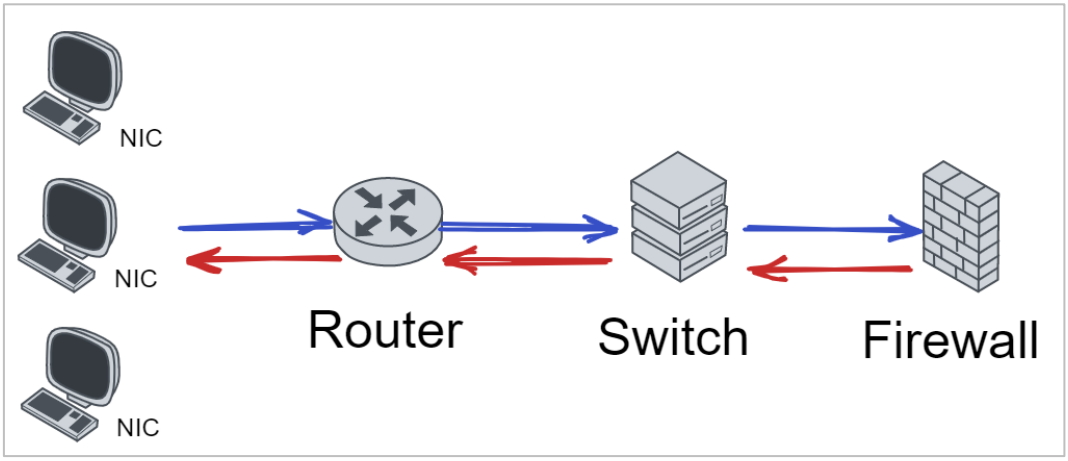


图3-9 计算机外部的流量访问过程

- 网卡（Network Interface Card，NIC）：网卡是计算机和网络之间的接口，负责将数据从计算机发送到网络，并将网络接收到的数据传输到计算机。它通常包含一个网口（Ethernet Port），用于连接计算机和交换机、路由器等网络设备。网卡还可以支持不同

的物理层协议，如以太网、Wi-Fi、蓝牙等，以满足不同的网络需求。计算网络流量时，网卡是数据传输的起点和终点，也是性能优化的关键。

- 路由器（Router）：路由器是连接不同网络的设备，负责在不同的网络之间进行数据传输和转发，将数据从源地址路由到目标地址。路由器通常具有多个网口，可以同时连接多个网络，并支持路由选择算法，根据网络拓扑和路由表选择最优路径进行数据传输。在大型企业和互联网上，路由器是网络基础设施的核心组件，可以实现高效、可靠的数据传输和网络管理。
- 交换机（Switch）：交换机是在同一网络中进行数据传输和转发的设备，负责将数据从源地址交换到目标地址。与路由器不同，交换机不需要进行路由选择，只需要根据目标地址进行转发即可。交换机通常具有多个端口，可以连接多个计算机和其他网络设备，并支持各种流量控制和优先级设置，以保证网络性能和服务质量。
- 防火墙（Firewall）：防火墙是一种网络安全设备，负责检测和阻止网络攻击和恶意软件，保护网络安全。防火墙可以根据规则和策略对网络流量进行过滤和审核，防止未经授权的访问和数据泄露。防火墙通常包括网络边界防火墙、内部防火墙、主机防火墙等多种类型，可以根据不同的网络环境和需求选择合适的防火墙设备。

以上硬件模块相互配合，构成了网络基础设施，能够实现高效、安全、可靠的数据传输和存储。在实际应用中，不同的网络环境和需求可能需要不同的硬件模块组合，以满足不同的性能和安全要求。因此，在设计和实现网络基础设施时，需要进行充分的规划和测试，以确保网络的稳定性和可扩展性。

从计算机内部来看，流量包从网卡进入计算机系统后，会经过一系列环节才能被处理成为数据。

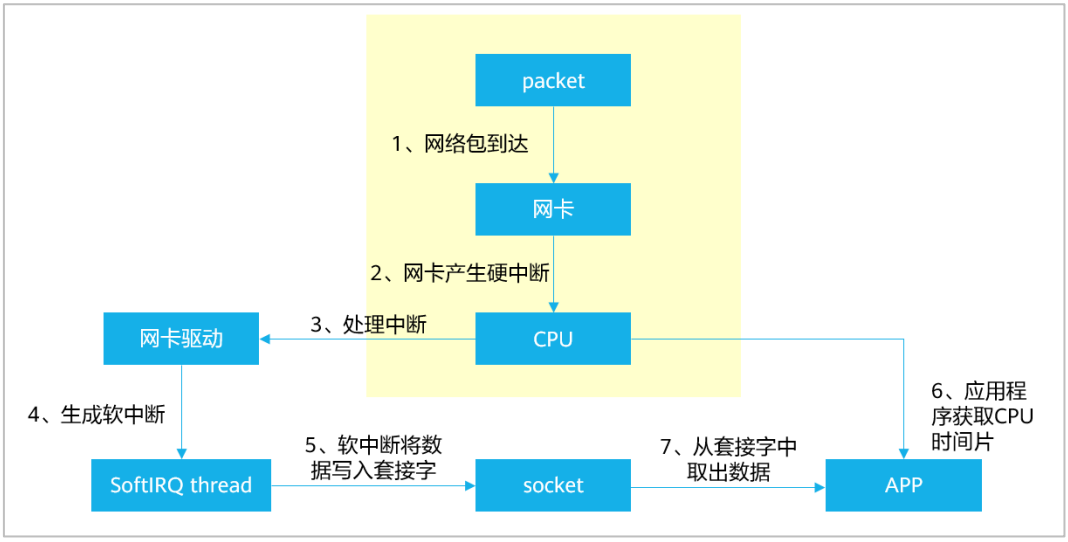


图3-10 流量包从网卡进入计算机系统

1. 首先，它会进入网络协议栈，该栈通常由不同的协议层组成，如物理层、数据链路层、网络层、传输层、应用层等。每个协议层都负责不同的功能和处理流程，如数据封装、路由选择、错误检测等。

2. 一旦流量包通过网络协议栈，就会被传递给相应的应用程序进行处理。应用程序可以是基于不同协议的网络服务，如 Web 服务器、电子邮件服务器、FTP 服务器等。应用程序会根据流量包的协议和内容进行不同的操作，如解析 HTTP 请求、发送 SMTP 邮件等。

3. 应用程序通常会使用系统调用来访问底层的网络接口和硬件设备。系统调用可以让应用程序直接控制网络流量和数据传输，如打开 socket、发送数据、接收数据等。系统调用会触发相应的网络驱动程序，该程序负责与网卡和其他网络设备进行通信和数据传输。网络驱动程序通常是操作系统内核的一部分，可以提供高效、可靠的数据传输服务。

4. 最终，流量包会到达网卡，网卡负责将流量包转换为电信号，并将其发送到网络中。网卡还可以接收从网络中传输过来的数据，并将其转换为流量包，传递给上述的处理流程。

在以上流程中，每个环节都需要高效、可靠地处理数据和流量包，以确保网络的稳定性和性能。因此，网络基础设施的设计和实现需要充分考虑各个环节的要求和特点。

除了这些环节之外，网络流量在处理过程中还需要考虑一些其他因素。例如，流量包的大小、传输速度、网络拓扑、安全防护等都会对网络性能和稳定性产生影响。此外，不同类型的网络服务也会对流量包的处理产生不同的要求，如实时性、可靠性、可扩展性等。因此，在设计和实现网络基础设施时，需要综合考虑各种因素，以满足不同的性能和安全要求。

了解了网络流量包从计算机外部硬件设备，到计算内部的网络协议栈的全过程，接下来介绍网络管理工具 ethtool。

ethtool 是一个用于查询和修改网络接口卡参数的命令行工具。它支持多种网络接口卡，如以太网卡、无线网卡、FDDI 卡等，并可以显示和修改各种接口卡的属性和状态。常用的 ethtool 命令包括：

```
ethtool -i <interface>
```

显示指定网络接口卡的驱动信息和版本号。

```
ethtool -p <interface> <duration>
```

让指定网络接口卡的 LED 灯闪烁一段时间，以便用户找到该接口卡的位置。

```
ethtool -S <interface>
```

显示指定网络接口卡的统计信息和性能数据，如接收和发送的流量、丢包率、错误率等。

```
ethtool -a <interface>
```

显示指定网络接口卡的自适应速率和双工模式设置。

```
ethtool -c <interface>
```

显示指定网络接口卡的 RX/TX 流控设置。

```
ethtool -g <interface>
```

显示指定网络接口卡的网卡缓冲区大小和描述符环大小。

```
ethtool -k <interface>
```

显示指定网络接口卡的卸载卸载特性，如 GRO、LRO、TSO 等。

```
ethtool -m <interface>
```

显示指定网络接口卡的光模块信息，如光功率、波长、传输距离等。

```
ethtool -r <interface>
```

重置指定网络接口卡的 RX 和 TX 错误计数器。

通过使用 ethtool 命令，用户可以了解和调整网络接口卡的各种属性和状态，以优化网络性能和稳定性。ethtool 是一个非常实用的系统工具，尤其是在网络故障排除和性能分析方面，可以帮助用户快速定位和解决网络问题。

3.2.4 磁盘 IO 子系统调优

介绍完计算、网络两个偏操作层面的优化场景，接下来为大家介绍磁盘 IO 子系统。

首先要理解为什么叫 IO 子系统，即 Input/Output，输入输出系统，服务器的计算资源主要用于处理代码逻辑，过程中要不断加工原始数据，那么数据和 CPU 计算资源之间的通路是影响业务的另一个关键因素。

- 多级存储

多级存储，是磁盘 IO 子系统中一个经典设计方法。

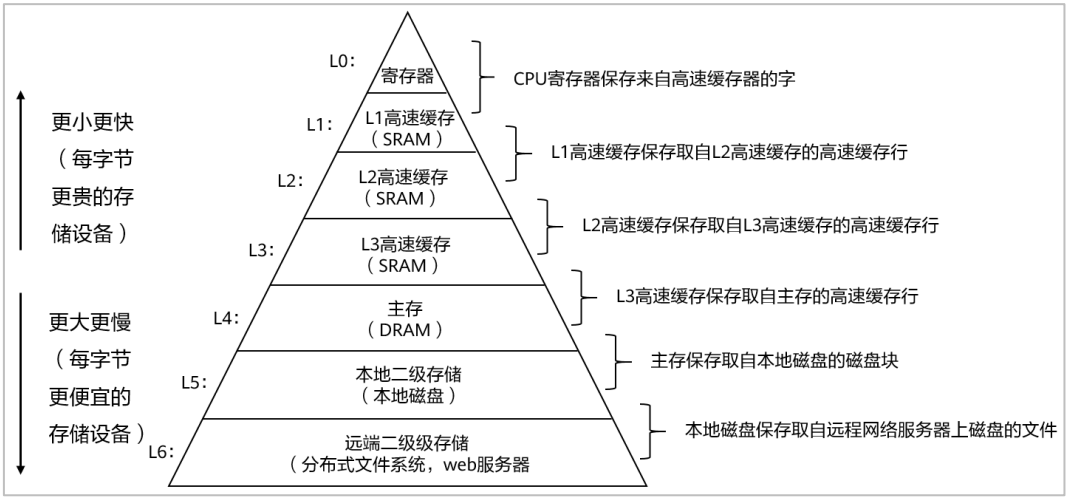


图3-11 多级存储架构

在多级存储中，高速缓存是最快的存储介质，它通常位于 CPU 内部或者 CPU 和内存之间，用于缓存最近经常访问的数据和指令，以提高 CPU 的运行速度。高速缓存分为 L1、L2、L3 等多个层次，每个层次的容量和访问速度不同，L1 缓存容量最小，速度最快，L3 缓存容量最大，速度最慢，但仍然比内存快得多。

内存是速度较快、容量较大的存储介质，通常用于存储应用程序和操作系统的数据和代码。内存的访问速度比磁盘快得多，但容量比磁盘小。内存的容量和访问速度也有多个层次，例如 DDR4、DDR3、DDR2 等，速度和容量随着层次的增加而降低。

磁盘是速度较慢、容量较大的存储介质，通常用于长期存储数据和文件。磁盘通常分为机械硬盘（HDD）和固态硬盘（SSD）两种类型，SSD 的访问速度比 HDD 快得多，但价格也更高。磁盘也有多个层次，例如 SATA、SAS、NVMe 等，速度和容量随着层次的增加而提高。



图3-12 华为 SSD 硬盘

多级存储的优点是可以提高数据访问速度和存储容量，同时降低存储成本和能耗。高速缓存可以加速 CPU 的运行，提高系统的响应速度；内存可以缓存应用程序和操作系统的数据，加快数据访问速度；磁盘可以长期存储数据和文件，同时提供较大的存储容量。缺点是需要管理多个存储层次，涉及到数据的迁移、备份和恢复等操作，需要额外的管理和维护成本。

多级存储在现代计算机系统中得到广泛应用，尤其是在大型数据库、云计算和大数据分析等领域，可以提高数据处理和分析的效率和性能。在数据中心中，通常采用多级存储来存储不同类型和访问频率的数据，以实现高效、可靠的数据存储和访问服务。同时，多级存储还可以通过数据迁移和备份等技术来提高数据的可靠性和可用性，保证数据的安全性和完整性。

多级存储是一种有效的存储技术，可以根据不同的访问需求和成本要求进行灵活配置和管理，以实现高效、可靠的数据存储和访问服务。随着计算机系统的不断发展和应用场景的不断扩展，多级存储将会发挥越来越重要的作用，为我们提供更加高效、安全和可靠的数据存储和访问服务。

除了多级存储的概念，还有一些影响存储效率的因素，比如：文件系统（FileSystem）、IO 调度策略、缓冲区（Buffer）。

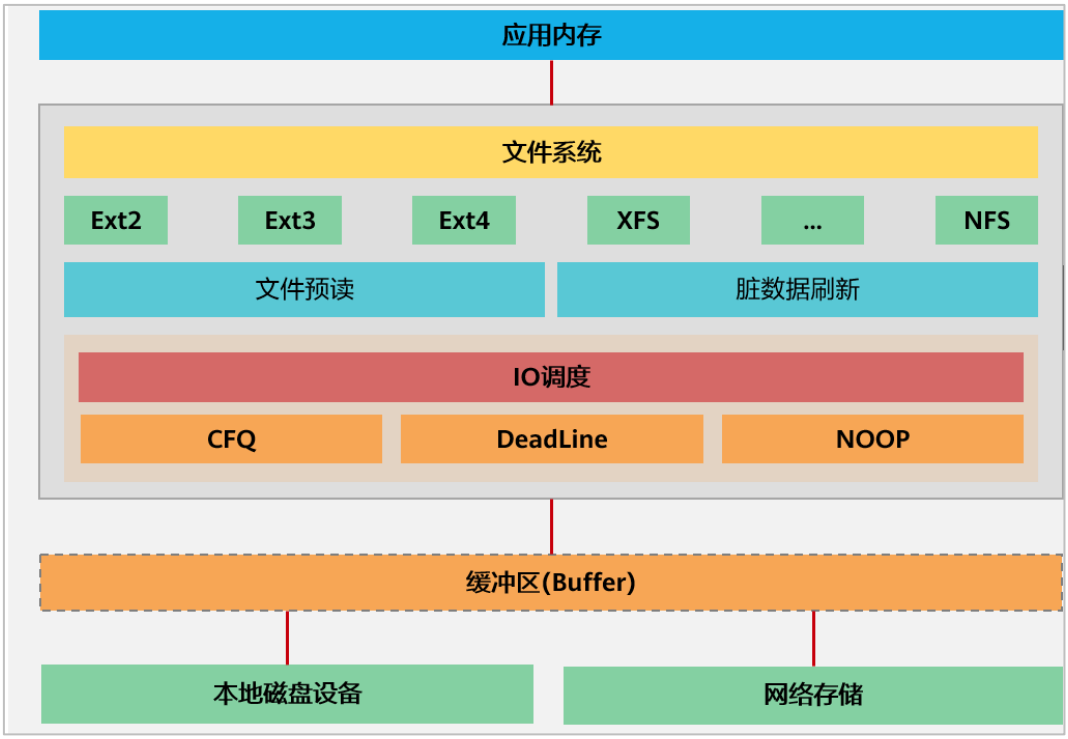


图3-13 IO 子系统内部原理

- 文件系统

文件系统是操作系统用来管理文件和目录的一种机制，它定义了文件和目录的组织方式、访问权限和安全策略等。不同的文件系统具有不同的特点和设计目标，因此选择合适的文件系统可以对存储效率和性能产生重要影响。

一般来说，文件系统的选择应该考虑以下因素：

存储介质类型：不同类型的存储介质（如磁盘、SSD 等）具有不同的性能和特点，因此需要选择适合该存储介质的文件系统，以获得最佳的性能和可靠性。

文件大小和数量：文件系统的设计和实现通常会对文件大小和数量产生影响，因此需要根据实际的存储需求来选择合适的文件系统。

并发访问：文件系统的并发访问能力通常会影响存储效率和性能，因此需要选择支持高并发访问的文件系统，以提高存储效率和性能。

安全性和数据完整性：文件系统的安全性和数据完整性是存储效率和性能的关键因素之一，因此需要选择具有良好安全性和数据完整性保护机制的文件系统，以确保数据的安全和完整性。

兼容性和可移植性：文件系统的兼容性和可移植性通常会影响存储效率和性能，因此需要选择具有良好兼容性和可移植性的文件系统，以便在多个操作系统和平台之间进行数据共享和传输。

文件系统的选择对存储效率和性能有重要影响，需要根据实际的存储需求和特点来选择合适的文件系统。同时，文件系统的设计和实现也需要考虑不同的存储介质、并发访问、安全性和数据完整性等因素，以提高存储效率和性能。

- IO 调度策略

常见的 IO 调度策略包括 CFQ、Deadline、NOOP 等。

CFQ 调度策略是基于队列的调度算法，它将访问请求按照进程的优先级和 I/O 负载来调度，以保证公平性和稳定性。CFQ 调度策略适用于多用户和多任务的环境下，可以有效地避免 I/O 阻塞和 I/O 饥饿现象。Deadline 调度策略是基于截止时间的调度算法，它将访问请求按照截止时间和 I/O 负载来调度，以提高 I/O 响应速度和吞吐量。Deadline 调度策略适用于需要快速响应的访问请求，可以有效地减少 I/O 延迟和提高系统的响应速度。NOOP 调度策略是一种简单的 FIFO 算法，它不进行任何调度，只按照先进先出的原则处理 I/O 请求。NOOP 调度策略适用于低负载和高吞吐量的环境下，可以有效地提高 I/O 性能和吞吐量。

选择合适的 IO 调度策略需要根据实际的存储需求和特点来进行评估和测试。通常，需要考虑以下因素：

I/O 访问模式和负载特点：不同的 I/O 访问模式和负载特点对于不同的 IO 调度策略有不同的影响，需要根据实际的访问模式和负载特点来选择合适的 IO 调度策略。

I/O 延迟和响应速度：对于需要快速响应的访问请求，需要选择能够提供最佳响应速度和最小延迟的 IO 调度策略。

I/O 吞吐量和性能：对于需要高吞吐量和高性能的访问请求，需要选择能够提供最大吞吐量和最高性能的 IO 调度策略。

系统负载和公平性：对于多用户和多任务的环境下，需要选择能够提供公平性和稳定性的 IO 调度策略，避免 I/O 阻塞和 I/O 饥饿现象。

数据可靠性和安全性：选择 IO 调度策略时需要考虑数据的可靠性和安全性，以保证数据的完整性和保密性。

选择合适的 IO 调度策略可以提高存储效率和性能，从而满足不同的存储需求和特点。在实际应用中，需要对不同的 IO 调度策略进行评估和测试，以选择最优的 IO 调度策略。同时，还需要进行系统优化和配置，以最大化 IO 调度策略的性能和效率。

- 缓冲区

缓存区（buffer）是一种用于临时存储数据的机制，它通常位于数据的源和目的地之间，用于缓存和调节数据的传输速度，以提高数据的传输效率和可靠性。缓存区可以存储一定数量的数据，当源和目的地之间的传输速率不一致时，缓存区可以调节数据的传输速率，以避免数据的丢失和重传。缓存区的大小和数量可以根据实际的传输需求和性能要求进行调整和配置，以最大化数据的传输效率和可靠性。

缓存区的大小和数量对存储效率有重要影响。通常来说，缓存区的大小应该足够大，以避免数据的丢失和重传。如果缓存区太小，当数据的传输速率超过缓存区的处理能力时，就会出现数据的丢失和重传，从而降低数据的传输效率和可靠性。另外，缓存区的数量也应该足够多，以处理多个数据流和多个数据源之间的传输。如果缓存区的数量太少，就会出现数据的阻塞和丢失，从而降低数据的传输效率和可靠性。

缓存区是一种提高数据传输效率和可靠性的重要机制，可以调节数据的传输速率和处理能力，以满足不同的传输需求和性能要求。缓存区的大小和数量可以根据实际的传输需求和性能要求进行调整和配置，以最大化数据的传输效率和可靠性。

那么磁盘 IO 子系统应该如何设计来提高业务系统数据的读取、写入能力呢?

具体策略要结合具体场景来分析，下面总结磁盘 IO 主要调优思路：

1. 减少磁盘读写：优化应用减少磁盘读写；优化脏数据刷新策略。

2. 减少阻塞：利用缓存，磁盘预取；利用异步读写机制。

3. 提高文件读写效率：选择合适的文件系统；选择合适的文件读写方式（fopen/open 和 directio）；选择合适的磁盘调度方式。

最后给大家介绍排查磁盘 IO 问题使用最频繁的工具 iostat。

iostat 命令是 Linux 系统中的一个性能分析工具，用于监测和报告系统的磁盘和 CPU 性能。iostat 命令可以实时地显示磁盘的 I/O 使用情况、CPU 的使用情况以及系统的负载情况等，以帮助管理员诊断和优化系统的性能问题。iostat 命令的输出包括以下几个方面的信息：

CPU 使用情况：显示 CPU 的使用率、上下文切换次数、中断次数等。

磁盘 I/O 使用情况：显示磁盘的 I/O 吞吐量、响应时间、等待时间等。

网络使用情况：显示网络的吞吐量、错误率、丢包率等。

系统负载情况：显示系统的负载情况、运行进程数等。

iostat 命令的使用方法如下：

```
iostat [-c] [-d] [-h] [-k] [-m] [-N] [-p] [-t] [-x] [interval [count]]
```

其中，参数 interval 表示输出的时间间隔，单位为秒；参数 count 表示输出的次数；其他参数详细说明如下：

- c：显示 CPU 使用情况。

- d：显示磁盘 I/O 使用情况。

- h：以人类可读的方式显示输出结果。

- k：以 KB 为单位显示输出结果。

- m：以 MB 为单位显示输出结果。

- N：显示 NFS 文件系统的使用情况。

- p：显示指定进程的磁盘 I/O 使用情况。

- t：显示输出结果的时间戳。

- x：显示扩展的磁盘 I/O 使用情况。

3.2.5 应用程序调优

前面几个小节，从计算、网络、存储三个资源层面为大家介绍了系统调优的主要思想和一般方法。

请记住，上述理论和方法是任何系统调优都会涉及的基础方面，并非鲲鹏平台专有，只不过鲲鹏处理器因为指令集架构差异，在优化实际业务系统的过程中，需要着重考虑、重新思考性能调优这一命题。

只有掌握基础理论，并在实际业务场景中不断积累实操经验，才能在性能优化的漫漫长路中找到最终出路。

在本小节中，将为大家从应用层面介绍鲲鹏平台的调优方法。主要分为：编译选项优化、Cacheline 优化、文件缓冲机制、Neon 指令。

与资源层面调优稍有不同，应用层面的调优，将更关注业务具体场景，所使用的调优方法也更加精细，请大家仔细阅读。

- 编译选项优化

编译选项（Compilation options）是编译器提供的一系列参数，用于控制编译过程中的各种行为和特性。编译选项可以影响编译的速度、优化级别、生成的代码质量等方面。通常，编译选项是通过命令行参数来指定的。编译选项的具体内容和含义通常与编译器的版本和类型有关。例如，GCC 编译器提供了大量的编译选项，包括优化选项、调试选项、警告选项等。不同的编译器可能会提供不同的编译选项，因此需要根据实际的编译需求和编译器类型来选择合适的编译选项。

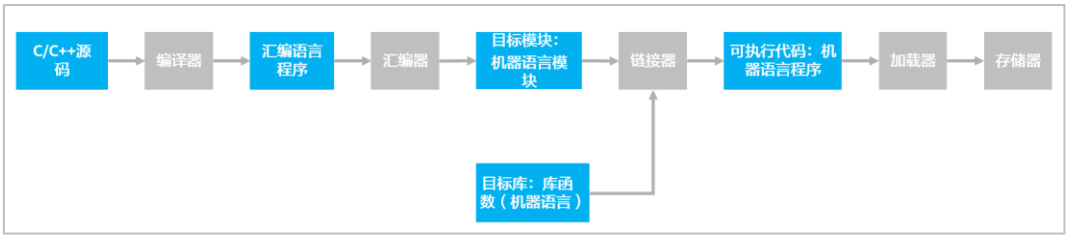


图3-14 从源码到可执行程序-编译型语言

常见的编译选项包括：

- 优化选项（Optimization options）：用于控制编译器的优化级别和优化策略，以提高生成代码的效率和性能。

- 警告选项（Warning options）：用于控制编译器的警告级别和警告类型，以帮助开发者检查和修复代码中的潜在问题。

- 调试选项（Debugging options）：用于控制编译器生成的调试信息和符号表，以便进行源代码级别的调试和跟踪。

- 目标选项（Target options）：用于指定编译器的目标平台和架构，以生成与目标平台兼容的代码。

- 库选项（Library options）：用于指定编译器链接的库文件和库路径，以便在编译过程中使用库中的函数和变量。

- 输入输出选项（Input/output options）：用于指定编译器的输入文件和输出文件，以及生成的目标文件的类型和名称。

编译选项是编译器提供的一系列参数，用于控制编译过程中的各种行为和特性。编译选项的具体内容和含义通常与编译器的版本和类型有关，需要根据实际的编译需求和编译器类型来选择合适的编译选项。

在鲲鹏平台，提供了适配底层架构的毕昇编译器，即 BiSheng Compiler。毕昇编译器除了具备开源 GCC 的编译能力外，还专门开发了适配鲲鹏处理器的增强特性：支持鲲鹏微架构芯片及指令优化，通过软硬协同提供相比开源 LLVM 更高的性能；集成 Auto-tuner 特性支持编译器自动调优。

更多关于毕昇编译器的介绍，可移步查阅鲲鹏社区文档-毕昇编译器(https://www.hikunpeng.com/document/detail/zh/kunpengdevps/compiler/ug-bisheng/kunpengbisheng_06_0001.html)。

- Cacheline 优化

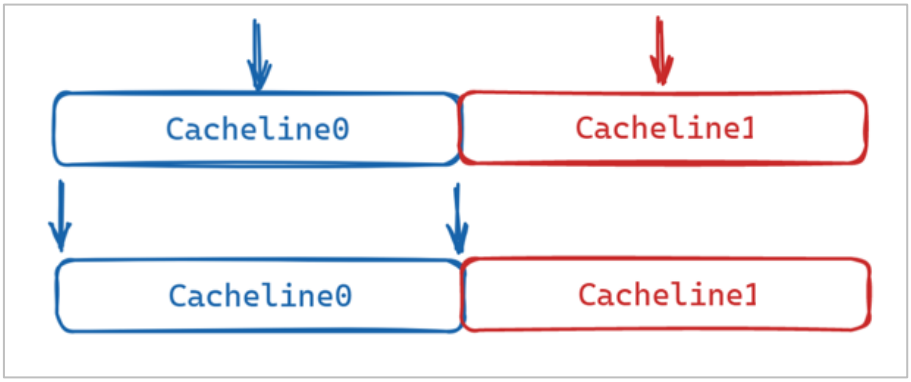


图3-15 Cacheline 优化

缓存行（Cacheline）是计算机体系结构中的一个概念，是一种存储器单元，用于缓存处理器访问的数据。缓存行通常由多个字节或字（byte/word）组成，其大小通常为 64 个字节或 128 个字节。缓存行的出现是为了解决处理器和主存之间速度不匹配的问题。在处理器访问主存时，由于主存速度较慢，处理器需要花费较长的时间来等待主存的响应。而缓存行的出现可以将部分主存数据缓存在处理器缓存中，从而提高数据的访问速度和效率。

缓存行的大小和数量对系统性能和效率有重要影响。通常来说，缓存行的大小应该足够大，以最大化缓存的效率和性能。如果缓存行太小，就会出现缓存失效的情况，从而降低数据的访问速度和效率。另外，缓存行的数量也应该足够多，以缓存更多的数据和提高数据的访问效率。如果缓存行的数量太少，就会出现缓存的竞争和冲突，从而降低数据的访问速度和效率。

缓存行是计算机体系结构中的一个重要概念，用于缓存处理器访问的数据，提高数据的访问速度和效率。缓存行的大小和数量对系统性能和效率有重要影响，需要根据实际的访问需求和性能要求来进行调整和配置。

Cacheline 对齐是鲲鹏处理器在内存使用效率上做出的改进，配合 openEuler 操作系统新内核的设计与使用，在虚拟化/java 内存消耗性应用场景，能优化系统程序的执行效率。

大部分处理器中 Cache 到主存是以 Cacheline（一般为 64Byte，也有地方称 Cache 块，本文统一使用 Cacheline）传输的，CPU 从内存加载数据是一次一个 Cacheline，CPU 往内存写数据也是一次一个 Cacheline。假设处理器首次访问数据对象 A，其大小刚好为 64Byte，如果数据对象 A 首地址并没有进行对齐，即数据对象 A 占用两个不同 Cacheline 的一部分，此时处理器访问该数据对象时需要两次内存访问，效率低。但是如果数据对象 A 进行了内存对齐，即刚好在一个 Cacheline 中，那么处理器访问该数据时只需要一次内存访问，效率会高很多。编译器可以通过合理安排数据对象，避免不必要地将它们跨越在多个 Cacheline 中，尽量使得同一对象集中在一个 Cache 中，进而有效地使用 Cache 来提高程序的性能。通过顺序分配对象，即如果下一个对象不能放入当前 Cacheline 的剩余部分，则跳过这些剩余的部分，从下一个 Cacheline 的开始处分配对象，或者将大小（size）相同的对象分配在同一个存储区，所有对象都对齐在 size 的倍数边界上等方式达到上述目的。

- 文件缓存机制选择

在磁盘 IO 子系统介绍了缓存区的概念，讨论了缓存区的大小和数量对数据存取的影响。除此之外，文件缓存机制的选择也直接影响文件/数据的存取效率，这里为大家介绍 Clib Buffer 和 PageCache 两种机制。

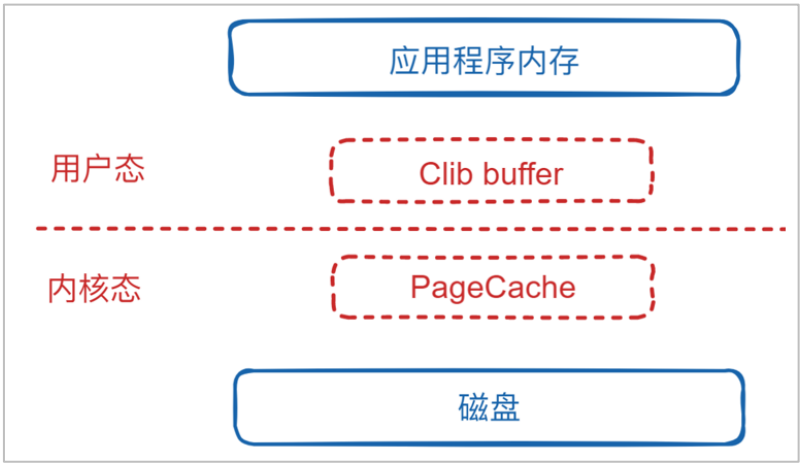


图3-16 文件缓存机制

Clib buffer：用户态的一种数据缓冲机制，开启时，数据从应用程序 buffer 拷贝至 clib buffer 后，不会立即将数据同步到内核，而是缓冲到一定规模或者主动触发，才会同步到内核；查询数据时，优先从 clib buffer 查询数据，从而减少用户态和内核态的切换。

PageCache：内核态的一种文件缓存机制，用户在读写文件时，先操作 PageCache，内核根据调度机制或者被应用程序主动触发时，会将数据同步到磁盘，从而减少磁盘访问。

了解两种文件机制的影响，应用程序需要根据具体场景来匹配缓存机制和文件读写方式。

那么有哪些文件读写方式呢？这里的文件读写方式，落到操作系统层面，即系统调用方法。

从内存拷贝次数维度来排序：fopen > open > direct io ≈ mmap

从系统调用次数维度来排序：open= direct io > fopen > mmap

为什么要了解这些系统调用对资源的消耗程度呢？主要是为了在具体业务场景，更合理的匹配，从而达到系统资源利用效率最大化的目的。

fread/fwrite 函数使用了 clib buffer 缓存机制，read/write 没有，因此 fread/fwrite 比 read/write 多一层内存拷贝，即从应用程序 buffer 至 clib buffer 的拷贝，但是 fread/fwrite 比 read/write 有更少的系统调用。对于每次读写字节数较大的操作，内存拷贝比系统调用占用更多资源，可以使用 read/write 来减少内存拷贝；对于每次读写字节数较少的操作，系统调用比内存拷贝占用更多资源，建议使用 fread/fwrite 来减少系统调用次数。

- Neon 指令

Intel 处理器在过去 50+多年的发展过程中，为提高单位面积处理器的算力，曾开发出一套 SIMD 技术，其最为知名的实现是 SSE（Streaming SIMD Extensions）指令集，包括 SSE、SSE2、SSE3 等多个版本，目前最新的是 AVX-512 指令集。这些指令集在 Intel 处理器中得到广泛应用，可以大大提高计算机的运算速度和效率。

SIMD（Single Instruction Multiple Data）技术是一种并行计算技术，其基本思想是通过一条指令处理多个数据，从而提高数据处理的效率和性能。SIMD 技术通常采用向量寄存器来存储多个数据，并使用特定的指令来对这些数据进行操作，如加、减、乘、除等。

在 SIMD 技术中，每个向量寄存器包含多个数据元素，可以在单个时钟周期内同时执行对这些元素的操作。这种并行计算方式使得 SIMD 技术在计算密集型应用中表现出色，例如图像处理、信号处理、科学计算等。

SIMD 技术的主要优点是可以在单个指令周期内对多个数据进行处理，从而大大提高了数据处理的效率和性能。此外，SIMD 技术还可以减少指令的数量和数据的传输量，从而降低了功耗和延迟。相比于传统的单指令单数据（SISD）计算方式，SIMD 技术可以大大提高计算机的运算速度和效率。

在实际应用中，SIMD 技术被广泛运用于多媒体领域，例如视频编解码、音频处理、图像处理等。此外，SIMD 技术还被用于科学计算、数据挖掘、人工智能等领域。

虽然 SIMD 技术带来了诸多优点，但也存在着一些缺点。首先，SIMD 技术需要特定的硬件支持，如果处理器不支持 SIMD 技术，则无法使用该技术。其次，使用 SIMD 技术需要特定的编程技术，对程序员的要求较高。此外，不是所有的计算任务都适合使用 SIMD 技术，对于一些分支较多或数据依赖性较强的任务，SIMD 技术的效果并不显著。

SIMD 技术是一种并行计算技术，可以通过一条指令处理多个数据来提高数据处理的效率和性能。SIMD 技术在多媒体领域、科学计算、数据挖掘等领域有着广泛的应用，但需要特定的硬件支持和编程技术。

鲲鹏处理器的多核特性，与生俱来就具备并行处理的能力，那么鲲鹏处理器能运行 SIMD 指令吗?

答案是不能！但能运行 Neon 指令。

Neon 技术是 ARM 架构下的 SIMD（Single Instruction Multiple Data）技术，可以通过一条指令处理多个数据来提高数据处理的效率和性能。Neon 技术最初是为移动设备的多媒体处理而设计的，如图像处理、音频处理等。Neon 技术的核心是 SIMD 寄存器，其大小为 128 位，可以存储多个数据元素。Neon 技术可以在单个时钟周期内同时对多个数据进行操作，从而实现高效的数据处理。

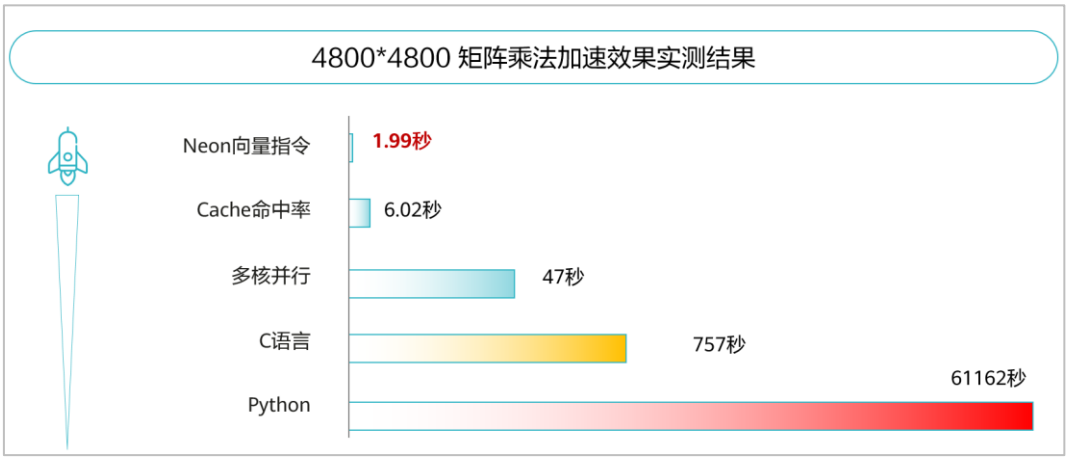


图3-17 Neon 指令执行效果

除了在移动设备的多媒体处理中应用，Neon 技术还被广泛应用于诸如游戏、虚拟现实、图像处理、数字信号处理等计算密集型应用中。Neon 技术的优势在于其具有低功耗、高性能、低延迟等特点，可以显著提高设备的性能和效率。

Neon 技术与其他 SIMD 技术相比，具有以下优势：

低功耗：Neon 技术采用低功耗的设计，可以在保证高性能的同时节省能源。

高性能：Neon 技术可以在单个时钟周期内同时对多个数据进行操作，从而实现高效的数据处理。

低延迟：Neon 技术具有低延迟的特点，可以快速地响应用户的操作。

易于编程：Neon 技术采用与 ARM 指令集类似的指令集，可以方便地进行编程和优化。

Neon 技术是 ARM 架构下的 SIMD 技术，可以通过一条指令处理多个数据来提高数据处理的效率和性能。Neon 技术具有低功耗、高性能、低延迟等特点，在移动设备的多媒体处理、游戏、虚拟现实、图像处理等计算密集型应用中得到广泛应用。

3.3 鲲鹏性能分析工具

本章的前面几个小节主要为大家介绍了软件性能测试的基本方法和常用工具，着重从计算、网络、存储三个资源层面，阐述性能优化的关键理论，并介绍对应的工具使用方法，旨在为学员建立宏观的性能测试、优化全景图。

本章最后一节，将为大家介绍鲲鹏性能分析工具，一款基于鲲鹏平台测试、调优的工具箱。

3.3.1 工具介绍

鲲鹏性能分析工具是一款针对华为鲲鹏服务器的性能分析软件，它能够帮助用户快速地定位系统性能瓶颈，优化系统性能，提高系统的稳定性和可靠性。

该工具有以下特点：

支持对鲲鹏服务器系统的各种性能参数进行实时监控和分析，包括 CPU 使用率、内存使用率、磁盘 IO、网络带宽等。

支持对系统中各个应用程序的性能数据进行实时监控和分析，包括应用程序响应时间、应用程序并发请求数、应用程序错误率等。

支持将性能数据以图表形式展示，方便用户进行可视化分析。

鲲鹏性能分析工具是一款非常实用的性能分析软件，能够帮助用户快速地定位系统性能瓶颈，提高系统的稳定性和可靠性，是华为鲲鹏服务器用户不可或缺的利器。

工具的使用场景主要分为：调优助手、系统性能分析、Java 性能分析、系统诊断。后续将围绕四个场景展开介绍。

需要说明的是：鲲鹏性能分析工具的使用环境，要求底层处理器为鲲鹏 916/920，其他 ARM 版本处理器的兼容性，尚未有明确测试数据。

鲲鹏性能分析工具工作原理

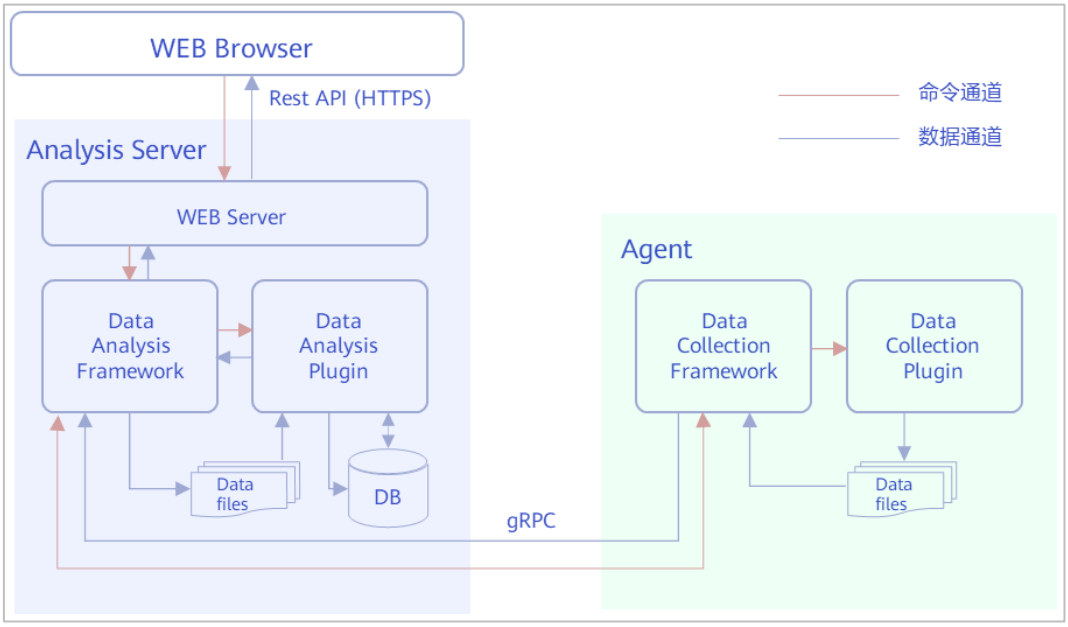


图3-18 鲲鹏性能分析工具逻辑模型

上图是鲲鹏性能分析工具的最新逻辑模型。

从结构上来看，工具分为 AnalysisServer 端和 Agent 端，其中 Analysis Server 实现性能数据分析及分析结果呈现；Agent 实现性能数据采集。

主要组件的功能介绍，如下表：

表3-1 Analysis Server 模块介绍

模块名	功能
Web Browser	Web浏览器，用于操作交互和数据呈现。
Web Server	Web服务器，接收Web浏览器的请求，并触发Data Analysis Framework进行具体的业务处理。
Data Analysis Framework	数据分析框架，主要作用是：通知Data Collection Framework进行数据采集，并接收采集的数据文件。调用相应的Data Analysis Plugin对数据文件进行入库和分析，并保存分析结果。为Web 服务器提供分析结果查询通道。
Data Analysis Plugin	数据分析插件，不同性能分析功能有对应的分析插件，主要作用是：对数据文件进行预处理，并导入数据库中。分析原始数据，得出更适合展示的数据格式及数据间的关联关系，并结合以往项目调优经验值，给出优化建议。提供性能分析结果查询接口。

表3-2 Agent 模块介绍

模块名	功能
Data Collection Framework	数据采集框架，主要作用是：接收Data Analysis Framework的采集通知，调用相应的Data Collection Plugin进行数据采集。将采集的数据文件推送给Data Analysis Framework。
Data Collection Plugin	数据采集插件，不同性能分析功能有对应的采集插件，主要作用是完成具体的性能数据采集，并保存到文件。

实际工作中工具 Analysis Server 和 Agent 可以工作在不同服务器上（服务器 A 安装性能分析工具，服务器 B 运行业务软件）即分布式部署，也可以同时工作在一台服务器上（服务器 A 运行业务软件，同时安装性能分析工具），即单机部署。Sever 端是工具分析的大脑，而 Agent 端是收集信息的插件。

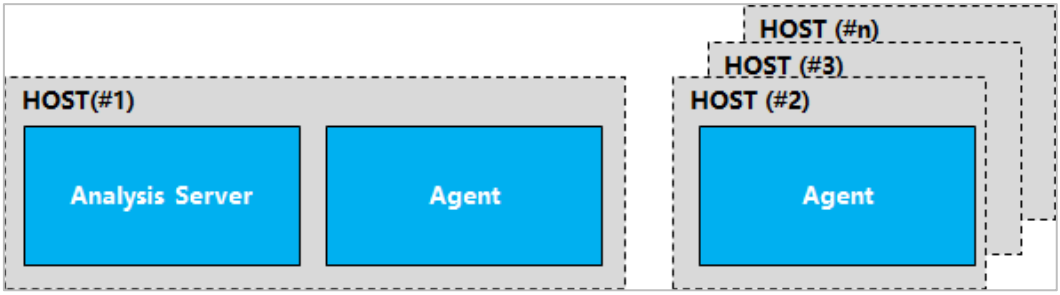


图3-19 分布式部署

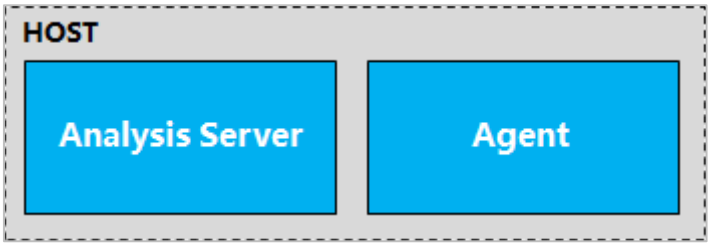


图3-20 单机部署

3.3.2 调优助手

调优助手工具通过系统化组织和分析性能指标、热点函数、系统配置等信息，形成系统资源消耗链条，引导用户根据优化路径分析性能瓶颈，并针对每条优化路径给出优化建议和操作指导，以此实现快速调优。

主要功能有：系统配置、热点函数、系统性能、进程/线程性能。

- 系统配置

采集和分析与性能强相关的系统配置信息，引导用户根据优化路径分析与此相关的性能瓶颈。

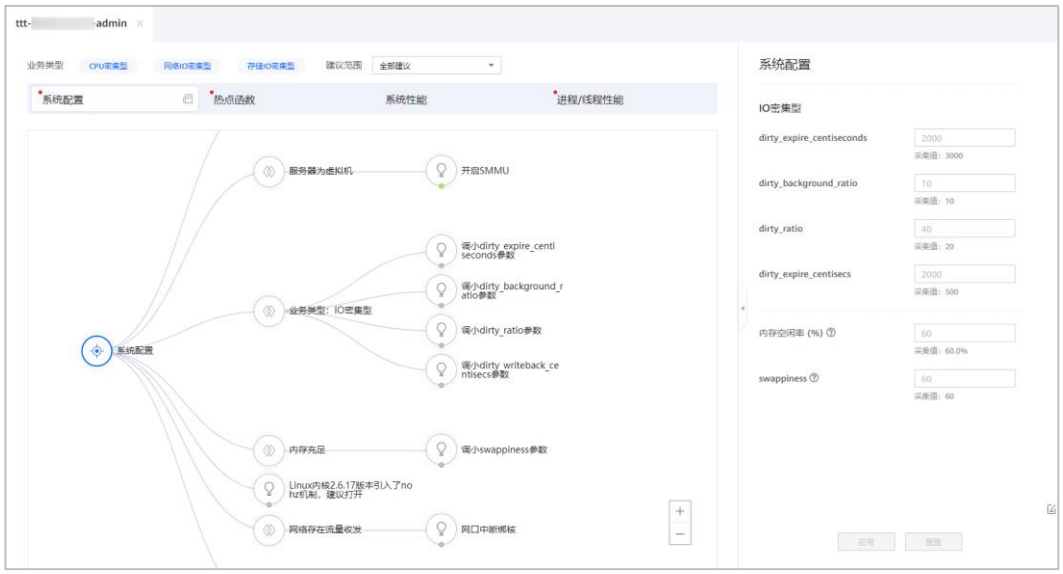


图3-21 系统配置调优指标

针对每条优化路径都会给出优化建议和操作指导。

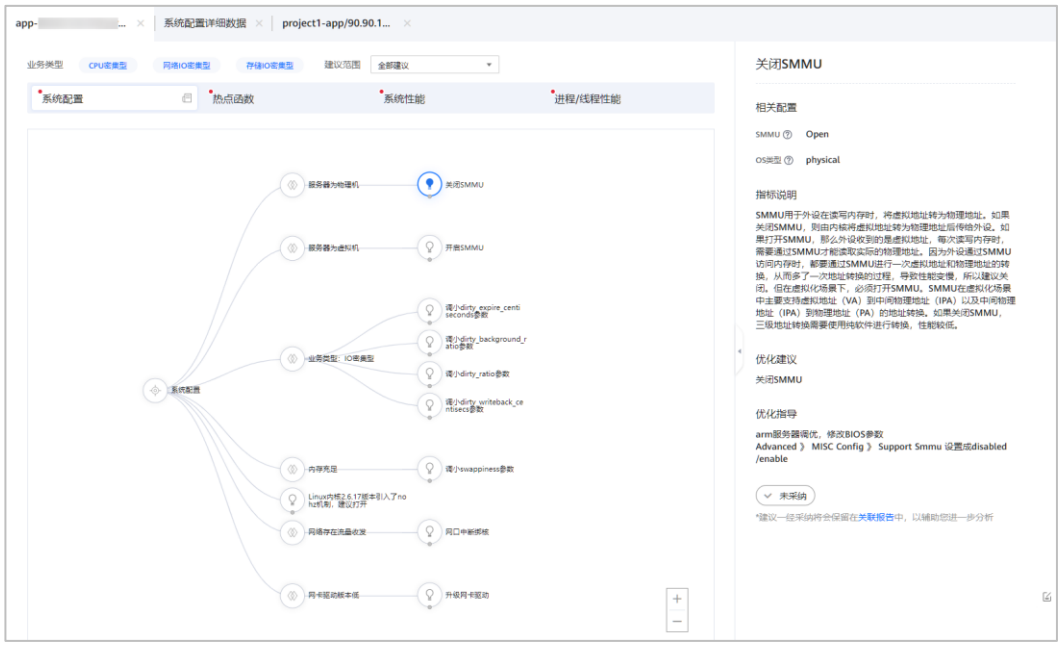


图3-22 优化建议拓扑树图

- 热点函数

采集和分析热点函数信息，引导用户根据优化路径分析与此相关的性能瓶颈。

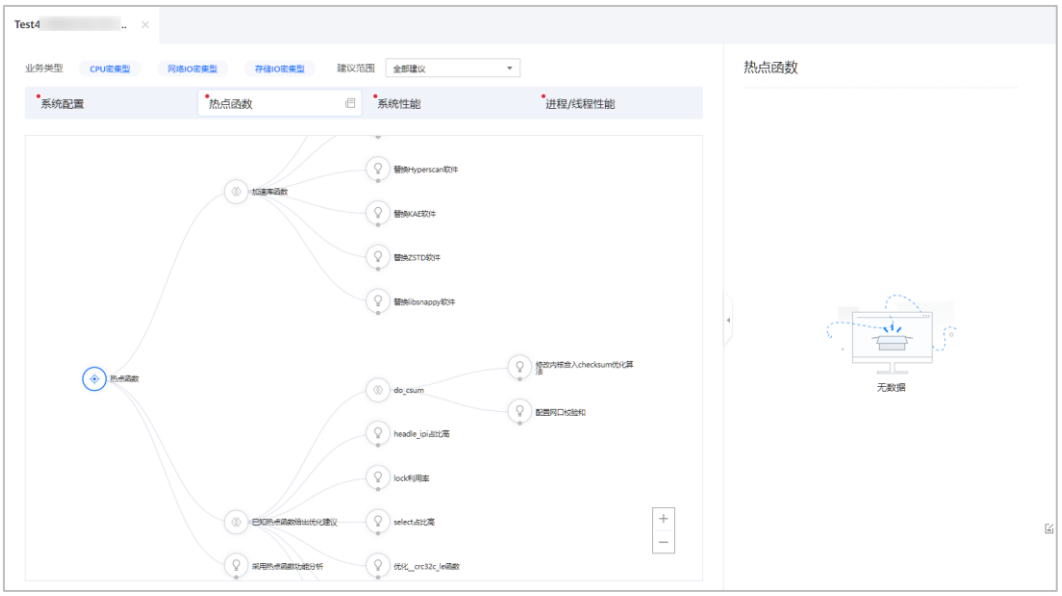


图3-23 查看热点函数分析结果

针对每条优化路径都会给出优化建议和操作指导。

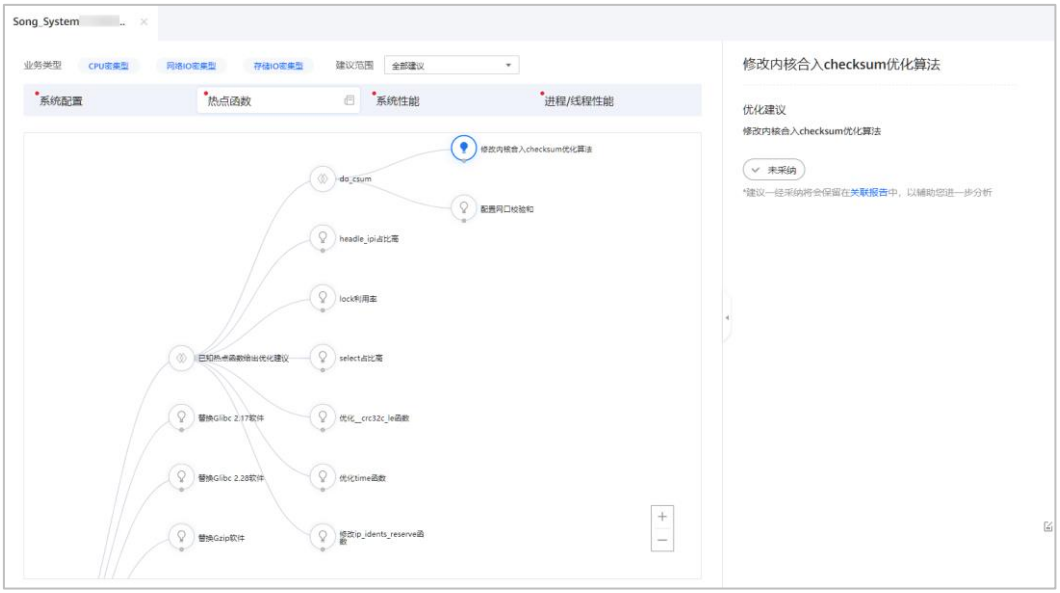


图3-24 优化建议拓扑树图

3.3.3 系统性能分析

系统性能分析是针对基于鲲鹏的服务器的性能分析工具，能收集服务器的处理器硬件、操作系统、进程/线程、函数等各层次的性能数据，分析出系统性能指标，定位到系统瓶颈点及热点函数，给出优化建议。该工具可以辅助用户快速定位和处理软件性能问题。

主要功能：全景分析、OpenMP/MPI 分析、微架构分析、访存分析等。

下面介绍常用的全景分析。

通过采集系统软硬件配置信息，以及系统 CPU、GPU、内存、存储 IO、网络 IO 资源（含 RDMA RoCE）的运行情况，获得它们的使用率、饱和度、错误次数等指标，以此识别系统瓶颈。针对部分系统指标项，根据当前已有的基准值和优化经验提供优化建议。支持采集部分性能数据，以此控制数据量，提高处理速度。

针对大数据和分布式存储场景的硬件配置、系统配置和组件配置进行检查，显示不是最优的配置项，同时分析给出典型硬件配置及软件版本信息。

- 系统配置

展示 CPU 及子系统部件图（物理机环境时显示）以及各个配置项信息，针对部分配置项会根据当前已有经验给出优化建议。

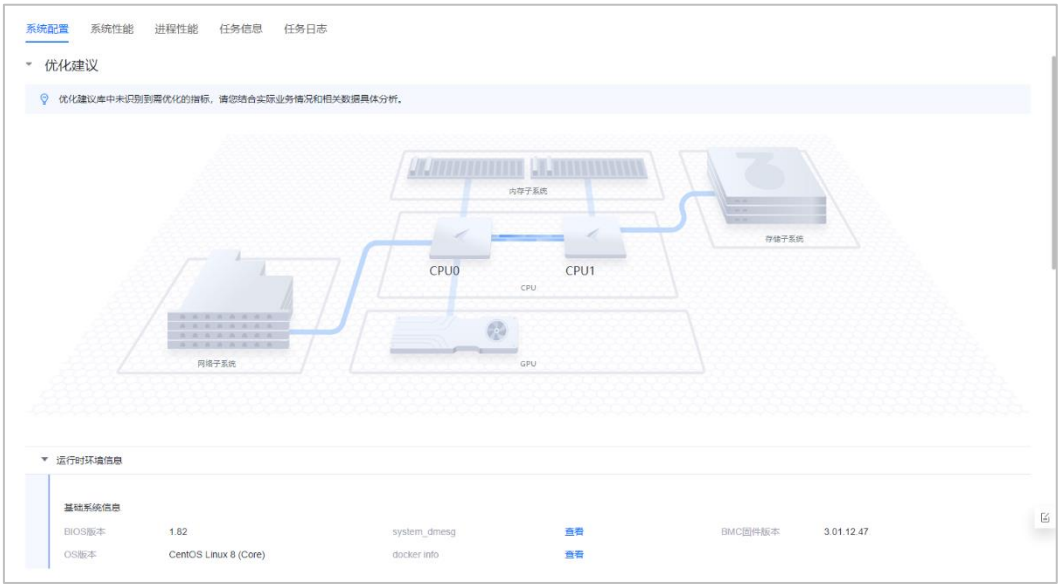


图3-25 系统配置

- PCIe 拓扑信息

以拓扑图方式展示设备各个详细信息。

鼠标悬停至拓扑图中设备图标上可显示设备详细信息和建议，单击对应设备图标显示该设备的详细信息。

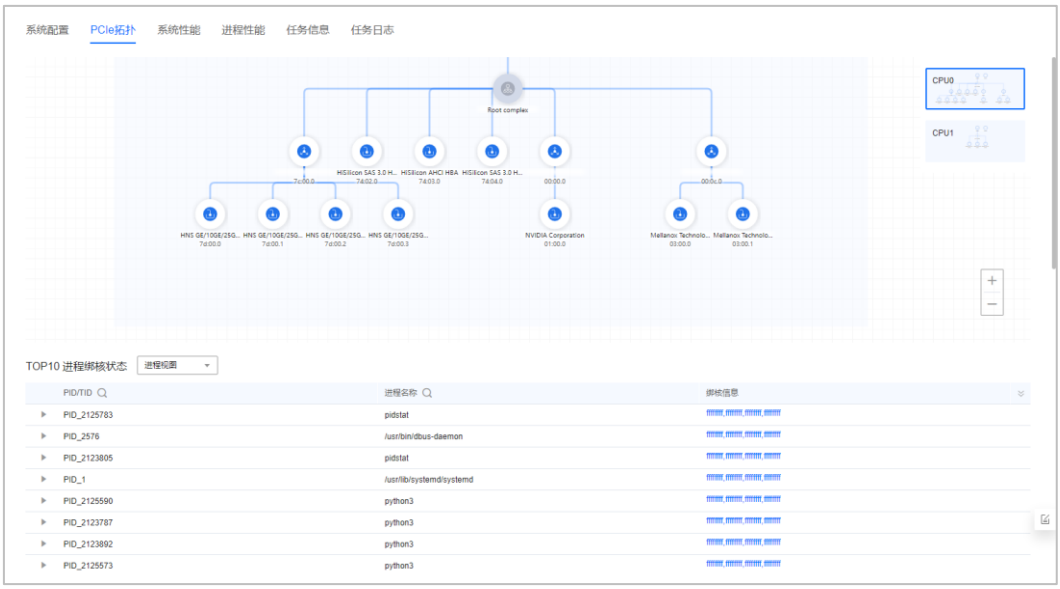


图3-26 PCIe 拓扑信息

- 系统性能

以折线图方式展示系统各个性能指标在整个采集过程中的时序数据。

以列表视图方式展示优化建议和各个性能指标的平均值。如果该指标的平均值超过其基准值，则采用红色标识该平均值，并用方向区分是大于还是小于基准值。在鼠标靠近平均值时，能冒泡显示优化建议。

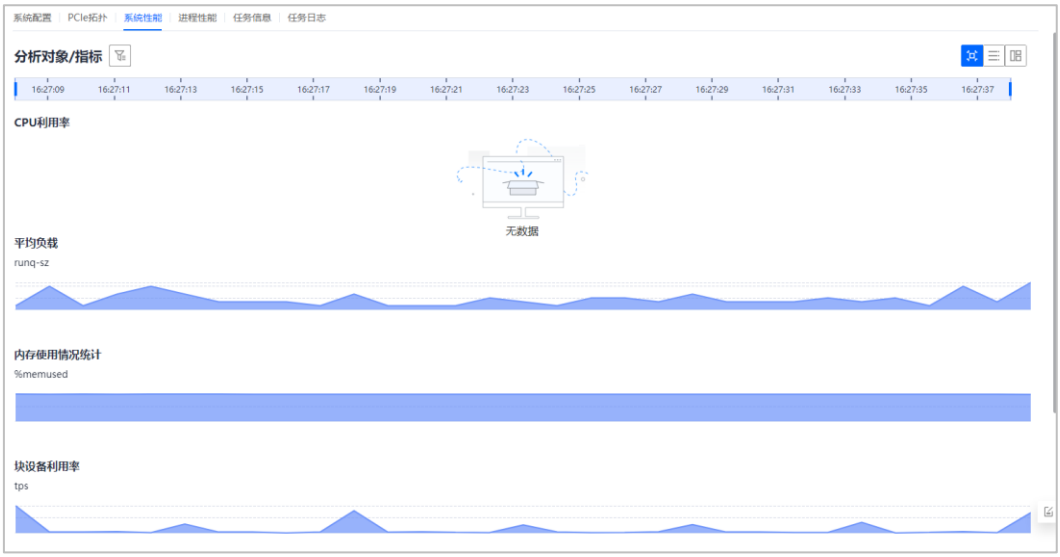


图3-27 系统性能

- 典型配置

针对大数据和分布式存储场景的硬件配置、系统配置和组件配置进行检查并显示不是最优的配置项，同时分析给出典型硬件配置及软件版本信息。



图3-28 大数据场景下的典型配置

3.3.4 Java 性能分析

Java 性能分析是针对基于鲲鹏的服务器上运行的 Java 程序的性能分析和优化工具，能图形化显示 Java 程序的堆，线程，锁，垃圾回收等信息，收集热点函数，定位程序瓶颈点，帮助用户采取针对性优化。

Java 性能分析支持的功能特性如下：

- 在线分析

在线分析包含对于目标 JVM 和 Java 程序的双重分析。包括 Java 虚拟机的内部状态如 Heap，GC 活动，线程状态及上层 Java 程序的性能分析，如调用链分析，热点函数，锁分析，程序线程状态及对象生成分布等。通过 Agent 的方式在线获取 JVM 运行数据，进行精确分析。

主要分析结果包含：

【概览】

在线显示 Java 虚拟机系统状态。

在线显示 JVM 的 Heap 大小、GC 活动、Thread 数量、Class 加载数量和 CPU 使用率。

【热点分析】

通过 PMU 事件采样分析目标 Java 应用内消耗 CPU 资源的热点调用以及其调用关系，追踪 JAVA/C++/OS Kernel 热点方法及其关联汇编指令。

【内存信息】

通过抓取堆快照，分析应用在某时刻堆的直方图分布和支配调用关系，追溯堆内存中各 Java 存活对象到 GC root 的引用关系链，帮助定位潜在的内存问题；对比分析不同时刻的堆快照，给出堆使用与分配变化，辅助用户发现堆内存在分配和使用过程中的异常情况。

获取 Java 堆中各个对象创建的数量及大小，显示相关内存使用情况并实时刷新。

通过采集 GC 事件的各项指标，分析 GC 性能问题。

【GC 信息】

通过抓取目标 Java 应用的内存快照，分析某一时刻目标 Java 应用中堆的直方图分布、支配调用关系；用户可以追溯堆内存中各 Java 存活对象到 GC root 的引用关系链，帮助定位潜在的内存问题。

【IO 信息】

在线分析应用中的文件 IO、Socket IO 时延、消耗带宽等数据，找出热点 IO 操作。

【线程信息】

获取当前 JVM 中实时的活动线程状态和当前线程转储，图形化显示线程锁定状态，分析线程死锁情况。

【数据库信息】

监控和分析数据库连接池。监控数据库连接池连接的情况、帮助用户定位潜在的连接泄露，对不合适的连接池配置给出优化建议。

分析 JDBC 热点 SQL 操作。记录应用中的 SQL 调用时间、耗时和堆栈跟踪，帮助用户定位耗时最长的热点 SQL 操作。

分析 NoSQL 热点操作。记录应用中访问/操作 NoSQL 数据库调用时间、耗时和堆栈跟踪，帮助用户定位耗时最长的热点 NoSQL 操作。

【HTTP 信息】

记录应用中的 HTTP 请求时间和耗时，找出热点 HTTP 请求。

【SpringBoot 信息】

经用户授权后，采集第三方框架，如 Spring Boot Actuator，提供指标数据与性能数据，更加贴近用户业务。

【快照信息】

支持在堆、IO、Workload 在线分析过程中生成快照，对快照进行比对，辅助用户发现资源、业务相关指标的变化趋势，定位潜在的资源泄露问题或性能指标恶化问题。

- 采样分析

通过采样的方式，收集 JVM 的内部活动/性能事件，通过录制及回放的方式来进行离线分析。这种方式对系统的额外开销很小，对业务影响不大，适用于大型的 Java 程序。

主要分析结果包括：

【概览】

显示 Java 虚拟机系统状态。

通过采样及回放的方式显示 JVM 的 Heap 使用情况、GC 活动、IO 消耗和 CPU 使用率。

【线程转储及锁分析结果】

分析程序线程状态及锁。获取采样时间内的线程的状态变化和当前线程转储，根据线程转储图形化地显示线程锁定状态，分析线程死锁情况。

分析程序所用堆积对象。获取采样时间内 Java 堆中各个对象的数量、大小与堆栈跟踪。

分析估计线程阻塞对象和阻塞时间。

【方法采样分析结果】

分析 Java 及 native 代码中热点函数 CPU Cycles 的占比及定位。

支持通过火焰图查看热点函数及其调用栈。

【内存分析结果】

通过对存留周期长的对象进行采样，分析 Java 应用中潜在的堆内存泄漏，并辅助用户定位潜在原因。

根据采样分析记录出具关于启动参数、GC 方面的报告和优化建议。

【IO 分析结果】

分析应用中文件 IO、Socket IO 时延、消耗带宽，找出热点 IO 操作。

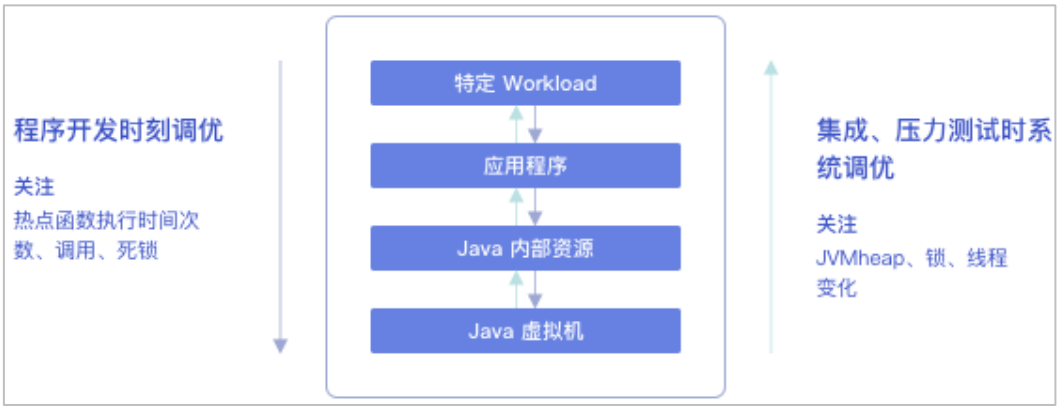


图3-29 性能优化视角

3.3.5 系统诊断

系统诊断工具通过分析系统运行指标，识别异常点，例如：内存泄漏、内存越界、网络丢包等，并给出优化建议。工具还支持压测系统，如：网络 IO、存储 IO，评估系统最大性能。

系统诊断工具支持的功能如下：

内存诊断

提供内存诊断能力，帮助用户识别应用程序中存在的内存使用的问题点，提升程序的可靠性，具体包括：内存泄漏诊断、内存越界诊断。

内存泄漏诊断

分析应用程序中存在的内存泄漏点（包括内存未释放和异常释放），得到具体的泄漏信息，并支持关联出调用栈信息和源码。

实时跟踪应用程序运行期间系统层、应用层（调用内存申请函数）、分配器层的内存消耗情况。

分析发生 OOM 时的进程内存状态、系统内存状态和调用栈信息。

内存越界诊断：分析应用程序中存在的内存异常访问点，给出异常访问类型和内存访问信息，并支持关联出调用栈和源码。

网络 IO 诊断

压测网络，获得网络最大能力，为网络 IO 性能优化提供基础参考数据；诊断网络，定位网络疑难问题，解决因网络配置和异常而导致的网络 IO 性能问题。具体功能如下：

网络拨测：获得网络运行情况和最大能力，包括：连通情况、带宽、重传、丢包率、时延、抖动等，评估网络质量。

丢包诊断：获取网络丢包点，定位网络丢包根因，给出修复建议。

网络抓包：辅助网络拨测和丢包诊断，根据抓取的网络消息包定位网络异常根因。

系统负载监控：用于诊断期间监控系统负载，包括：CPU、内存、网络 IO、软硬件中断等，以此了解在网络拨测或发生网络丢包时，是否存在系统资源不足。

存储 IO 诊断

压测存储 IO，获得存储设备最大能力，包括：吞吐量、IOPS、时延等，并以此评估存储能力，为存储 IO 性能优化提供基础参考数据。

4 鲲鹏即时交流平台综合实验

4.1 鲲鹏即时交流平台介绍

4.1.1 平台功能简介及实现方法

4.1.1.1 鲲鹏即时交流平台需求简介

根据业务需求规划，本实验旨在搭建一个综合交流平台，名为“鲲鹏即时交流平台”。鲲鹏即时交流平台是一个提供给用户进行即时交流的平台，用户可在平台上与其他用户创建对话并且进行即时交流，提供发送文字和图片的功能，并且可以查看最近的聊天记录，还可以在大数据中搜索最近的热门词汇。

4.1.1.2 鲲鹏及时交流平台功能设计

鲲鹏及时交流平台的核心功能主要分为登录、实时发送和接收消息、实时发送和接收图片、查询历史信息四个方面，四个功能都使用 Java 程序编写完成。

1、登录：包含登录、请求、密码校验等子功能。

2、实时发送、接收消息：包含发送文本、存放文本、推送消息、显示聊天信息等子功能。

3、实时发送、接收图片：包含发送图片、图片上传文件系统、图片推送、显示聊天信息等子功能。

4、查询历史信息：包含检索请求、请求数据、推送信息、显示信息等子功能。

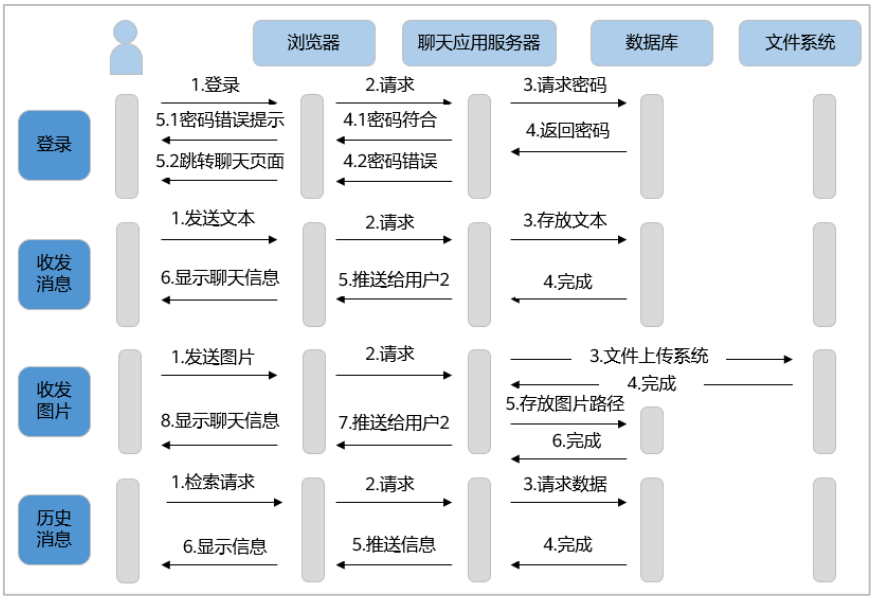


图4-1 功能设计

4.1.1.3 实时聊天功能实现方案：http 轮询

平台聊天功能中的信息推送能力有多种实现方式，首先来了解 http 轮询技术。

轮询是指由客户端每隔一段时间向服务器发出 HTTP 请求，服务端接收到请求后向客户端返回最新的数据。客户端的轮询方式一般分为短轮询和长轮询。

- 短轮询

客户端每隔一段时间向服务器发起一次普通 HTTP 请求，服务端查询当前接口是否有数据更新，若有数据更新则向客户端返回最新数据，若无则提示客户端无数据更新。

短轮询意味着客户端需要频繁发出请求，这回占用网络流量，而服务端也需要频繁接受请求，消息推送的实时性也较差。

- 长轮询

客户端向服务端发出一个设置较长网络超时时间的 HTTP 请求，并在 HTTP 链接超时前，不主动断开连接；若在超时时间内有数据返回，则将数据传递给客户端，并主动断开连接；若等待期间没有数据更新，则超时后主动断开连接，此后客户端将再次建立一个同样的 HTTP 请求，并重复以上过程。

长轮询中，客户端请求次数大量减少了，节省了网络流量，但是把请求长期挂起依然会导致资源浪费。

然而不论短轮询还是长轮询，都有一些缺陷。

首先，客户端要不断向服务器发出请求，这会消耗较多客户端资源，但并不是每一次请求都能获取新的信息推送。

其次，HTTP 协议的请求与回复消息中，需包含较长的头部信息，其中真正有效的数据占比较少，这也是额外的带宽资源消耗。

最后，基于 HTTP 协议的原理，通信只能由客户端发起，HTTP 协议做不到服务器主动向客户端推送信息，所以如果服务器有连续的状态变化，客户端获取更新数据会存在滞后性，无法保证用户端的体验。

4.1.1.4 实时聊天功能实现方案：WebSocket

WebSocket 协议是基于 TCP 的一种新的网络协议。它实现了浏览器与服务器全双工（full-duplex）通信，即允许服务器主动发送信息给客户端，从而解决了 HTTP 协议中获取数据滞后性的问题。

下图是 WebSocket 的通信过程，分为三个步骤：

1、客户端发起 http 请求，经过 3 次握手后，建立起 TCP 连接；

2、服务器收到客户端的握手请求后，发送数据；

3、客户端收到连接成功的消息后，开始借助于 TCP 传输信道进行全双工通信。

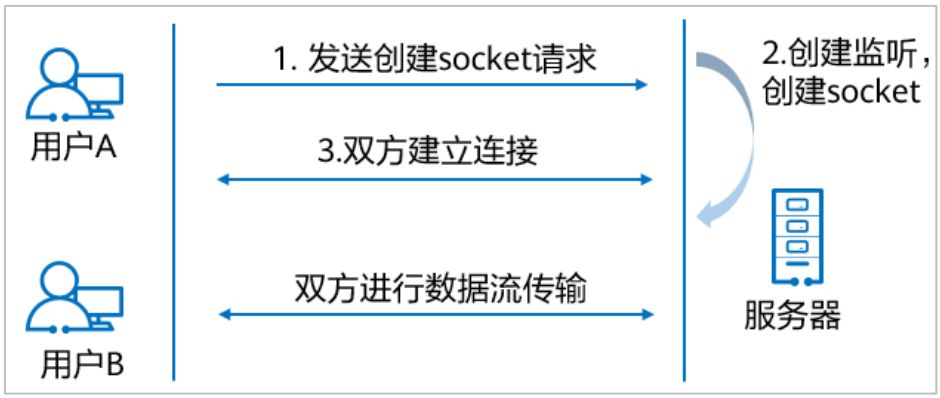


图4-2 WebSocket 的通信过程

4.1.1.5 即时交流平台实时聊天功能实现

WebSocket 和 HTTP 都是基于 TCP/IP 的应用层协议，但 WebSocket 是双向通信协议，模拟 Socket 协议，可以双向发送或接受信息。而 HTTP 是单向的通信协议，只能由客户端发起请求。

此外，WebSocket 和 HTTP 还有一定的联系。WebSocket 在建立握手时，数据是通过 HTTP 传输的。但是建立之后，真正传输数据的时候是不需要 HTTP 协议的。

4.1.1.6 鲲鹏即时交流平台逻辑架构设计

平台的数据流向如下，用户通过客户端访问 Nginx Web 服务器，Nginx 对接了即时交流平台，该平台后端基于 Java 语言编写，平台的数据统一存储在 MySQL 数据库中。

其中 Java 语言编写的后端程序是该平台功能实现的核心，后文中将说明平台如何通过对接大数据组件实现热词搜索功能。

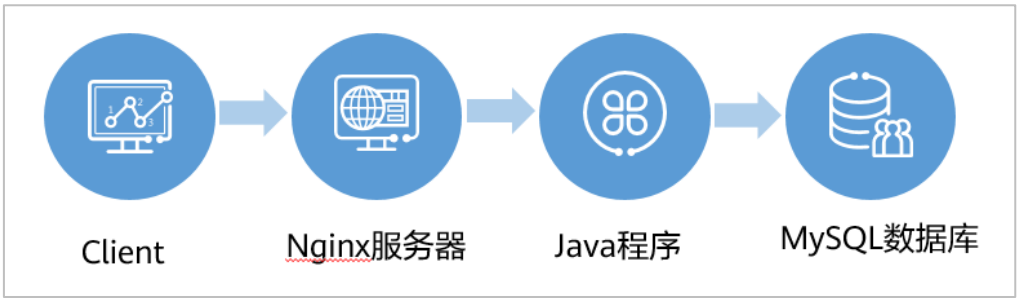


图4-3 逻辑架构设计

4.1.1.7 鲲鹏即时交流平台架构

平台的架构主要包含五个部分：Nginx、Java 程序、MySQL、Hadoop 和 Hive。其中 Java 后端程序是基于 SpringBoot 和 Mybatis 框架开发的。

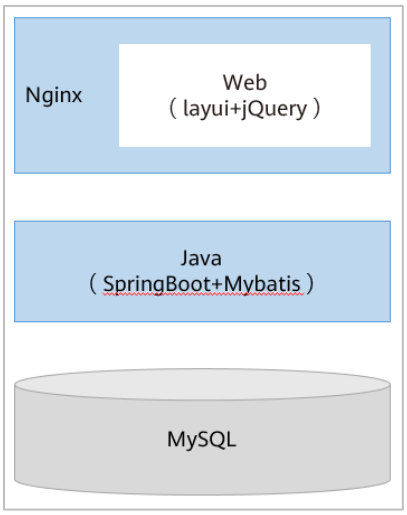


图4-4 平台架构

- SpringBoot 框架

Spring Boot 是 Pivotal 团队在 Spring 的基础上提供的一套全新的开源框架，其目的是为了简化 Spring 应用的搭建和开发过程。Spring Boot 去除了大量的 XML 配置文件，简化了复杂的依赖管理。

Spring Boot 具有 Spring 一切优秀特性，Spring 能做的事，Spring Boot 都可以做，而且使用更加简单，功能更加丰富，性能更加稳定而健壮。由此，我们可以使用 SpringBoot 快速的开发基于 Spring 框架的项目。随着近些年来微服务技术的流行，Spring Boot 也成了时下炙手可热的技术。

Spring Boot 集成了大量常用的第三方库配置，Spring Boot 应用中这些第三方库几乎可以是零配置的开箱即用（out-of-the-box），大部分的 Spring Boot 应用都只需要非常少量的配置代码（基于 Java 的配置），我们在开发业务代码时能够非常方便的、不需要过多关注框架的配置，而只需要关注业务即可。

- Mybatis

目前，Java 的持久层框架产品有许多，常见的有 Hibernate 和 MyBatis。

MyBatis 本是 Apache 的一个开源项目——iBatis，2010 年这个项目由 Apache Software Foundation 迁移到了 Google Code，并且改名为 MyBatis。

MyBatis 提供的持久层框架包括 SQL Maps 和 Data Access Objects（DAO），它消除了几乎所有的 JDBC 代码和参数的手工设置以及结果集的检索，使用简单的 XML 或注解用于配置和原始映射，将接口和 Java 的 POJOs（Plain Old Java Objects，普通的 Java 对象）映射成数据库中的记录。

由此，MyBatis 作为一个开源、轻量级的数据持久化框架，成为了 JDBC 和 Hibernate 的替代方案。其内部封装了 JDBC，简化了加载驱动、创建连接、创建 Statement 等繁杂的过程，开发者只需要关注 SQL 语句本身。

4.1.1.8 实验任务总览

实验任务分成四大模块，包括基础环境准备、鲲鹏代码迁移、及时交流平台部署以及数据处理调优。

基础环境准备：实验开始前，需要在华为云服务器上搭建后续实验需要的环境，包括 Nginx、MySQL，方便后续实验的进行。

鲲鹏代码迁移：使用鲲鹏代码迁移工具进行即时交流平台后端依赖库 JNA 的源码迁移，并对源码进行编译。

即时交流平台部署：鲲鹏即时交流平台的部署分为前端前面和后端项目的部署。前端采用 layui+jQuery 实现，后端使用 JAVA 语言进行编写，基于 Springboot+Mybatis 框架。

数据处理调优：对平台的数据处理过程中，基于大数据解决方案，部署 hadoop 和 hive，完成一系列数据分析任务，并针对大数据场景进行性能调优。

4.1.2 MySQL 介绍

4.1.2.1 什么是关系型数据库

关系型数据库的定义：指采用了关系模型来组织数据的数据库。
关系模型指的就是二维表格模型，而一个关系型数据库就是由二维表及其之间的联系所组成的一个数据组织。

- 关系型数据库的优点：

1、容易理解：二维表结构是非常贴近逻辑世界的一个概念，关系模型相对网状、层次等其他模型来说更容易理解。

2、使用方便：通用的 SQL 语言使得操作关系型数据库非常方便。

3、易于维护：丰富的完整性（实体完整性、参照完整性和用户定义的完整性）大大减低了数据冗余和数据不一致的概率。

- 常用关系型数据库：

Oracle，MySQL，SQL Server，PostgreSQL，GaussDB，openGauss 等。

4.1.2.2 MySQL 简介

MySQL 是一个关系型数据库，使用 SQL 语言进行增删改查操作。

MySQL 数据库开源免费，能够跨平台，支持分布式，可以和 PHP、Java 等 Web 开发语言完美配合，非常适合中小型企业作为 Web 数据库（网站数据库）。

- MySQL 的优点

1、MySQL 是开放源代码的数据库

MySQL 是开放源代码的数据库，任何人都可以获取该数据库的源代码。这就使得任何人都可以修正 MySQL 的缺陷，并且任何人都能以任何目的来使用该数据库。MySQL 是一款可以自由使用的数据库。

2、MySQL 的跨平台优势

MySQL 不仅可以在 Windows 系列的操作系统上运行，还可以在 UNIX、Linux 和 Mac OS 等操作系统上运行。因为很多网站都选择 UNIX、Linux 作为网站的服务器，所以 MySQL 的跨平台性保证了其在 Web 应用方面的优势。虽然微软公司的 SQL Server 数据库是一款很优秀的商业数据库，但是其只能在 Windows 系列的操作系统上运行。因此，MySQL 数据库的跨平台性是一个很大的优势。

3、功能强大且使用方便

MySQL 是一个真正的多用户、多线程 SQL 数据库服务器。它能够快速、有效和安全的处理大量的数据。相对于 Oracle 等数据库来说，MySQL 的使用是非常简单的。MySQL 主要目标是快速、健壮和易用。

4.1.3 Nginx 介绍

4.1.3.1 Nginx 简介

Nginx 是一个高性能的 HTTP 和反向代理服务器，特点是占有内存少，并发能力强。

Nginx 具有几项强大的功能，包括正向代理、反向代理、静态页面的 web 服务器、负载均衡、动静分离等。

1、Nginx 可以作为静态页面的 web 服务器，同时还支持 CGI 协议的动态语言，比如 perl、php 等。但是不支持 java。Java 程序只能通过与 tomcat 配合完成。Nginx 专为性能优化而开发， 性能是其最重要的考量，并且能经受高负载的考验，有报告表明 Nginx 能支持高达 50000 个并发连接数。

2、负载均衡：增加服务器的数量，然后将请求分发到各个服务器上，将原先请求集中到单个服务器上的情况改为将请求分发到多个服务器上，将负载分发到不同的服务器，也就是我们所说的负载均衡。

3、动静分离：为了加快网站的解析速度，可以把动态页面和静态页面由不同的服务器来解析，加快解析速 度。降低原来单个服务器的压力。

4.1.3.2 反向代理

反向代理（Reverse proxy）在电脑网络中是代理服务器的一种。服务器根据客户端的请求，从其关联的一组或多组后端服务器（如 Web 服务器）上获取资源，然后再将这些资源返回给客户端，客户端只会得知反向代理的 IP 地址，而不知道在代理服务器后面的服务器集群的存在，客户端不需要任何配置就可以访问。

我们只需要将请求发送到反向代理服务器，由反向代理服务器去选择目标服务器获取数据后，在回给客户端，此时反向代理服务器和目标服务器对外就是一个服务器，暴露的是代理服务器地址，隐藏了真实服务器 IP 地址。

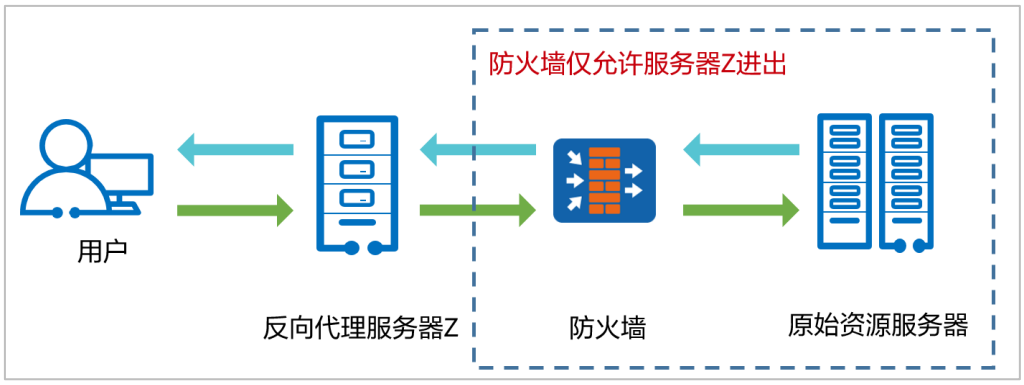


图4-5 反向代理

4.1.3.3 正向代理

如果把局域网外的 Internet 想象成一个巨大的资源库，则局域网中的客户端要访问 Internet，则需要通过代理服务器来访问，这种代理服务就称为正向代理。

正向代理（forward）是一个位于客户端【用户 A】和原始服务器（origin server）【服务器 B】之间的服务器【代理服务器 Z】，为了从原始服务器取得内容，用户 A 向代理服务器 Z 发送一个请求并指定目标（服务器 B），然后代理服务器 Z 向服务器 B 转交请求并将获得的内容返回给客户端。客户端必须要进行一些特别的设置才能使用正向代理。

通过代理服务器来访问服务器的过程就叫正向代理，需要在客户端配置代理服务器进行指定网站访问。

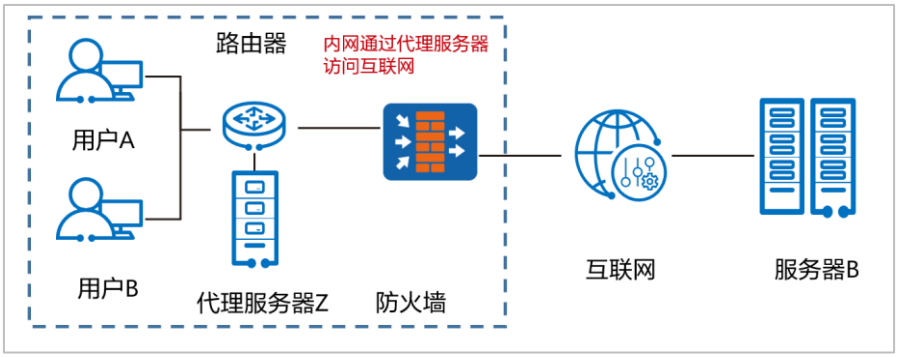


图4-6 正向代理

4.2 基础实验环境部署

4.2.1 主机环境选择

为优化即时交流平台的性能，主机选择鲲鹏弹性云服务器。鲲鹏弹性云服务器（ECS）作为基础云服务之一，也是用户可以直接感知到鲲鹏的最重要的服务。用户通过购买鲲鹏弹性云服务器 ECS，之后为云服务器添加磁盘、网络等资源，使其成为开发环境或者生产业务集群的一部分。本实验中选择的鲲鹏弹性云服务器规格如下：

实验资源	云主机名称	配置	OS 版本
鲲鹏弹性云服务器	ecs-kunpeng	kc1.xlarge.2 4vCPUs | 8GiB	openEuler 22.03 64bit with ARM

4.2.2 基础环境部署

基础环境部署分为两个步骤：

1、部署 MySQL：MySQL 作为鲲鹏即时交流平台的数据库，存储平台产生的数据，包括用户信息、聊天信息等。

2、部署 Nginx：Nginx 作为鲲鹏即时交流平台的前端服务器，负责运行前端应用。

4.2.3 MySQL 部署步骤

MySQL 的部署分为四个步骤：

1、配置安装环境：数据盘、系统用户等配置。

2、安装依赖包：获取并安装 MySQL 运行所需要的依赖包。

3、安装 MySQL：使用安装包配置和安装 MySQL。

4、运行 MySQL：修改配置文件初始化数据库并验证。

4.2.4 Nginx 部署步骤

Nginx 部署步骤分为三个部分：

1、编译安装：获取依赖包、源码，并进行源码编译。

2、配置修改：修改配置文件，如端口号、运行内存等信息。

3、运行测试：通过 IP:PORT 访问 Nginx。

4.3 鲲鹏代码迁移

4.3.1 JNA 简介

JNA（Java Native Access）是一个开源的 Java 框架，是 Sun 公司推出的一种调用本地方法的技术，是建立在经典的 JNI 基础之上的一个框架。

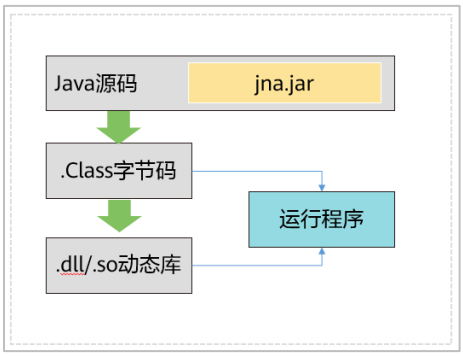


图4-7 Java 框架

即时交流平台后端程序调用了使用 c 语言开发的 so 库函数，该函数需要重新编译才可在鲲鹏 ECS 服务器上运行。

- 在软件领域，JAR 文件（Java 归档，英语：Java Archive）是一种软件包文件格式，通常用于聚合大量的 Java 类文件、相关的元数据和资源（文本、图片等）文件到一个文件，以便开发 Java 平台应用软件或库。 JAR 文件是一种归档文件，以 ZIP 格式构建，以.jar 为文件扩展名。
- Java 字节码类文件（.class）是 Java 编译器编译 Java 源文件（.java）产生的“目标文件”。它是一种 8 位字节的二进制流文件， 各个数据项按顺序紧密的从前向后排列，相邻的项之间没有间隙，这样可以使得 class 文件非常紧凑，体积轻巧，可以被 JVM 快速的加载至内存，并且占据较少的内存空间（方便于网络的传输）。
- DLL(Dynamic Link Library)文件为动态链接库文件，又称“应用程序拓展”，是软件文件类型。 在 Windows 中，许多应用程序并不是一个完整的可执行文件，它们被分割成一些相对独立的动态链接库，即 DLL 文件，放置于系统中。 当我们执行某一个程序时，相应的 DLL 文件就会被调用。
- 一般来说.so 文件就是常说的动态链接库，都是 C 或 C++编译出来的。 与 Java 比较就是：它通常是用的 Class 文件（字节码） Linux 下的.so 文件时不能直接运行的，一般来讲，.so 文件称为共享库。

4.3.2 任务流程

在这一步骤，需要使用鲲鹏代码迁移工具进行即时交流平台后端源码的迁移，并对源码进行编译。

任务流程包括三个步骤：

1、安装鲲鹏代码迁移工具。

2、使用工具进行分析扫描：对于鲲鹏即使交流平台的源码，使用鲲鹏代码迁移工具分析扫描，并根据修改建议修改源码。

3、编译源码：进行编译源码，并生成动态 so 库，以便后续即时交流平台调用。

4.4 鲲鹏即时交流平台搭建

在这一步骤，需要实现平台的核心功能，即聊天功能。

一共包括三个步骤：

1、部署前端项目

前端采用 layui+jQuery 实现，部署在 Nginx 服务器上。

2、部署后端项目

后端使用 Java 实现，基于 Springboot+Mybatis 框架。使用 Maven 进行编译打包。

3、聊天功能实现

主要需实现消息入库的功能，即将消息数据存入到 MySQL 的 messages 表中。

4.5 基于 Hive 的数据处理

4.5.1 大数据简介

4.5.1.1 大数据概述

IDC（International Data Corporation）在它编制的年度数字宇宙研究报告《从混沌中提取价值》（Extracting Value from Chaos）中给大数据下了一个定义：大数据技术是新一代的技术与架构，它被设计用于在成本可承受（Economically）的条件下，通过非常快速（Velocity）的采集、发现和分析，从大体量（Volumes）、多类别（Variety）的数据中提取价值（Value）。

上文中提到的大量（Volume）、多样（Variety）、高速（Velocity）和价值（Value），就是大数据的 4V 特点。

1、Volume（大量），大数据的特征首先就是数据规模大。随着互联网、物联网、移动互联技术的发展，人和事物的所有轨迹都可以被记录下来，数据呈现出爆发性增长。

2、Variety（多样）。数据来源的广泛性，决定了数据形式的多样性。大数据可以分为三类，一是结构化数据，如财务系统数据、信息管理系统数据、医疗系统数据等，其特点是数据间因果关系强；一是非结构化的数据，如视频、图片、音频等，其特点是数据间没有因果关系；三是半结构化数据，如 HTML 文档、邮件、网页等，其特点是数据间的因果关系弱。有统计显示，目前结构化数据占据整个互联网数据量的 75%以上，而产生价值的大数据，往往是非结构化数据。

3、Velocity（高速），数据的增长速度和处理速度是大数据高速性的重要体现。与以往的报纸、书信等传统数据载体生产传播方式不同，在大数据时代，大数据的交换和传播主要是通过互联网和云计算等方式实现的，其生产和传播数据的速度是非常迅速的。另外，大数据还要求处理数据的响应速度要快，例如，上亿条数据的分析必须在几秒内完成。数据的输人、处理与丢弃必须立刻见效，几乎无延迟。

4、Value（价值），大数据的核心特征是价值，其实价值密度的高低和数据总量的大小是成反比的，即数据价值密度越高数据总量越小，数据价值密度越低数据总量越大。任何有价值的

信息的提取依托的就是海量的基础数据。当然目前大数据背景下有个未解决的问题，如何通过强大的机器算法更迅速地在海量数据中完成数据的价值提纯。

4.5.1.2 大数据介绍及组件关系分布

大数据平台主要提供四大功能：

1、数据采集，通过 kafka 完成实时数据采集，汇总数据到平台中；

2、数据存储，通过 HDFS 和 HBase 完成海量数据存储；

3、数据处理计算，通过 MapReduce（批处理）、spark（内存计算）以及 Storm（实时流处理）三个处理引擎完成各场景数据的处理和计算；

4、数据应用：通过 Hive 实现数仓建设以及类 SQL 查询，通过 ElasticSearch 对海量文档数据进行实时检索以及 Redis 对 KV 数据进行内存查询。

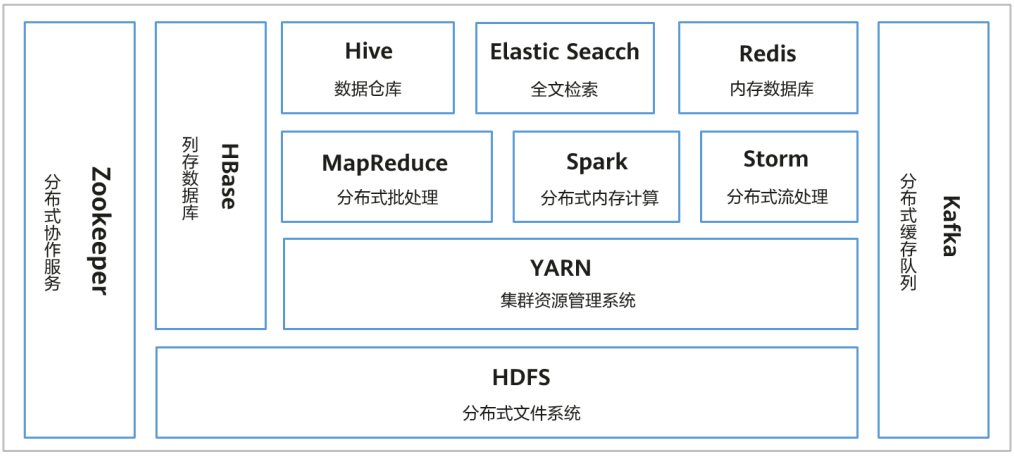


图4-8 大数据相关组件

以下是各组件的详细介绍：

- HDFS Hadoop Distributed File System，简称 HDFS，是一个分布式文件系统。HDFS 是一个高度容错性的系统，适合部署在廉价的机器上。HDFS 能提供高吞吐量的数据访问，非常适合大规模数据集上的应用。
- HBASE 是 Hadoop 的数据库，一个分布式、可扩展、大数据的存储。是为有数十亿行和数百万列的超大表设计的，是一种分布式数据库，可以对大数据进行随机性的实时读取/写入访问。提供类似谷歌 Bigtable 的存储能力，基于 Hadoop 和 Hadoop 分布式文件系统（HDFS）而建。HDFS 是 Apache 的 Hadoop 项目的子项目，HBase 利用 Hadoop HDFS 作为其文件存储系统。HBase 位于结构化存储层，Hadoop HDFS 为 HBase 提供了高可靠性的底层存储支持。除了 HBase 产生的一些日志文件，HBase 中的所有数据文件都可以存储在 Hadoop HDFS 文件系统上。
- Redis 是一个高性能的 key-value 存储系统，和 Memcached 类似，它支持存储的 value 类型相对更多，包括 string（字符串）、list（链表）、set（集合）和 zset（有序集合）。Redis 的出现，很大程度补偿了 memcached 这类 key/value 存储的不足，在部分场合可以对关系数据库起到很好的补充作用。

- Kafka 是一种高吞吐量的分布式发布订阅消息系统，它可以处理消费者规模网站中的所有动作流数据，目前已成为大数据系统在异步和分布式消息之间的最佳选择。
- MapReduce 是一种编程模型，用于大数据集（大于 1TB）的并行运算。在 MapReduce 程序中计算的数据可以来自多个数据源，如 Local FileSystem、HDFS、数据库等。最常用的是 HDFS，可以利用 HDFS 的高吞吐性能读取大规模的数据进行计算。同时在计算完成后，也可以将数据存储到 HDFS。
- Spark 是一个高速、通用大数据计算处理引擎。拥有 Hadoop MapReduce 所具有的优点，但不同的是 Job 的中间输出结果可以保存在内存中，从而不再需要读写 HDFS，因此 Spark 能更好地适用于数据挖掘与机器学习等需要迭代的 MapReduce 的算法。通常，Spark 中计算的数据可以来自多个数据源，如 Local File、HDFS 等。最常用的是 HDFS，用户可以一次读取大规模的数据进行并行计算。在计算完成后，也可以将数据存储到 HDFS。它可以与 Hadoop 和 Apache Mesos 一起使用，也可以独立使用。
- Storm 是 Twitter 开源的一个类似于 Hadoop 的实时数据处理框架。编程模型简单，显著地降低了实时处理的难度，也是当下最人气的流计算框架之一。与其他计算框架相比，Storm 最大的优点是毫秒级低延时。
- Hive 是基于 Hadoop 的一个数据仓库工具，可以将结构化的数据文件映射为一张数据库表，并提供简单的 sql 查询功能，可以将 sql 语句转换为 MapReduce 任务进行运行。 其优点是学习成本低，可以通过类 SQL 语句快速实现简单的 MapReduce 统计，不必开发专门的 MapReduce 应用，十分适合数据仓库的统计分析。
- Elasticsearch 能建立全文索引（把文本中的内容拆分成若干关键词，然后根据关键词创建索引。查询时根据关键词查询索引，最终找到包含关键词的文章），它将数据和索引分离，把索引分片，分布式的保存到不同节点，节点可以扩展到上百个，能实时检索、处理 PB 级别的结构化或非结构化数据。同时分片可以进行副本备份保证数据的可靠性，各分片副本协同工作也大大提高检索性能，且通过简单的 RESTful API 让全文搜索变得高效简单。
- YARN 是一种新的 Hadoop 资源管理器，它是一个通用资源管理系统，可为上层应用提供统一的资源管理和调度，解决了旧 MapReduce 框架的性能瓶颈。它的基本思想是把资源管理和作业调度/监控的功能分割到单独的守护进程。
- ZooKeeper 是一个分布式的应用程序协调服务，是 Hadoop 和 Hbase 的重要组件。它是一个为分布式应用提供一致性服务的工具，让 Hadoop 集群里面的节点可以彼此协调。ZooKeeper 现在已经成为了 Apache 的顶级项目，为分布式系统提供了高效可靠且易于使用的协同服务。

4.5.1.3 Hadoop 介绍

Hadoop 是一个分析和处理大数据的软件平台，由四个主要模块组成：

Hadoop 分布式文件系统（HDFS）：一个在标准或低端硬件上运行的分布式文件系统。除了更高容错和原生支持大型数据集，HDFS 还提供比传统文件系统更出色的数据吞吐量。

Yet Another Resource Negotiator (YARN)：管理与监控集群节点和资源使用情况。它会对作业和任务进行安排。

MapReduce：一个帮助计划对数据运行并行计算的框架。Map 任务会提取输入数据，转换成能采用键值对形式对其进行计算的数据集。Reduce 任务会使用 Map 任务的输出来对输出进行汇总，并提供所需的结果。

Hadoop Common：提供可在所有模块上使用的常见 Java 库。

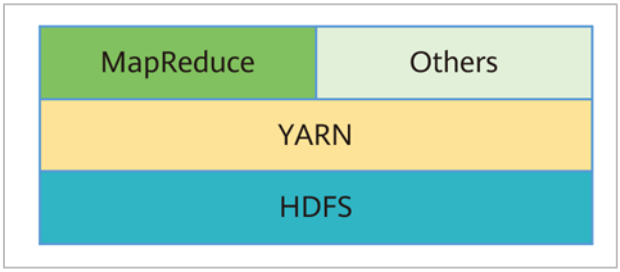


图4-9 Hadoop 组件

4.5.1.4 Hive 介绍

Hive 是建立在 Hadoop 上的数据仓库基础构架。它提供了一系列的工具，可以用来进行数据提取转化加载（ETL），这是一种可以存储、查询和分析存储在 Hadoop 中的大规模数据的机制。Hive 定义了简单的类 SQL 查询语言，称为 HiveQL，它允许熟悉 SQL 的用户查询数据。Hive 的数据计算依赖于 MapReduce、Spark、Tez。

使用新的执行引擎 Tez 代替原先的 MapReduce，性能有了显著提升。Tez 可以将多个有依赖的作业转换为一个作业（这样只需写一次 HDFS，且中间节点较少），从而大大提升 DAG 作业的性能。

- Hive 主要特点：

1、海量结构化数据分析汇总。

2、将复杂的 MapReduce 编写任务简化为 SQL 语句。

3、灵活的数据存储格式，支持 JSON，CSV，TEXTFILE，RCFILE，SEQUENCEFILE，ORC（Optimized Row Columnar）这几种存储格式。

- Hive 体系结构：

1、用户接口：用户接口主要有三个：CLI，Client 和 WebUI。其中最常用的是 CLI，CLI 启动的时候，会同时启动一个 Hive 副本。Client 是 Hive 的客户端，用户连接至 Hive Server。在启动 Client 模式的时候，需要指出 Hive Server 所在节点，并且在该节点启动 Hive Server。WebUI 是通过浏览器访问 Hive。

2、元数据存储：Hive 将元数据存储在数据库中，如 MySQL、Derby。Hive 中的元数据包括表的名字，表的列和分区及其属性，表的属性（是否为外部表等），表的数据所在目录等。

4.5.1.5 HiveQL 介绍

Hive 提供类似 SQL 的 HiveQL（Hive Query Language）语言操作结构化数据，其基本原理是将 HiveQL 语言自动转换成 MapReduce 任务或 Spark 任务，从而完成对 Hadoop 集群中存储的海量数据进行查询和分析，从而让会 SQL 语言但不会编程 MapReduce 的用户也能使用 Hadoop 进行数据处理。

HiveQL，简称 HQL，是 Hive 提供一个 SQL 方言。Hive 查询操作过程严格遵守 Hadoop MapReduce 的作业执行模型，Hive 将用户的 HiveQL 语句通过解释器转换为 MapReduce 作业提交到 Hadoop 集群上，Hadoop 监控作业执行过程，然后返回作业执行结果给用户。

HiveQL 是一种用于 Hive 分析和处理元存储中结构化数据的查询语言。它与 SQL 非常相似，具有高度的可扩展性。它重用关系数据库世界中熟悉的概念，如表、行、列和模式，以简化学习。

HiveQL 主要有如下特点：

- HQL 不支持行级别的增、改、删，所有数据在加载时就已经确定，不可更改。
- 不支持事务。
- 不支持等值连接，一般使用 left join、right join 或者 inner join 替代。

4.5.1.6 大数据计算模式、问题场景、代表产品

大数据主流的计算模式有以下四种：

- 批处理

批处理计算主要解决针对大规模数据的批量处理，也是我们日常数据分析工作中非常常见的一类数据处理需求。

MapReduce 是最具有代表性和影响力的大数据批处理技术，可以并行执行大规模数据处理任务，用于大规模数据集（大于 1TB）的并行运算。

MapReduce 极大地方便了分布式编程工作，它将复杂的、运行于大规模集群上的并行计算过程高度地抽象到了两个函数——Map 和 Reduce 上，编程人员在不会分布式并行编程的情况下，也可以很容易地将自己的程序运行在分布式系统上，完成海量数据集的计算。

- 即席分析查询

适用于大规模数据的存储管理和查询分析，代表产品有 Dremel、Presto、Impala 等。

- 流处理

流数据也是大数据分析中的重要数据类型。

流数据（或数据流）是指在时间分布和数量上无限的一系列动态数据集合体，数据的价值随着时间的流逝而降低，因此必须采用实时计算的方式给出秒级响应。

流处理可以实时处理来自不同数据源的、连续到达的流数据，经过实时分析处理，给出有价值的分析结果。

目前业内已涌现出许多的流计算框架与平台，第一类是商业级的流计算平台，包括 IBM InfoSphere Streams 和 IBM StreamBase 等；第二类是开源流计算框架，包括 Twitter Storm、Yahoo! S4（Simple Scalable Streaming System）、Spark Streaming 等；第三类是公司为支持自身业务开发的流计算框架，如 Facebook 使用 Puma 和 HBase 相结合来处理实时数据。

- 图处理

在大数据时代，许多大数据都是以大规模图或网络的形式呈现，如社交网络、传染病传播途径、交通事故对路网的影响等，此外，许多非图结构的大数据也常常会被转换为图模型后再进行处理分析。

MapReduce 作为单输入、两阶段、粗粒度数据并行的分布式计算框架，在表达多迭代、稀疏结构和细粒度数据时，往往显得力不从心，不适合用来解决大规模图计算问题。

因此，针对大型图的计算，需要采用图计算模式，目前已经出现了不少相关图计算产品。

Pregel 是一种基于 BSP（Bulk Synchronous Parallel）模型实现的并行图处理系统。

为了解决大型图的分布式计算问题，Pregel 搭建了一套可扩展的、有容错机制的平台，该平台提供了一套非常灵活的 API，可以描述各种各样的图计算。Pregel 主要用于图遍历、最短路径、PageRank 计算等。

其他代表性的图计算产品还包括 Facebook 针对 Pregel 的开源实现 Giraph、Spark 下的 GraphX、图数据处理系统 PowerGraph 等。

4.5.1.7 大数据并行计算特点天然匹配鲲鹏多核架构

鲲鹏多核架构能够端到端对大数据平台的计算、存储和网络多方面进行性能提升。下面以优化 MapReduce 过程为例具体说明。

- 传统架构下 MapReduce 的问题

1、Hive 中经常出现 reduce task 启动过早，导致 map 资源被提前占用，整体性能偏差，过晚又导致整体的时间被延长，较大的影响了性能。

2、MapReduce task、vcore、mem 默认设置可能会导致实际运行过程中 CPU 和内存资源不能匹配，影响整体性能。

3、Hive 在多个 MapReduce 任务会串行执行，影响整体性能。

4、map 阶段单个 maptask 内存使用过高，导致开启 spill 操作，增大了磁盘压力，也降低了 MR 任务的效率。

- 鲲鹏架构优化方案

1、分析用例，合理设置 Reduce Task 的启动时机，使得计算和 I/O 达到平衡，减少端到端系统执行时间，提升性能约 10-20%。

2、根据应用类型（CPU 密集/内存计算型）合理分配 dfs 的 block size、MR 阶段的 vcore 和 mem 信息，使资源在各阶段能高效执行，提升性能约 10-20%。

3、分析用例，在无依赖关系可以并发的地方开启并发，充分地拆解任务，并设置合理的并发任务数较复杂的 SQL 可以使用，单条语句提升约 5%。

4、设置合理的 io sort mem 和 factor 值，减少 map 时对磁盘 spill 操作，提升约 5%。

4.5.1.8 平台数据分析流向图

对于本实验中的鲲鹏即时交流平台而言，由前端应用侧产生的平台数据会存储在 MySQL 数据库当中，当我们要基于即时交流数据进行数据分析时，需要先将数据转存至 Hadoop，之后再配合 Hive 完成数据处理分析、热词检索等操作。

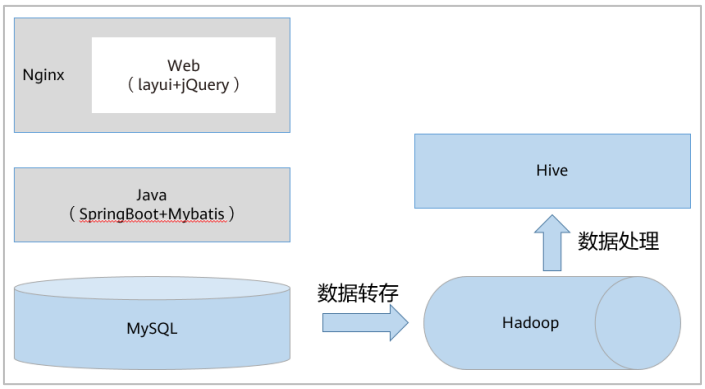


图4-10 平台数据分析流向

4.5.2 性能调优思路

4.5.2.1 调优原因

- 组件参数默认值保守

应用程序和操作系统为了兼容不同环境，涉及性能的参数默认值较小，不能发挥集群资源的最大性能。

- 合理配置上下游组件的资源分配

同一套大数据集群环境中会安装不同的组件，而不同组件对 CPU、磁盘、网络等资源需求不同，需合理配置。

- 性能瓶颈因硬件配置而异

因硬件环境常无法统一，当某个硬件资源提前到达瓶颈，需根据实际硬件配置进行针对性的调优。

4.5.2.2 常用调优思路

调优思路可以分为两大类，一种是基础调优操作，目的是确保集群拥有较优的性能；一种是重复资源监控、确定瓶颈、优化动作，从而针对性解决问题，提升性能。

- 保障测试压力

当客户端压力不足以发挥大数据集群的性能时，需优先提高客户端压力。

- 分配物理资源

根据组件特点，尽可能多地分配该组件依赖的物理资源（CPU、磁盘、内存、网络等）。

- 监控资源使用情况

使用性能监控工具观察系统状态并进行记录，如 CPU、磁盘、内存、网络、应用程序 GC 状况、热点函数等。

- 确定性能瓶颈

基于组件、应用程序特点和监控数据识别性能瓶颈，瓶颈可能是物理资源、组件参数、测试工具、测试组网、JVM、锁等。

- 实施优化

根据识别的瓶颈针对性地进行优化，其中，优化手段有时并不会生效，需进一步确定是否锁定瓶颈及优化手段是否正确。

4.5.2.3 常见调优问题介绍

大数据的调优问题可以分为三个层面，分别是应用层面、硬件层面和客户端层面。

其中应用层面的常见问题有 CPU 占用率低、内存耗尽但 CPU 资源富宇等问题；硬件层面问题有网络 IO 占用率高、内存占用多等问题；如果组件参数已最优但性能仍不好则有可能是客户端问题。

Hadoop 由 HDFS、Yarn、MapReduce 三个核心模块组成，分别负责分布式存储、资源分配和管理、分布式计算，对于这三个模块的调优也有对应思路。

Hadoop-HDFS 模块负责分布式存储，对于这一模块的调优，可以考虑尽可能保证 DataNode 节点间的磁盘性能统一，并从磁盘 IO 和网络 IO 两方面进行优化。

Hadoop-Yarn 模块负责资源分配和管理，对于这一模块的调优，尽可能将 Slaves 的 CPU 和内存资源提供给 Yarn 管理和使用，并根据应用程序的数据量，适当调整 Container 内存大小，将所有 CPU 核都利用起来，发挥鲲鹏多核的优势。

Hadoop- MapReduce 模块负责分布式计算，对于这一模块的调优，集群同时运行的 Container 数量在 Yarn 参数确定后确定，适当调整 Map 和 Reduce 任务的数量与 Reduce 任务启动的时间，保证总任务数能覆盖到所有 CPU 核，避免资源浪费。

4.5.2.4 性能定位：问题定位流程

性能定位需要用到一系列的资源监控工具：

nmon：支持收集一段时间内，整机的 CPU、磁盘、网络、内存等各项资源的使用情况。

perf：获取指定进程内的调用情况、各线程调用的 CPU 资源消耗情况，并支持生成火焰图。

Dstat：监控系统整体的性能信息，包括 CPU、磁盘、网络、分页等。

top：监控进程和整机的 CPU、内存资源消耗情况，并支持查看每个 CPU 核的使用情况。

iostat：监控每块磁盘的读写次数、数据量大小、使用率。

sar：监控每张网卡的网络 IO 读写次数和数据量大小。

jstat：JVM 统计监控工具，监控 Java 应用程序的资源和性能，包括堆大小和应用程序 GC 状况。

Java VisualVM：图形化的 Java 程序性能分析工具，能监控应用程序性能消耗、GC 状况、线程堆信息等。

根据性能定位工具给出的数据，我们可以对问题进行定位，判断问题出在以下哪一个部分。

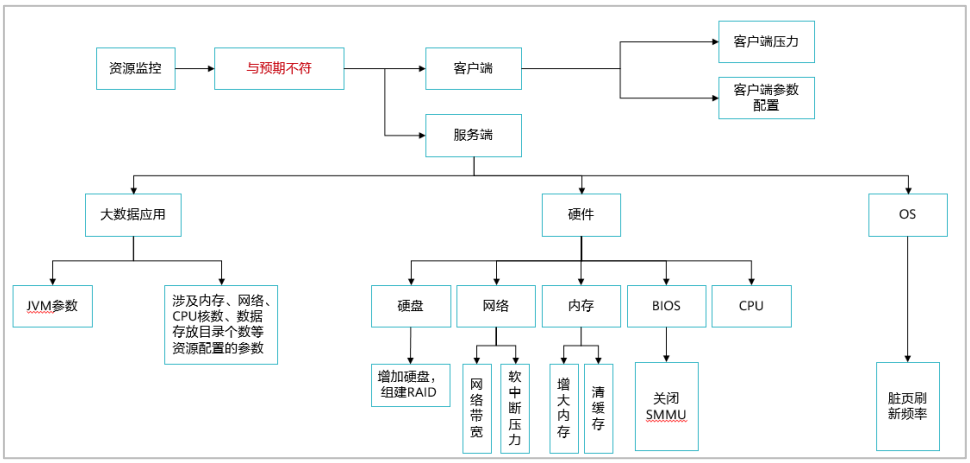


图4-11 问题定位流程

4.5.2.5 性能定位：调优流程

针对不同的性能分析结果，我们可以采取对应的调优动作，具体参考下图。

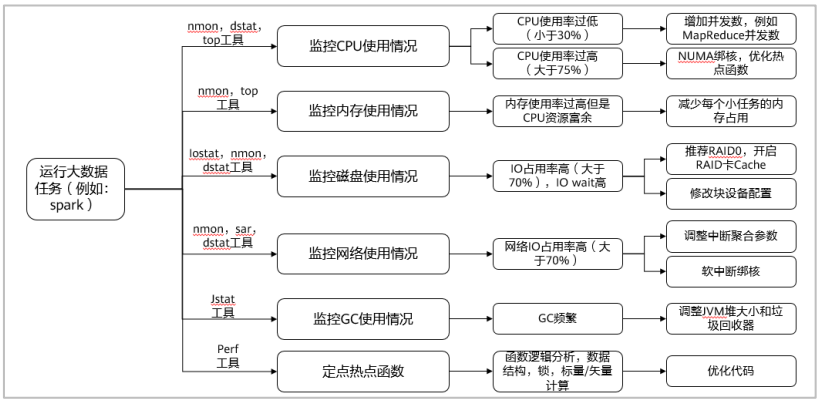


图4-12 调优流程

4.5.3 实验流程介绍

4.5.3.1 任务流程

下面介绍调优的任务流程，一共分为四个步骤：

1、部署 Hadoop、Hive

Hadoop 作为大数据分析工具，为海量的数据提供了存储和计算。Hive 是基于 Hadoop 的数据仓库工具，将结构化的数据文件映射为一张数据库表，并提供 SQL 查询功能，方便进行数据分析工作。

2、使用 Hive 进行数据分析

结合实验手册中设置的数据分析测试题，练习编写 HQL 完成数据分析任务。

3、部署鲲鹏性能分析工具

依照步骤安装鲲鹏性能分析工具。

4、使用鲲鹏性能分析工具优化

使用鲲鹏性能分析工具进行系统扫描，根据优化建议修改优化项并验证。

4.5.3.2 Hadoop、Hive 部署步骤

- Hadoop 部署步骤

1、部署环境配置；

2、安装 Hadoop；

3、修改环境变量、配置文件；

4、启动与验证 Hadoop。

- Hive 部署步骤

1、MySQL 配置；

2、安装 Hive；

3、修改环境变量、配置文件；

4、启动与验证 Hive。

5 华为鲲鹏解决方案

5.1 华为鲲鹏解决方案全景

5.1.1 华为鲲鹏解决方案全景

当前，全球进入以数据为关键生产要素的数字经济时代，数字化应用的蓬勃发展促使数据资源发生新变化，数据体量爆发式增长、数据结构多样性演变、数据资源泛在分布以及每比特数据应用成本不断下降。数据要素变化引发计算技术演进，异构、极致、泛在、协同、绿色、普惠成为数字经济时代“新计算”特征，为全球计算产业带来发展新机。鲲鹏计算产业致力于打造具有全球竞争力的计算产业集群，通过对业界提供基于鲲鹏处理器的软硬件基础设施，高效满足市场对“新计算”的需求，支撑全社会数字化转型。同时，鲲鹏计算产业的落地将带动计算产业链集聚发展，促进产业协同创新，激发地方投资就业活力，形成数字经济发展新动能。

中国加速新型基础设施建设，以新发展理念为引领、以技术创新为驱动、以信息网络为基础，面向高质量发展需要，打造升级、融合、创新的基础设施体系。各省份陆续出台政策和规划，加速产业数字化转型。作为数字经济发展的支撑底座，计算产业在此次浪潮中将迎来巨大的发展机遇。

ICT 技术的蓬勃发展加速计算应用创新，面对自动驾驶、云游戏、智能制造、人工智能、物联网等应用场景的多样化和数据类型的多元化趋势，传统单一架构难以满足数据处理和用户体验要求，对计算平台提出新的挑战，驱动计算架构向多样性发展。其中，以 ARM 为代表的 RISC 通用架构处理器，在场景多样化计算时代具备显著优势。例如在分布式数据库、大数据、Web 前端等高并发应用场景，单芯片核数更多的 ARM 架构处理器相比传统处理器拥有更好的并发处理效率。此外，绝大多数移动终端采用 ARM 架构处理器，端云同构为开发人员在整个生态系统的编写与优化上提供便利，而且能够降低异构环境开发所造成的性能损失和潜在漏洞风险。随着边缘计算的兴起和 5G 技术的快速发展，云-边-端协同将成为主流，ARM 计算平台的协同优势将更加突显。在高性能计算领域，以 ARM、RISC-V 为代表的多样性计算平台也逐渐发挥重要作用，例如欧盟 EPI（欧洲处理器计划）项目致力于打造本土基于 ARM 架构核心处理器和 RISC-V 架构加速器芯片的百亿级超级计算机；日本“富岳”超算系统采用自主开发的 ARM 架构处理器，成为全球首台基于 ARM 芯片的 TOP500 冠军超级计算机。未来，不同应用领域间的巨大差异将催生更多的细分生态，结合上层应用的负载特征、调配处理效率最优的底层计算资源、实现计算资源利用率的最大化，将成为计算产业发展方向。

新计算发展的成功关键在于构建完善的生态体系。鲲鹏计算产业是基于鲲鹏处理器的基础软硬件设施、行业应用及服务，涵盖从底层硬件、基础软件到上层行业应用的全产业链条。纵观鲲鹏计算产业生态全景，硬件方面，围绕鲲鹏处理器，涵盖包括昇腾 AI 芯片、智能网卡芯片、底板管理控制器（BMC）芯片、固态硬盘（SSD）、磁盘阵列卡（RAID 卡）、主板等部件以

及个人计算机、服务器、存储等整机产品。基础软件方面，涵盖操作系统、虚拟化软件、数据库、中间件、存储软件、大数据平台、数据保护和云服务等基础软件及平台软件。行业应用方面，鲲鹏计算产业生态覆盖政府、金融、电信、能源、大企业等各大行业应用，提供全面、完整、一体化的信息化解决方案。

华为作为鲲鹏计算产业的成员，聚焦计算架构创新、处理器和开源基础软件的研发，以及华为云服务，致力于推动鲲鹏生态发展。通过战略性、长周期的研发投入，吸纳全球计算产业的优秀人才和先进技术，持续推进全栈计算技术的创新发展，加快构筑面向多样性计算的全球开源体系与产业标准。基于“硬件开放、软件开源、使能合作伙伴”的策略推动鲲鹏计算产业发展。

基于华为鲲鹏计算平台（云平台或物理平台）的解决方案分为通用解决方案及行业解决方案，通用解决方案包括高性能计算 HPC、大数据基础设施、分布式存储、企业核心应用等；行业解决方案包括为运营商、政府、金融等行业提供的针对性解决方案。

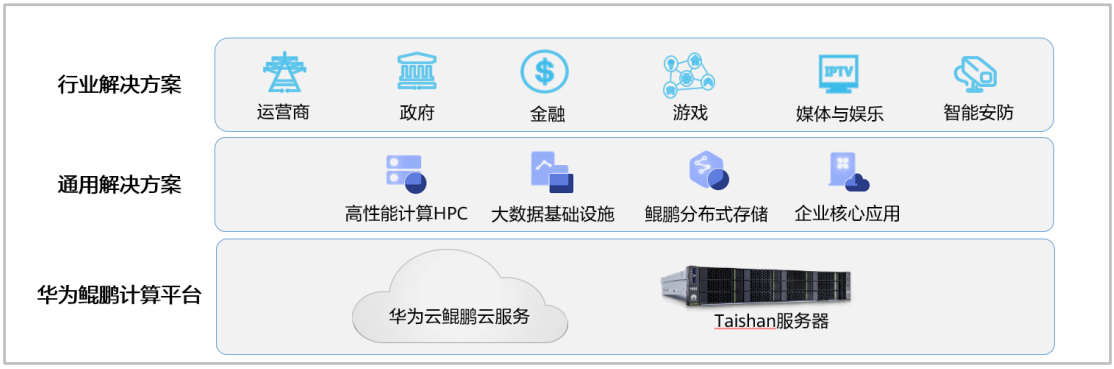


图5-1 华为鲲鹏解决方案全景

5.1.2 华为鲲鹏云全栈解决方案竞争力

华为鲲鹏云解决方案全栈基础设施、云服务、长期生态投入，做到了全栈鲲鹏云生态、全栈鲲鹏云服务，全栈鲲鹏云基础设施。

云上适配各类鲲鹏云服务，包括 ECS、BMS、容器、RDS、Redis 等，从芯片、基础设置、操作系统到场景化解决方案，覆盖全栈解决方案。

鲲鹏全栈解决方案深入行业，与 50+行业 ISV 建立高度合作。

5.1.3 Taishan 服务器覆盖主流应用场景

Taishan 服务器高性能、多核高并发、高吞吐和原生算力同构的优势。适合为大数据、分布式存储、原生应用、高性能计算和数据库等应用高效加速，旨在满足数据中心多样性计算、绿色计算的需求。Taishan 服务器解决方案能够覆盖数据中心主流应用场景。

在过去的 3 年时间里，华为与各行业的 ISV 共同孵化解决方案，在优势场景进行了深度优化，为政府、金融、运营商、电力、互联网等广大行业客户提供了基于鲲鹏处理器的数据中心基础设施和服务。

5.2 华为鲲鹏通用解决方案

5.2.1 高性能计算 HPC 解决方案

5.2.1.1 高性能计算 HPC 解决方案概述

在工业电子信息化时代，高性能计算发展由单机处理发展到集群化，企业主要以自建高性能计算集群为主，但在企业生产及使用方面存在诸多方面的问题：

高性能计算集群建设属于重资产，建设成本高、周期长，往往无法满足企业灵活的业务形态，造成资源的浪费或不足；

维护工作繁重、压力大 ，集群维护需要企业投入很多物力、人力，导致企业无法全力聚焦企业自身的业务；

传统自建集群方式下，企业需要自建机房，包括水费、电费、系统维护费等会占用较高的运营成本；

硬件设备更新迭代速度快，计算量快速膨胀，规划赶不上变化，新技术层出不穷，设备老化严重，硬件更新速度快，无法快速适应新的业务需求。

随着工业云时代的兴起，PC、工作站、集群机全面云化，高性能计算云化发展是大势所趋；云计算的到来，使数据和计算全部可以在云上完成，企业数据不下云，内部协同更高效，减少企业重资产以及维护工作，使客户聚焦在自身的业务发展上。

HPC（High Performance Computing，高性能计算）是一个计算机集群系统，它通过各种互联技术将多个计算机系统连接在一起，利用所有被连接到系统的综合计算能力来处理大型计算问题，所以又通常被称为高性能计算集群。

高性能计算云解决方案（HPC Cloud）是利用云部署方式来提供一种高效、可靠、灵活、安全的计算服务；能够为工业设计、海量数据处理等场景提供卓越的计算服务；帮助客户降低TCO，部署周期短，成本低，缩短产品上市周期， 提升企业产品竞争力。

华为 HPC 解决方案主要分两层：

1、华为主要提供相关的计算所需要的 IAAS 资源，包括计算、存储、网络等资源。

2、集群调度层在不同的应用场景使用的软件略有不同，如工业仿真场景更多的使用的 PBS，而针对基因测序使用的更多的是开源软件 Sge 以及 Slurm。

解决方案核心价值与优势如下：

1、部署周期短，成本低。

2、部署灵活，不会造成资源的浪费或不足。

3、公有云方式维护成本低。

4、满足技术快速发展要求。

解决方案瞄准内存敏感型应用及整型计算（或者单精度计算）的应用，典型应用场景如下：

科学计算、气象预测、环境预测、工业仿真、基因测序、能源勘探、动漫渲染和金融分析等等。

5.2.1.2 高性能计算 HPC 解决方案优势

- 高性能

基于鲲鹏计算平台，鲲鹏 920 处理器，最高 64 核，8 内存通道，适合访存密集型及整型计算为主应用，华为智能网卡、SSD、Atlas 加速卡，加速高性能计算应用，提升客户应用性能表现。

- 高能效

板级液冷，内存间走水，扰流强化换热，全液冷机柜散热，PUE 低至 1.05，降低客户管理运维成本。

- 安全可靠

自研芯片、操作系统、MPI、编译器、数学库等，掌握关键核心技术，提供产品可持续供应能力，遵从商业环境要求，信息安全主动防护，保障客户放心部署应用。

- 开放生态

开放合作，与 ISV、合作伙伴、开发者、产业联盟共建全栈生态，兼容主流企业应用软件，支持丰富应用快速平滑移植，降低客户迁移成本。

5.2.1.3 HPC 解决方案性价比高

在公有云上，HPC 可以按需发放，快速部署与扩容，加速产品上市时间和缩短科研周期。公有云的 HPC 可以按需租用，成本低，降低中小客户使用 HPC 的门槛。基因测序、流体力学、气象环保主要为内存敏感型应用以及整型计算（或者单精度计算）为主，适合鲲鹏云 HPC 应用场景。

5.2.1.4 其他优势特性

- 多组织、区域租户共享

动态申请，动态共享，计算节点按需申请、释放；

按需租用，避免过度投资，避免重复建设；

用户隔离措施保障安全性；

数据集中，跨组织和区域的合作共享。

- 自服务能力

自动发放虚拟机、云化裸机，自动创建集群，长时间自动状态检测；

将各种不同的 HPC 应用模板进行初始化导入，在 VM 模板中部署 MPI 库、编译库及优化配置等。

- 部署方式灵活

节省建设周期，近乎无限的基础架构；

虚机/云化裸机、各计算/存储实例灵活可选。

5.2.1.5 华为鲲鹏面向 HPC 研发创新技术点

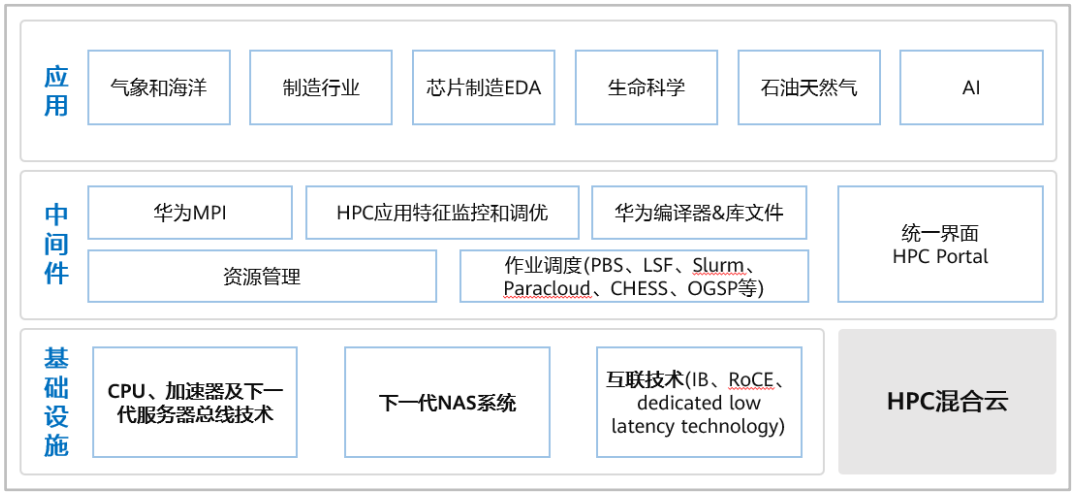


图5-2 华为鲲鹏 HPC 解决方案

华为鲲鹏 HPC 混合云解决方案中包含很多技术创新点。

在基础设施层面：采用搭载有鲲鹏系列芯片的 CPU 的服务器作为主机环境，内置多种加速器，可支持下一代服务器总线技术，存储设备支持构成下一代 NAS 系统，支持多台主机互联，可支持的互联技术有 IB、RoCE、特定的低延时技术等。

在中间件层面：提供统一的华为 MPI 支持调用，统一的操作界面 HPC Portal，丰富的华为编译器和库文件，鲲鹏开发套件 Devkit 支持应用开发、编译、调测，基于集群管理与调度组件可实现灵活的资源管理、性能作业调度（组件包括 PBS、LSF、Slurm、Paracloud、CHESS、OGSP 等）、HPC 应用特征监控和调优等。

在应用层面：基于鲲鹏基础设施，配合完善的平台组件功能共同实现的 HPC 解决方案可应用于气象和海洋、制造行业、芯片制造 EDA、生命科学、石油天然气、AI 等诸多应用场景。

5.2.1.6 华为自研鲲鹏（ARM）HPC 方案加速科研创新

教育科研领域中广泛使用高性能计算集群对高能物理、天文探索、地球物理、量子化学、分子动力学等前沿方向加速探索。通过数学模型构建、定量分析方法以及利用计算机来分析和解决传统实验科学中成本高昂的实验与无法模拟的实验环境，助力前沿理论科学进行快速迭代与验证，已经成为助力高校研究团队提升科研水平的最有力工具之一。华为鲲鹏 HPC 解决方案通过高性能、高能效比、自主可控等优势助力教育科学 HPC 的快速发展。

5.2.1.7 气象 HPC 鲲鹏云解决方案

气象场景下的高性能计算有以下挑战：

- 计算量巨大

中尺度气象预报模式（MM5、WRF、GRAPES 和 AREMS）有着惊人的计算量，同时由于气象预报的特点决定了其极高的实时性，要求模式必须在指定的时间内完成运算（一般不超过 2 小时）。另一方面，人们对气象预报的精度提出了越来越高的要求，目前预报精度从几百公里、几十公里提高到几公里，这大幅度提高了模式的计算量。预报精度提高一倍，其所需计算量将提高 16 倍。数值气象预报对计算的这一需求，靠单个 CPU 或普通的计算机根本不可能完

成，必须利用并行计算。一方面，将模式预报软件通过消息传递或者共享存储的方式并行化，另一方面需要高性能并行计算机。目前绝大部分中尺度气象预报模式都已经完成了并行化，如MM5、WRF 既支持 MPI 消息传递并行，又支持 OpenMP 共享存储内存并行，也支持MPI+OpenMP 的混合运行模式。GRAPES 只支持 MPI 并行模式，AREMS 也完成了 MPI 并行化工作。

- 通讯极为密集

由于模式都是并行软件，同时一般都采用有限差分格点模式并行计算，所以运行中尺度气象预报模式时，各个 CPU 之间的通讯量很大，模式对通讯的性能要求非常高。如 MM5、WRF 的通讯既包括母域和嵌套域之间的域间通讯，又有各个域内部不同数据划分之间的通讯。所以这就要求高性能计算机有高性能的通讯网络。

- 实时性强、定时运行

气象预报本身的特点决定了其要求很强的实时性。同时，预报系统要求定时定点自动运行，无需人工干预。一般每天在固定的 2-4 个时段运行，每个时段 2 小时内运行完。这些作业每天在相同时刻运行，必须保证这些模式可以按时计算完毕。

- 主模式在于计算量

从软件的处理流程上看，一般分为前处理、主模式和后处理。前处理包括资料的下载、数据同化等，后处理主要是指图形化处理生成产品等，前/后处理一般对计算机要求不是太高。主模式是整个系统的主要部分，也是主要计算量所在，这个部分是对计算机性能要求非常高。

华为云 HPC 解决方案是将 HPC 以服务的形式交付用户 HPC 资源，用于解决气象同化处理、海量数据处理等场景所需的大规模计算需求。华为云 HPC 解决方案针对 HPC Workload 进行优化，实现 HPC 资源按需租用，帮助客户降低 TCO，提升气象预报的效率和准确度，提升气象业务快速向智能化、云化、和数字化转变。并支持 x86、GPU、FPGA 和鲲鹏混合部署，基于高性能的鲲鹏有 46%内存带宽提升；同时，支持统一管理和编排物理资源、多类型裸机、虚机、容器资源、公有云等多种资源，实现“一个气象一朵云”。

5.2.1.8 鲲鹏加速 CAE/CFD、气象和基因仿真应用性能

生命科学领域广泛运用脑科学、基因组学、表观遗传学、蛋白质组学等手段进行科学研究，再借助数学分析与模拟、生物信息学等对整个生命系统行为的精准理解和预测，其需要海量基因数据的分析与挖掘，需要高性能的计算集群才能满足需求。生物医药行业和脑科学领域中分子建模，数据管理，药物分子设计、基因测序、冷冻电镜等，基于华为鲲鹏 HPC 解决方案，全栈自主可控，极大地加速新药研发的效率和节约运算时间。

- 基因测序

基因测序技术是一种新型基因检测技术，能够从血液或唾液中分析测定基因全序列，预测罹患多种疾病的可能性，已经广泛应用于医疗和非医疗等各个领域，如产前检查，遗传检测，癌症靶向治疗等，高通量低成本的测序技术让基因测序走进千家万户。

基因数据分析过程根据研究目的不同，会有 I/O 密集型、CPU 密集型、内存密集型等各种计算实例需求，需要海量基因数据的分析与挖掘。目前已经有 GATK、Ont-tombo 、HISAT2 等 50+应用完成鲲鹏适配。

- 冷冻电镜

冷冻电镜（cryo-electron microscopy，cryo-EM）技术是将生物大分子在毫秒时间尺度内快速冷冻在玻璃态的冰中，应用低温透射电子显微镜收集生物大分子的二维投影，并利用三维重构的方法得到大分子三维精细结构的生物物理学技术。对计算密集度极高：包括图像数据校正、有效大分子颗粒识别、颗粒聚类、空间构造和数据转换，一次作业的数据处理时间往往需要数天。高速 I/O：需要在很短的时间读入包含数百 GB 数据的大文件；relion、 eman2 等已经完成鲲鹏适配迁移，性能更优。

5.2.1.9 鲲鹏 HPC 开源生态

OpenHPC 是一套基于 Linux 的 HPC 的社区驱动的 FOSS（Free and open source software，自由及开放源代码软件）工具。OpenHPC 对硬件没有特殊要求。

OpenHPC 提供了一个集成且经过测试的软件组件集合，以及支持的标准 Linux 发行版，可用于实现功能齐全的计算集群。组件涵盖整个 HPC 软件生态系统，包括配置和系统管理工具，资源管理，I/O 服务，开发工具，数值库和性能分析工具。

5.2.1.10 鲲鹏 HPC 已验证平台及应用

HPC 是计算领域金字塔尖上的一颗明珠，从气象预报到大飞机制造到新冠疫苗研发都需要用到，关系到科学进步、国计民生。鲲鹏 HPC 已经完成气象海洋、制造仿真、生命科学以及基础科研等多个科研领域的平台验证。

5.2.2 大数据基础设施解决方案

5.2.2.1 大数据的技术种类和代表场景

大数据主流的计算模式有以下四种：

- 批处理

批处理计算主要解决针对大规模数据的批量处理，也是我们日常数据分析工作中非常常见的一类数据处理需求。

MapReduce 是最具有代表性和影响力的大数据批处理技术，可以并行执行大规模数据处理任务，用于大规模数据集（大于 1TB）的并行运算。

MapReduce 极大地方便了分布式编程工作，它将复杂的、运行于大规模集群上的并行计算过程高度地抽象到了两个函数——Map 和 Reduce 上，编程人员在不会分布式并行编程的情况下，也可以很容易地将自己的程序运行在分布式系统上，完成海量数据集的计算。

- 即席分析查询

适用于大规模数据的存储管理和查询分析，代表产品有 Dremel、Presto、Impala 等。

- 流处理

流数据也是大数据分析中的重要数据类型。

流数据（或数据流）是指在时间分布和数量上无限的一系列动态数据集合体，数据的价值随着时间的流逝而降低，因此必须采用实时计算的方式给出秒级响应。

流处理可以实时处理来自不同数据源的、连续到达的流数据，经过实时分析处理，给出有价值的分析结果。

目前业内已涌现出许多的流计算框架与平台，第一类是商业级的流计算平台，包括 IBM InfoSphere Streams 和 IBM StreamBase 等；第二类是开源流计算框架，包括 Twitter Storm、Yahoo! S4（Simple Scalable Streaming System）、Spark Streaming 等；第三类是公司为支持自身业务开发的流计算框架，如 Facebook 使用 Puma 和 HBase 相结合来处理实时数据。

- 图处理

在大数据时代，许多大数据都是以大规模图或网络的形式呈现，如社交网络、传染病传播途径、交通事故对路网的影响等，此外，许多非图结构的大数据也常常会被转换为图模型后再进行处理分析。

MapReduce 作为单输入、两阶段、粗粒度数据并行的分布式计算框架，在表达多迭代、稀疏结构和细粒度数据时，往往显得力不从心，不适合用来解决大规模图计算问题。

因此，针对大型图的计算，需要采用图计算模式，目前已经出现了不少相关图计算产品。

Pregel 是一种基于 BSP（Bulk Synchronous Parallel）模型实现的并行图处理系统。

为了解决大型图的分布式计算问题，Pregel 搭建了一套可扩展的、有容错机制的平台，该平台提供了一套非常灵活的 API，可以描述各种各样的图计算。Pregel 主要用于图遍历、最短路径、PageRank 计算等。

其他代表性的图计算产品还包括 Facebook 针对 Pregel 的开源实现 Giraph、Spark 下的 GraphX、图数据处理系统 PowerGraph 等。

5.2.2.2 支持开源等多种大数据平台商用 - 生态开放

大数据平台主要提供四大功能：

1、数据采集，通过 kafka 完成实时数据采集，汇总数据到平台中；

2、数据存储，通过 HDFS 和 HBase 完成海量数据存储；

3、数据处理计算，通过 MapReduce（批处理）、spark（内存计算）以及 Storm（实时流处理）三个处理引擎完成各场景数据的处理和计算；

4、数据应用：通过 Hive 实现数仓建设以及类 SQL 查询，通过 ElasticSearch 对海量文档数据进行实时检索以及 Redis 对 KV 数据进行内存查询。

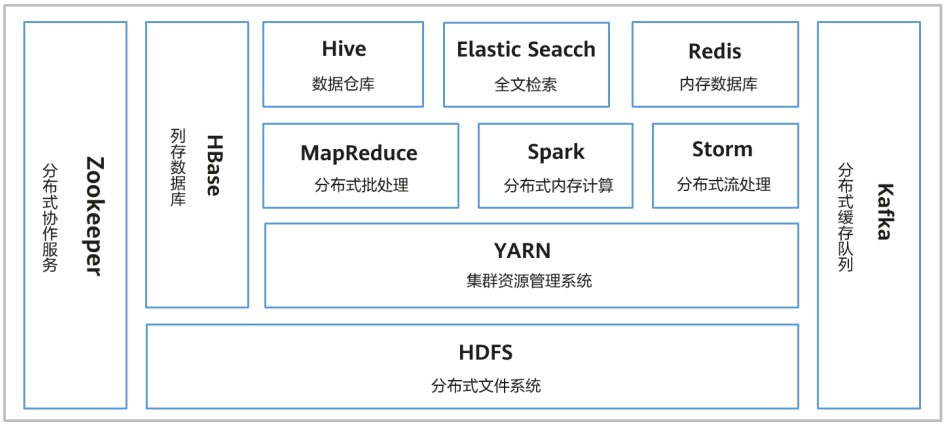


图5-3 大数据组件关系分布

以下是各组件的详细介绍：

HDFS Hadoop Distributed File System，简称 HDFS，是一个分布式文件系统。HDFS 是一个高度容错性的系统，适合部署在廉价的机器上。HDFS 能提供高吞吐量的数据访问，非常适合大规模数据集上的应用。

HBASE 是 Hadoop 的数据库，一个分布式、可扩展、大数据的存储。是为有数十亿行和数百万列的超大表设计的，是一种分布式数据库，可以对大数据进行随机性的实时读取/写入访问。提供类似谷歌 Bigtable 的存储能力，基于 Hadoop 和 Hadoop 分布式文件系统（HDFS）而建。HDFS 是 Apache 的 Hadoop 项目的子项目，HBase 利用 Hadoop HDFS 作为其文件存储系统。HBase 位于结构化存储层，Hadoop HDFS 为 HBase 提供了高可靠性的底层存储支持。除了 HBase 产生的一些日志文件，HBase 中的所有数据文件都可以存储在 Hadoop HDFS 文件系统上。

Redis 是一个高性能的 key-value 存储系统，和 Memcached 类似，它支持存储的 value 类型相对更多，包括 string（字符串）、list（链表）、set（集合）和 zset（有序集合）。Redis 的出现，很大程度补偿了 memcached 这类 key/value 存储的不足，在部分场合可以对关系数据库起到很好的补充作用。

Kafka 是一种高吞吐量的分布式发布订阅消息系统，它可以处理消费者规模网站中的所有动作流数据，目前已成为大数据系统在异步和分布式消息之间的最佳选择。

MapReduce 是一种编程模型，用于大数据集（大于 1TB）的并行运算。在 MapReduce 程序中计算的数据可以来自多个数据源，如 Local FileSystem、HDFS、数据库等。最常用的是 HDFS，可以利用 HDFS 的高吞吐性能读取大规模的数据进行计算。同时在计算完成后，也可以将数据存储到 HDFS。

Spark 是一个高速、通用大数据计算处理引擎。拥有 Hadoop MapReduce 所具有的优点，但不同的是 Job 的中间输出结果可以保存在内存中，从而不再需要读写 HDFS，因此 Spark 能更好地适用于数据挖掘与机器学习等需要迭代的 MapReduce 的算法。通常，Spark 中计算的数据可以来自多个数据源，如 Local File、HDFS 等。最常用的是 HDFS，用户可以一次读取大规模的数据进行并行计算。在计算完成后，也可以将数据存储到 HDFS。它可以与 Hadoop 和 Apache Mesos 一起使用，也可以独立使用。

Storm 是 Twitter 开源的一个类似于 Hadoop 的实时数据处理框架。编程模型简单，显著地降低了实时处理的难度，也是当下最人气的流计算框架之一。与其他计算框架相比，Storm 最大的优点是毫秒级低延时。

Hive 是基于 Hadoop 的一个数据仓库工具，可以将结构化的数据文件映射为一张数据库表，并提供简单的 sql 查询功能，可以将 sql 语句转换为 MapReduce 任务进行运行。 其优点是学习成本低，可以通过类 SQL 语句快速实现简单的 MapReduce 统计，不必开发专门的 MapReduce 应用，十分适合数据仓库的统计分析。

Elasticsearch 能建立全文索引（把文本中的内容拆分成若干关键词，然后根据关键词创建索引。查询时根据关键词查询索引，最终找到包含关键词的文章），它将数据和索引分离，把索引分片，分布式的保存到不同节点，节点可以扩展到上百个，能实时检索、处理 PB 级别的结

构化或非结构化数据。同时分片可以进行副本备份保证数据的可靠性，各分片副本协同工作也大大提高检索性能，且通过简单的 RESTful API 让全文搜索变得高效简单。

YARN 是一种新的 Hadoop 资源管理器，它是一个通用资源管理系统，可为上层应用提供统一的资源管理和调度，解决了旧 MapReduce 框架的性能瓶颈。它的基本思想是把资源管理和作业调度/监控的功能分割到单独的守护进程。

ZooKeeper 是一个分布式的应用程序协调服务，是 Hadoop 和 Hbase 的重要组件。它是一个为分布式应用提供一致性服务的工具，让 Hadoop 集群里面的节点可以彼此协调。ZooKeeper 现在已经成为了 Apache 的顶级项目，为分布式系统提供了高效可靠且易于使用的协同服务。

5.2.2.3 业界的技术趋势 - Hadoop 存算分离之路

Hadoop 一出生就是存算一体，当时设计思想就是存储不动而计算动，负责调度的 Yarn 会把计算任务尽量发到要处理数据所在的实例上，这也是与传统集中式存储最大的不同。为何当时 Hadoop 设计存算一体的耦合？要知道 2006 年服务器带宽只有 100Mb/s 到 1Gb/s，但是 HDD 也即是磁盘吞吐量有 50MB/s，这样带宽远远不够传输数据，网络瓶颈尤为明显，无奈之举只好把计算任务发到数据所在的位置。

分久必合合久必分，随着云计算技术的发展，数据库也开始拥抱云原生时代，在当前越来越强调云原生的环境下，存储计算分离已经是大势所趋，“存算分离”作为一种架构思想在企业项目研发过程中逐渐为大家所熟知和使用，随着数字化转型带来的企业 IT 架构的重塑，存算分离技术将逐渐走入历史的舞台。存算是指存储和计算组成，通常所说的计算是指由 CPU 和内存组成的算力单元，存储指的是持久化的数据存放单元。在企业生产实践过程中没有一成不变的架构，只有不变的以业务为核心的架构意识。

带宽的速度特别是 IDC 机房内带宽的速度，已经从 1000mps、2000mps、10000mps，甚至 100000mpbs，网络带宽提升 100 倍甚至更高，网络已经不再是瓶颈，但是从磁盘吞吐性能上并没有太大提升，仅仅提升 1 倍左右(100MB/s)，由于硬件的变化带来了软件架构的变化。高效的压缩算法与列式存储也进一步减少 I/O 的压力，大数据的瓶颈逐渐由 I/O 变成 CPU。同时集群规模越来越大，存算利用率不均衡，选型受限的问题越来越明显。

随着累积的数据量的增大，大数据业务量的增多，数据存储和处理的成本越来越高，企业数据基础设施的投资越来越大。同时，大数据处理组件多，不同组件使用不同的数据处理格式，比如大家熟悉的数据湖、数据仓库使用的就是不同的格式，多样化的数据格式导致数据存储变得复杂，系统中应对不同的场景，往往同样的数据需要存储多份，不同组件之间还需要大量的数据拷贝和格式转换，消耗大量的资源。经过十几年大数据发展，随着海量负载和大数据用例的出现，单一 Hadoop 集群的规模变大，多个 Hadoop 集群需同时支撑不同的业务。

因此在存储和计算耦合架构下，大数据集群将面临如下问题：

- 成本高

业务中对算力和存储需求是不平衡的，增长速度也是不均衡的。扩容时同时要扩容计算和存储，通常算力是有浪费的。

- 资源利用率低

由于多个 Hadoop 集群承接不同的工作负载，随着支撑业务需求的波动，系统负载出现峰谷，然而存算一体的架构导致各集群的资源完全独立隔离不能共享（跨行业的存算一体架构下

的 Hadoop 集群平均资源利用率在 25%以下）。高密度存储型和算力增强型都难有用武之地；数据会倾斜，计算任务不一定能有效的调度到数据所在实例上。

- 运维困难

随着业务复杂度的增加和新业务上线的速度加快，对服务器资源配比的要求也会随之增加，如果服务器款型繁杂，维护难度就会增大，同时导致机房空间占用多、能耗大。实例要考虑计算与存储的均衡，机器选型受限。缩容较复杂，要对实例上的数据内容做迁移，这种情况无法做到弹性伸缩。

针对传统 hadoop 架构中计算资源和存储资源按某一比例强绑定，系统扩容必须按节点数目增加，导致内存或磁盘浪费的问题。“存算分离”不仅能节约成本，还可以让资源根据业务需求弹性伸缩，按需分配，降低了系统部署和扩展成本，解决了资源利用不均衡的问题。

- 提升资源利用率

“存算一体”架构需要考虑 CPU、内存、存储容量/IOPS/带宽，网络 IO/带宽，多达 7 个维度，任意一个维度的资源不满足就会导致无法满足应用诉求。而“存算分离”，按照计算和存储两个维度，将这 7 个维度拆分为两个领域，降低选型复杂度。分别构建计算资源池和存储资源池，提升资源利用率。扩容可实现计算和存储的按需扩容，节约投资。

- 高可用

外置存储阵列本身有非常好的可靠性设计，如 RAID 冗余、静默数据校验、两地三中心等可靠性保障，其可靠性等级高于服务器至少两个数量级，因此存储应该是一个“长期”的共享存储。服务器的可靠性偏弱，“存算分离”后，即使服务器故障，由于全局共享一份数据，可实现免数据复制快速恢复业务，实现分布式数据库整体的高可用。

- 高性能

采用“存算分离”架构后，由于全局共享一份数据，一些不必要的消耗可以被避免，可提升数据库整体的性能表现。例如半同步，每增加一个从节点都会导致 10%的性能下降。采用“存算分离”后，不再需要半同步技术。外置存储阵列对性能的优化更加深入，全闪存存储，尤其是全 NVME 的存储，可提供极致的性能体验。存储共享资源池不仅可提升资源利用率，还能利用不同应用不同时间对存储性能峰谷要求不同的特点，将所有的 IO 分散到所有的磁盘，实现削峰填谷，达到最佳性能。

5.2.2.4 全分布式存储 - 一份数据支持多种协议，简化接入和使用流程

计算存储合一部署模式，限制于 HDFS 单一接口，在应用访问数据多样化的背景下，逐渐难以应对。不同生态如果使用了多套存储，会造成数据割裂，管理工具不统一造成运维工作量增加。基于这样的背景，华为数据湖提供多协议访问和互通能力，数据只保存一份，避免数据多次拷贝，提高分析效率，且节约数据保存成本。

5.2.2.5 云原生时代 - 计算存储分离简化大数据开发

在大数据开发中，若仍然使用计算与存储融合的形式，整体管理非常复杂。

在计算与存储分离后，计算节点做到 Cloud Native，可根据计算量自动化弹性伸缩；存储按照使用量无感扩容，业务不受任何影响；资源利用率更高，集群管理更容易。

5.2.2.6 BigData Pro 存算分离方案

华为云推出了基于鲲鹏的 BigData Pro 大数据解决方案，以鲲鹏多核高效算力为底座，围绕以 OBS 为存储的数据湖，采用存算分离架构，打造全新的云上大数据解决方案。

BigData Pro 具有以下优点：

1、以鲲鹏多核高效算力为底座，支持弹性云服务器、裸金属、云容器部署方式；

2、围绕以 OBS 为存储的数据湖，计算和存储分离部署，利用高速云网络连接计算与存储，打造全新的云上大数据解决方案。

传统的大数据存算一体方案，计算服务器+存储本地盘，资源配比是固定的，计算和存储的配比受到硬件规格约束；当出现算力瓶颈时，必须计算和存储同时扩容，当业务非等比例增长时，就会导致资源的闲置浪费。据统计，计算和存储的利用率差异较大，大数据集群的计算资源利用率平均只有不到 50%，存储空间利用率更是不足 33%。

存算分离架构让计算和存储资源可以灵活配置，独立弹性扩缩容。

3、通过 OBS 提供的 HDFS、POSIX、S3 等多协议访问能力，打破数据孤岛，大数据应用系统可以直接读写云存储，无需协议转换；

4、统一数据湖使数据从产生、到数据分析、到最终老化归档，全程零数据搬迁。

5.2.2.7 计算 - 鲲鹏助力在 ETL 场景借助多核算力提升性能 30%

鲲鹏架构下的多核算力多核算力领先同 X86 平台 30%，更好满足大数据、HPC 等高并发场景。鲲鹏多核算力单整机最高可到 128 核，最高网络转发性能可达 900 万 RPS，支持极速云盘和 ARM Native。

5.2.2.8 存储 - 单流 300MB、多协议互通、超大总带宽、EB 数据量

BigData Pro 的数据湖存储使用了华为商用存储集群，规模轻易支持到 EB 级。

数据湖存储提升了大数据性能，适配计算集群的规模扩展，其优势如下：

1、数据持久度 12 个 9，99.995%高可用性，多 AZ 多活。

2、按需使用，无感扩容，免除手动扩容麻烦。

3、OBS 单流（单文件）上传和下载最大可达到 300MB/S。

5.2.2.9 部署方案 - Hadoop/Spark 对接 OBS 部署视图

OBSFileSystem 插件基于 OBS 服务，提供 HDFS 的 FileSystem 语义，实现大数据计算 MapReduce、Hive、Spark 等组件与 OBS 对象存储服务对接，为大数据计算提供“数据湖”存储：

场景一：华为云大数据服务 MRS、DLI 已在版本中包含 OBSFileSystem 插件，无需下载安装，即可使用 OBS 作为大数据存储；

场景二：对于 CDH、HDP、开源 Hadoop 等自建大数据场景，需要下载安装并配置 OBSFileSystem 插件，使用 OBS 作为大数据存储。

5.2.2.10 大数据典型客户案例

- 资源高效利用

1、计算资源利用率提升 75%；

2、存储资源利用率提升 140%。

- 性能极致

1、鲲鹏多核算力高并发性能；

2、大数据、分布式场景性能对标 X86；

3、部分组件可提升 10%-20%。

- 安全可控

1、国产自研芯片；

2、芯片级别安全；

3、自主可控。

5.2.3 鲲鹏云手机解决方案

5.2.3.1 什么是云手机

云手机（Cloud Phone，简称 CPH），是基于华为云裸金属服务器虚拟出的带有原生安卓操作系统，同时具有虚拟手机功能的云服务器。简单来说，云手机=云服务器+Android OS。用户可以远程实时控制云手机，实现安卓 APP 的云端运行；也可以基于云手机的基础算力，高效搭建应用，如云游戏、移动办公、直播互娱等场景。

作为一种新型服务，云手机对传统物理手机起到了非常好的延展和补充作用，可以用在诸如 APP 仿真测试、云手游、直播互娱、移动办公等场景，让移动应用不但可以在物理手机运行，还可以在云端智能运行。

面向如 APP 仿真测试等互联网行业场景，单台手机的处理效率非常有限，通过云手机的方式，大幅降低人工操作和设备采购维护成本。

云手机由于应用数据运行在云上，面向政府、金融等信息安全诉求较高的行业，提供更加安全高效的移动办公解决方案。员工通过使用云手机的方式登录办公系统，公私数据分离，同时企业也可对云手机进行智能管理，降本增效的同时，信息安全也更加有保障。

探索游戏、直播行业新可能

云手机还可以为游戏、直播等行业提供全新的互动体验方式，开拓新的商业模式和市场空间。以云手游场景为例，因为游戏的内容实际是在云上虚拟手机上运行，可以提前安装部署和动态加载，所以对于最终玩家来说，游戏可以做到无需下载，即点即玩，大幅提高玩家转换率。同时可以让中低配手机用户也能流畅运行大型手游，增大游戏覆盖的用户范围。

5.2.3.2 华为云鲲鹏云手机典型场景应用

- 云游戏场景

云游戏作为游戏行业的热门发展方向，通过视频流化的方式面向玩家提供免下载，脱离手机性能的一种游戏服务方式，其本身包含了 PC 游戏的流化和移动游戏的流化。云手机作为云端仿真手机可以发挥移动游戏指令同构的优势，在云端承载游戏应用。

架构说明如下：

1、手机游戏 APP 安装在云手机当中，通过将云手机的音视频画面进行流化编码输出到客户端进行显示，同时接收客户端的操作指令控制云手机中的游戏。

2、登录服务器集群采取负载均衡及弹性伸缩设计，能够轻松应对超大规模并发的场景。

3、云手机可分布部署在各大中心节点及边缘云当中，有效降低用户互动体验的时延，做到最佳体验及最优带宽性价比。

- APP 仿真测试场景

通常手机主要面向个人提供服务，但随着移动应用越来越多，数量越来越庞大，企业在特定的场景下也需要大量的运行手机上的移动应用 APP，来完成自动化或智能化的功能，为此需要大量的仿真手机来承载此类 APP 运行。

架构说明如下：

1、手机应用 APP 安装在云手机当中，通过企业事先编排好的编程脚本自动化的控制手机运行一个或多个 APP，通过拟人化的脚本操作，实现多种多样的场景应用。

2、可在弹性云服务器中构建企业自身的云手机管理运维平台、营销系统或自动化脚本平台等，并通过两个独立的 VPC 分别进行控制。

3、云手机中的应用程序 APP 可通过对象存储集中存放，节省大量应用程序安装或更新时的网络带宽消耗。

4、丰富多样的安全与监控服务可给客户业务系统提供齐全且安全的防护措施，保障业务的稳定运行。

- 直播互娱场景

直播互娱是云手机的一个创新应用场景，通过将手机画面直播给多个参与者的方式，提供多人互动的场景应用，提升用户体验和直播效果。

架构说明如下：

1、手机应用/游戏 APP 安装在云手机当中，将单个或多个手机画面合并输出到编码服务器进行集成编码，然后画面复制推流到多个客户端（PC、手机、PAD 等）上进行显示，同时云手机接收一个或多个客户端的操作指令。

2、登录服务器集群采取负载均衡及弹性伸缩设计，能够轻松应对超大规模并发的场景。

- 移动办公场景

随着移动应用的普及，越来越多的企业开始通过移动终端接入办公，但同时也带来了企业数据安全的隐患，采购定制安全手机虽然可以增强安全性，但仍然无法防止敏感数据泄露。基于云手机的移动办公应用可以将企业核心数据留在云端，而仅仅将手机画面向授权员工开放。

架构说明如下：

1、企业应用 APP 上传至对象存储以后，批量安装在云手机当中，通过将云手机的音视频画面进行流化编码输出到客户端进行显示，同时接收客户端的操作指令控制云手机中的应用，企业数据留在云端，更为安全可靠。

2、登录/客户业务服务器集群采取负载均衡及弹性伸缩设计，能够轻松应对超大规模并发的场景。

3、云手机可分布部署在各大中心节点及边缘云当中，有效降低用户互动体验的时延，做到最佳体验及最优带宽性价比。

5.2.3.3 云手机市场常见解决方案对比分析

- X86 模拟器

优势：x86 服务器易于获得，技术门槛低，资源可获得性高；

业务系统架构设计灵活，可充分借鉴市场成熟的产品应用，构建独特竞争力；

可灵活设定仿真手机规格，自由度高。

痛点：需要额外指令转换，多达几倍的性能损失，集群规模性价比很低；

仿真度和应用兼容性较差，应用兼容性问题难以解决。

基于外部各种开源或非商业模拟器软件实现，稳定性难以保证。

- 真机农场

优势：与真机基本一致，仿真度高，应用兼容性好；

技术应用门槛低，易于构建。

痛点：手工焊接与复杂接线难以保障产品质量，稳定性非常差；

二手手机市场变化快，设计对应的手机在市场上的可获得性极差。

- 手机开发板农场

优势：与真机硬件架构基本一致，仿真度高，应用兼容性好；

借助手机芯片高集成度设计，在游戏行业的应用场景下性价比高。

痛点：不能获得高于单一手机芯片的性能，难以满足高性能场景应用，只能基于单台手机性能进行整数向下切割；

供货周期长，供应链不稳定；

基于消费类电子电路进行设计，难以满足公有云的服务可靠性及大规模集群管理需求。

- 华为 TaiShan 服务器

优势：能够灵活调整云手机规格，性能无上限；

能够轻松应对 APP 对手机规格性能不断上升的需求；

基于服务器的硬件设计，更匹配企业级可靠性、稳定性与集群部署能力；

ARM 原生指令支持，兼容性好，性能高。

痛点：基于 ARM 服务器的软硬件设计复杂度高，自行构建技术门槛非常高；

生态应用还比不上 x86，GPU 配套方面还不够完善，需要大厂推动；

硬件发展初期迭代速度快，平均两年升级一代，迭代速度高于三年可用性。

5.2.3.4 华为云鲲鹏云手机技术架构

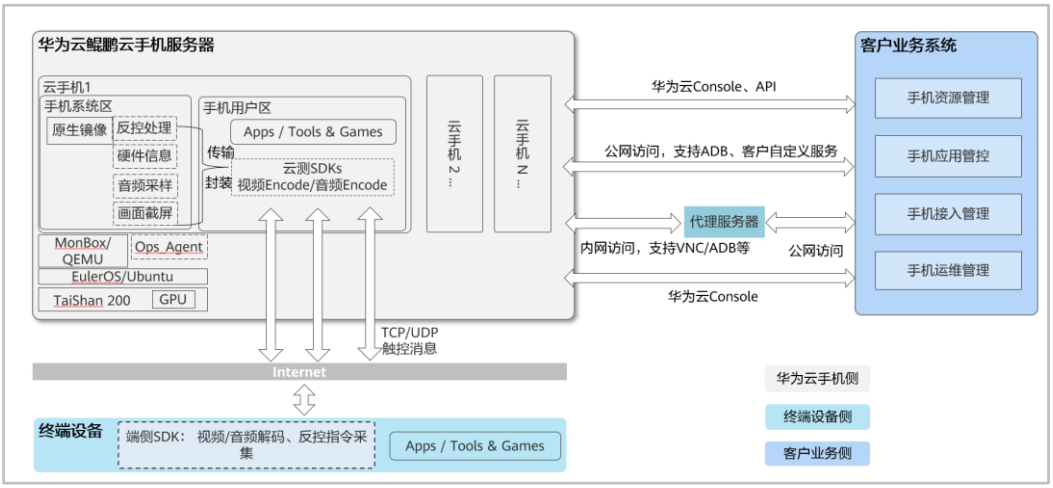


图5-4 华为云鲲鹏云手机技术架构

云手机基于华为 TaiShan ARM 服务器实现，集成多张高性价比的专业 GPU 显卡，可提供专业的图形图像处理能力。TaiShan 服务器中运行了 EulerOS 作为 Host OS，在 Host OS 中通过自研 MonBox 技术生成容器，在容器中运行开源 AOSP 系统，从而虚拟出多台云手机。由于 TaiShan 服务器基于 ARM 架构，而手机系统也基于 ARM 架构，所以减少了指令集转换所带来的翻译算力损耗，可以达到更好的使用体验。

云手机提供了视频、音频以及触控 SDK。客户基于终端设备可以开发相应的 APP，来获取云手机的音频、视频；也可以采集触控指令，如触摸屏、滑动、点击等操作指令，从而在云手机上来执行。

在业务侧，客户通过管理控制台、API、ADB 端口及其他自定义端口可以对云手机进行资源管理、应用管理、运维管理和接入管理等。

- 资源管理：购买、查询云手机。
- 应用管理：云手机应用程序推送、安装、卸载等。
- 运维管理：重启、重置、关机、开机云手机。
- 接入管理：云手机接入认证。

5.2.3.5 原生应用 - 端云同构优势，性能提升 80%

华为云手机支持二进制源码直接迁移上云，不需要将代码重新编译；

华为云手机兼容性极强，对 X86 + Intel Houdini 架构做到模拟器级兼容，对 ARM-Based 架构做到真机级全兼容。

5.2.3.6 自研 kbox 软件仿真架构强大性能提升

基于自研的 Kbox 软件，单服务器云手机密度提升 100%，端到端云手机接入时延持续优化，云手机网络接入流量迭代降低。

Kbox 是华为自研的高密容器仿真云手机方案，基于 Android 框架，实现硬件仿真、GPU 设备直通等功能，使能 Android 云手机。

Kbox 主要实现 vInput、传感器、GPS，Mock（IMEI/WIFI）等硬件仿真功能、GPU 设备直通等功能，使能 Android 云手机容器方案。Kbox 包含二进制和开源补丁两部分组成：

二进制包括：vInput（触屏输入模块），Sensors（Sensor 传感器相关的仿真），GPS（实现 GPS 相关的仿真），IMEI/WIFI Mock（仿真），Gralloc（离屏渲染），HWC（图像合成）。

Demo：MediaFramework，Vold，adbd，Init，netd 基于 Android 系统开源代码基础上提供增量 patch 给客户进行参考。

5.2.3.7 创新指令流分离渲染技术，为大屏带来高清画质

华为独有的 AVS3 视频编码技术，可大幅降低云上渲染带宽消耗，云手机创新指令流分离渲染技术，为大屏带来高清画质。

断线重连 —— 空渲染技术：客户端和服务器端维护相同的 OpenGL 状态机状态，当断线重连时，服务器将完整的状态机同步到客户端，重新建立连接。

快速加载 —— 顶点纹理缓存技术：通过对顶点纹理的特征值进行提取，识别和剔除重复传输的顶点纹理数据。

网络防抖 —— 数据映射技术：将服务器端分配的资源与客户端管理的资源建立 Map 映射，实现控制指令流的单向传输，避免网络时延增大时出现帧率大幅下降。

指令流动态压缩：对控制指令流进行无损压缩，对纹理数据流进行有损压缩，对顶点数据流使用残差数据传输方式。

OpenGL ES 3.0：完善 OpenGL 库 API 适配层，使得能支持 OpenGL ES 3.x 版本的游戏。

5.2.3.8 APP 仿真测试场景典型客户案例

华为云手机在某游戏公司成功为客户带来价值，该游戏公司作为一家专注于手机游戏研发、发行和运营服务的移动游戏公司，公司致力于通过能力、资源、资本联动打造一家世界领先的文创数字内容平台。

使用华为云手机前，有以下痛点：

1、传统服务器部署性能不足，管理复杂；

2、手机农场成本高，升级困难；

3、开发板稳定性差，成本高；

4、需要低成本，易管理，可升级，稳定可靠的系统进行大规模游戏包测试。

华为云手机从这四方面获取了客户的肯定：

1、原生架构处理器，性能保证；

2、服务器集群保障系统可靠稳定；

3、全云化资源易管理；

4、测试执行时间精确可控，计划管理有效，测试数据报告一键收集，效率高。

最终客户获得了以下收益：

测试机故障率从 60%降低至 1%；测试可用时长从 10 小时/日提升至 24 小时/日；测试规模从百级节点并发提升至千级节点并发；测试数据导出时间从小时级降为分钟级。

6 鲲鹏社区

6.1 鲲鹏社区的定位及作用

6.1.1 鲲鹏社区的定位

鲲鹏社区是鲲鹏开发者技术支持和生态使能的一站式资源、服务平台，可提供完善的鲲鹏领域软件资源、专业技能、技术支持、生态政策和产品方案等内容，与社区参与者共建基于鲲鹏计算产业的综合性社区。

6.1.2 鲲鹏社区的作用

一站式的技术支撑平台：

鲲鹏社区为合作伙伴和开发者提供一站式的技术支撑平台，包括最新产业资讯、资源与工具下载、技术支持中心、学习与在线实验等内容，平台将为用户提供全方位的技术支持。

鲲鹏计算产业的对外展示窗口：

鲲鹏社区是鲲鹏计算产业蓬勃发展的对外展示窗口。鲲鹏社区将为用户展示鲲鹏计算产业的最新动态和未来发展趋势，让用户更好地了解鲲鹏计算产业的发展方向和前景。

鲲鹏的产业布局、生态伙伴汇聚、产业进展造势：

鲲鹏社区为鲲鹏的产业布局、生态伙伴汇聚、产业进展造势。鲲鹏社区为用提供最新的鲲鹏产品和解决方案，同时也为用户提供鲲鹏产业生态伙伴的最新进展和合作成果，让用户能更好地了解鲲鹏的产业布局和发展进展。

6.2 鲲鹏社区各版块介绍

6.2.1 产品板块

产品板块包括鲲鹏计算产业概述、鲲鹏处理器、鲲鹏服务器主板及整机产品、openEuler、openGauss、openLooKeng、云服务与解决方案、产品文档等内容。

鲲鹏处理器是华为自主研发的服务器处理器，具有多核、高并发等优点。鲲鹏服务器主板及整机产品则是基于鲲鹏处理器的服务器硬件产品，包括多种不同规格的型号。

鲲鹏云服务是华为云提供的鲲鹏架构云计算服务，包括云主机、云存储、云数据库等多种形态的云服务产品。同时，基于鲲鹏计算平台，产品板块提供了开发者在应用软件移植、部署、调优等场景的技术文档。



图6-1 产品板块

6.2.2 解决方案板块

解决方案板块针对不同行业应用给出了行业概述、需求和挑战分析，并提供了行业中可参考的案例，供开发者学习与借鉴。

6.2.3 开发者板块

开发者板块包括鲲鹏开发套件 DevKit、鲲鹏应用使能套件 BoostKit、鲲鹏 HPC、openEuler 开发资源、openGauss 开发资源、开发资源、资源下载、兼容性查询、开发者文档、开发者计划等内容，为开发者提供一站式的学习支持平台。



图6-2 开发者板块

6.2.4 合作伙伴板块

鲲鹏生态伙伴网络是华为围绕鲲鹏产业技术路线推出的系列生态伙伴计划，它通过提供学习、构建、上市、销售等资源支持，致力于帮助鲲鹏生态伙伴构建产业竞争力、联接客户创造商机，在一个开放、创新、协作、互惠的生态系统里，共同成长、共创未来。

6.2.5 高校板块

高校人才发展社区可以联接校企主体、联接教学要素、联接时空，为教育工作者及学生提供鲲鹏生态相关的教学资源、师资培训、教学实践、智能基座社团和虚拟教研室，打造学习交流专

区。除此之外，为汇聚鲲鹏、昇腾、欧拉生态环境，共筑数字经济发展人才根基，提供多样的人才发展计划，包括但不限于鲲鹏优才实习计划以及 openEuler&openGauss 人才发展加速计划。

6.2.6 鲲鹏解决方案市场板块

一站式软硬件产品、解决方案、伙伴和专家技术服务的展示和互动平台，在这里，用户可以了解并查看 Powered by Kunpeng 精选解决方案，按照华为技术栈产品、软件、行业等维度开始探索鲲鹏合作伙伴解决方案。

6.2.7 产业资讯板块

在本板块可以获取到各种与华为鲲鹏计算产业相关的活动和新闻资讯。比如，华为全联接大会鲲鹏专区、华为伙伴暨开发者大会鲲鹏专区、信创“大比武”鲲鹏赛道、华为开发者大会鲲鹏专区等等。同时，展示鲲鹏官方发布的最新动态或消息，为用户提供关于鲲鹏的第一手资讯。

6.2.8 支持与服务板块

基于计算产品和解决方案，各领域专家使用专业工具，提供卓越的全流程专业服务，加速实现用户的业务价值；资深讲师面授，使能计算人才培养，助力企业数字化转型。

6.3 鲲鹏开发者 - 鲲鹏开发套件 DevKit

面向全研发作业流程，提升应用迁移和调优效率，加速原生开发，提供涵盖代码迁移、开发调试、编译、测试、调优及诊断等各环节的开发使能工具，方便开发者快速开发出鲲鹏亲和的高性能软件，可以通过以下链接查看鲲鹏开发套件的最新动态：
https://www.hikunpeng.com/document/detail/zh/kunpengdevps/wtsnew/wtsnew-kunpengdevps.html



图6-3 鲲鹏开发套件 DevKit

6.3.1 鲲鹏代码迁移工具

代码迁移工具可扫描并自动分析待迁移软件，提供可迁移性评估报告；也可对待迁移软件进行源码分析，准确定位需迁移的代码，并给出友好的迁移指导或一键代码替换；同时支持将 x86 软件包重构成鲲鹏平台软件包、专项软件一键迁移及其他鲲鹏亲和分析等。

6.3.2 鲲鹏开发框架

鲲鹏开发框架是一款使能用户在鲲鹏服务器上进行开发的工具。鲲鹏开发框架充分利用鲲鹏平台的各类型算力及性能更优的第三方组件，提供鲲鹏工程向导、启发式编程、代码亲和检查等能力，一键引入鲲鹏加速库、快速构建鲲鹏应用软件框架，帮助开发者更便捷地开发鲲鹏应用，使能开发者高效创新，可以通过以下链接查看鲲鹏开发框架的最新动态。

6.3.3 鲲鹏编译工具链

毕昇编译器：基于开源 LLVM 开发，并进行了优化和改进，同时支持 Fortran 语言前端，是针对鲲鹏平台的高性能编译器。

毕昇 JDK：基于 OpenJDK 开发，面向 Java 开发者和使用者的一款稳定可靠、高性能、易调测的 JDK，毕昇 JDK 针对鲲鹏平台进行了深度优化和质量增强。

GCC for openEuler：GCC for openEuler 是基于开源 GCC 开发的编译器工具链（包含编译器，汇编器，链接器），在 openEuler 社区开源发布，并通过鲲鹏社区免费提供二进制包，支持包含 ARM、x86 在内的多种处理器架构。

6.3.4 云测试服务

面向开发者提供基于鲲鹏平台的兼容性测试、可靠性测试、安全测试、功能测试、性能测试服务功能，帮助用户快速识别和定位应用程序在运行阶段的问题。

6.3.5 鲲鹏性能分析工具

性能分析工具支持鲲鹏平台上的系统性能分析、Java 性能分析和系统诊断，提供系统全景及常见应用场景下的性能采集和分析功能，并基于调优专家系统给出优化建议。同时提供调优助手，指导用户快速调优系统性能。

6.3.6 华为动态二进制翻译工具（ExaGear）

ExaGear 是华为自研动态二进制翻译工具，通过在运行时，将 x86 应用指令翻译为 ARM64 指令并执行，从而支持 Linux x86 应用无需重新编译就能运行在 ARM64 服务器上，帮助客户将 Linux x86 无源码应用快速迁移到 ARM 服务器上，且能稳定可靠运行。

6.4 鲲鹏开发者 - 鲲鹏应用使能套件 BoostKit

鲲鹏应用使能套件 BoostKit，基于硬件、基础软件和应用软件的全栈优化，提供高性能开源组件、基础加速软件包和应用加速软件包，使能应用极致性能。



图6-4 鲲鹏应用使能套件 BoostKit

6.4.1 大数据场景

聚焦大数据查询效率低、性能优化难等挑战，提供大数据组件的开源使能和调优、IO 智能预取等基础加速软件包、Spark 算法加速库等应用加速软件包，开源 openLooKeng 查询引擎，提升大数据分析效率。

6.4.2 分布式存储场景

聚焦 Ceph 性能低、成本高等挑战，提供 Ceph 参数优化、MD5 摘要算法、IO 直通等特性提升系统性能，并通过压缩算法、Zlib 压缩、EC Turbo 等特性降低存储成本，提供高性价比存储方案。

6.4.3 数据库场景

针对开源 MySQL OLAP 查询效率低、OLTP 场景锁性能问题等挑战，提供 MySQL AP 性能加速和 TP 锁性能优化等基础加速软件包，深度优化了 OLAP 查询分析效率和 OLTP 在线交易事务处理能力，充分发挥多核算力。

6.4.4 虚拟化场景

聚焦虚拟化性能低、网络损耗大、资源碎片严重等痛点，提供虚拟化开源使能调优指南，通过 V-Turbo 和 OVS 加速等特性提升系统性能，使用 NUMA 内存交织等特性减少资源碎片，充分发挥鲲鹏多核优势。

6.4.5 ARM 原生场景

支持移动应用无损上云，完整方案包括硬件、Host OS、ExaGear、虚拟化方案（Kbox）、Guest OS、端云协同引擎。

6.4.6 Web 应用场景

聚焦 Web 应用 HTTPS 连接性能和 Web 开源组件可用性等问题，提供开源 Web 组件使能调优指南，通过使能鲲鹏 RSA 加速引擎，帮助客户提升 Web 网站的安全性，并实现用户 HTTPS 访问的极致体验。

6.4.7 CDN

聚焦 CDN 节点吞吐低和开源组件可用性等问题，提供 CDN 主流开源软件使能调优指南，通过使能鲲鹏 RSA 加速引擎，提供 NUMA 优化等手段，充分发挥鲲鹏多核架构优势，实现更大吞吐，更低时延。

6.4.8 HPC 高性能计算场景

基于多瑙套件，提供大规模 HPC 集群的高效资源管理与调度能力，满足 HPC 场景下的集群资源利用率、作业调度吞吐能力，以及多集群管理等要求。

6.5 鲲鹏开发者 - 学习专栏

6.5.1 鲲鹏在线课程

鲲鹏在线课程是由鲲鹏社区设计的教学课程集合，以 MOOC 方式呈献给鲲鹏软件开发人员。课程的目的是帮助开发人员了解鲲鹏体系的基础知识、软硬件产品、云服务，并学习如何将 x86 应用迁移到鲲鹏平台上等，目前平台上已上线课程 110 个以上，涵盖 Kunpeng DevKit、Kunpeng BoostKit、openEuler、openGauss、openLooKeng、HPC 等多个方面。

6.5.2 鲲鹏在线实验

鲲鹏在线实验：根据详细的实验手册，在鲲鹏社区提供的在线虚拟化平台当中，一步步完成指导操作，快速体验线上云服务，在云端实现云服务的实践、调测和验证。量级轻、上手快、体验好，实验操作界面如下图所示：

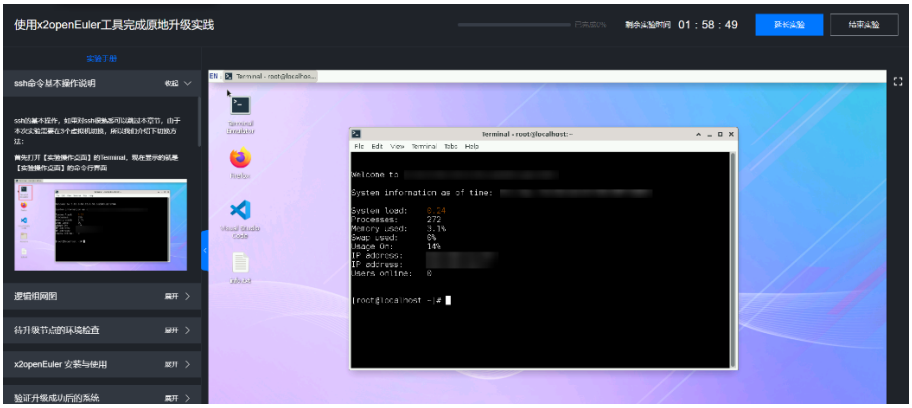


图6-5 鲲鹏在线实验界面

6.5.3 鲲鹏认证体系

鲲鹏认证体系是针对不同用户、产品类别，精心打造层次化的培训认证，提供职业认证和微认证。

鲲鹏应用开发者职业认证目前分为 HCIA 级别与 HCIP 级别，其中 HCIA 级别的认证面向能够在鲲鹏计算平台上进行业务应用的部署与迁移，性能测试与调优，以及具备处理应用在部署过程中遇到的常见问题的工程师；HCIP 级别的认证则是面向能够基于鲲鹏架构进行应用开发和高阶鲲鹏应用移植的高级工程师。

微认证则提供一站式在线学习、实验与考试，零基础也可学习前沿技术，快速获得场景化的技能提升，考取权威认证证书，微认证的认证流程为：在线学习->动手实验->购买认证->在线考试->获取证书，完成微认证即可享受相关权益：



图6-6 鲲鹏认证体系

6.6 合作伙伴 - 鲲鹏生态伙伴网络及伙伴计划

6.6.1 鲲鹏生态伙伴

鲲鹏生态伙伴网络（Kunpeng Eco-Partner Network，KEPN）是华为围绕鲲鹏产业技术路线推出的系列生态伙伴计划，它通过提供学习、构建、上市、销售等资源支持，致力于帮助鲲鹏生态伙伴构建产业竞争力、联接客户创造商机，在一个开放、创新、协作、互惠的生态系统里，共同成长、共创未来。

6.6.2 鲲鹏生态伙伴网络的优势

能力提升：获得生态伙伴专属的技术培训、管理培训，快速提升生态伙伴围绕鲲鹏全栈技术的产品与解决方案构建与创新能力，以及围绕企业成长的运营管理能力；

商业增长：获得联接华为客户与伙伴的机会，共享自主创新技术带来的行业数字化商业增长机会；

营销资源：获得参加华为与鲲鹏生态伙伴主办的系列市场营销活动、产业峰会、开发者技术沙龙、专题研讨会等；

产业人才：获得由全国优质合作高校精心培养的鲲鹏全栈产业技术人才，加速构建企业产品与解决方案竞争力；

6.6.3 鲲鹏生态伙伴类型

鲲鹏生态伙伴分为整机硬件伙伴、应用软件伙伴、基础软件伙伴、一体机解决方案伙伴、生态运营伙伴。申请加入时，生态伙伴可选择适合的类型，以获得对应的身份与支持。

6.6.4 鲲鹏生态伙伴网络成员级别

认证级：此等级适用所有身份伙伴。伙伴在完成身份注册，并达到认证级要求后，即可成为认证级伙伴，享受认证级学习培训权益；

优选级：此等级适用在行业或专有领域具有主流地位且对鲲鹏产业具有较高贡献的伙伴。他们在获得认证级伙伴权益基础上，还将获得更多学习培训资源，及数字化领导力培训、NRE 激励、MDF 激励、及受邀参加年度公司级旗舰活动等权益；

领先级：此等级适用在行业或专有领域具有主导地位且对鲲鹏产业具有卓越贡献的伙伴。他们在获得优选级伙伴权益基础上，还将获得更大额度的激励资源，及新品试用、路标共享与同步开发、伙伴专属能力构建支持服务等权益。

鲲鹏展翅伙伴计划：

鲲鹏展翅伙伴计划是华为围绕鲲鹏计算平台推出的一项生态伙伴计划，旨在帮助伙伴整合鲲鹏计算产业的全栈技术、品牌资源，构建产品与解决方案竞争力，联接客户与鲲鹏生态系统，创造无限商机。

欧拉扬帆伙伴计划：

欧拉扬帆伙伴计划是华为围绕 openEuler 推出的一项生态伙伴计划，旨在保障 openEuler 生态健康有序发展，加速生态伙伴基于 openEuler 系列产品和部件构建产品与解决方案，使能伙伴生态发展与商业成功。

鲲鹏科研创新使能计划：

鲲鹏科研创新使能计划是华为围绕鲲鹏计算基础设施推出的一项生态伙伴计划，旨在使能国内高校和科研院所依托鲲鹏计算基础设施开展科学技术研究和自主可信软件研发与技术攻关工作。

6.7 支持与资讯

鲲鹏论坛：

鲲鹏论坛是技术大拿、行业专家、热心开发者的交流圣地，各位名家高手坐而论道，共享实践经验、技术观点，目前拥有 12 大频道，涵盖鲲鹏生态覆盖的技术知识、活动等，帮助开发者快速拓展技术视野。

鲲鹏博客：

鲲鹏论坛是技术大拿、行业专家、热心开发者的交流圣地，各位名家高手坐而论道，共享实践经验、技术观点，目前拥有 12 大频道，涵盖鲲鹏生态覆盖的技术知识、活动等，帮助开发者快速拓展技术视野。

新闻资讯：

鲲鹏官方发布的最新动态或消息，为用户提供关于鲲鹏的第一手资讯。

开发者资料月刊：

开发者资料月刊定期汇总鲲鹏生态使能最新进展、精品热门文档、最新上线文档，集锦各技术领域热门案例，带领用户快速了解开发者资料最新动态。

支持与服务：

基于计算产品和解决方案，各领域专家使用专业工具，提供卓越的全流程专业服务，加速实现用户的业务价值；资深讲师面授，使能计算人才培养，助力企业数字化转型。